ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

A Contextualised Risk-Based Framework for Maven Dependency Updates

David Rodrigues Almeida

A dissertation submitted in partial fulfilment of the requirements for the degree of Master of Science, Specialisation Area of Cybersecurity and Systems Administration

Academic Supervisor: Isabel Azevedo

Porto, October 2026

Statement of Integrity

I hereby declare that I have conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore, the work presented in this document is original and authored by me, having not previously been used for any other end. The exceptions are explicitly recognised in the section “Ethical considerations” of the first chapter. This section also states how AI tools were used and for what purpose.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, October 17, 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor for her guidance, constructive feedback and support throughout this dissertation.

I am also grateful to my family for their patience, understanding and encouragement throughout this journey.

Abstract

Modern software systems rely heavily on dependencies distributed through Maven Central, where vulnerabilities may affect both direct and transitive dependencies. Conventional vulnerability reports often provide insufficient context about application-specific exposure and update compatibility. This can leave developers facing a binary choice: upgrade and risk breaking the build or postpone the update and retain a potentially vulnerable dependency.

This dissertation presents the Contextualised Risk-Based Framework (CRBF), implemented as a Maven plugin to support dependency update decisions. It combines vulnerability severity, exploitation likelihood, dependency staleness, reachability and upgrade-path information into contextual risk scores. Compatibility analysis informs estimated update effort, while a constraint-based optimiser selects updates within a configurable effort budget. The framework integrates vulnerability and ecosystem data with static call graph and compatibility analysis to produce contextualised reports and remediation recommendations.

The framework was evaluated on six open-source Maven projects against OSV-Scanner, with and without call analysis. The results demonstrated that CRBF provided more detailed reachability classifications and generated remediation plans within the configured effort budgets but required longer execution times.

Overall, the results demonstrate the feasibility of integrating contextual risk assessment and budget-constrained remediation into Maven dependency management.

Keywords: Maven, Dependency Management, Software Supply Chain Security, Reachability Analysis, Risk-Based Framework.

Resumo

Os sistemas de software modernos dependem de bibliotecas externas, frequentemente distribuídas através de repositórios como o Maven Central. Estas dependências podem introduzir vulnerabilidades, tanto diretamente como através de dependências transitivas. Embora ferramentas de análise de dependências identifiquem vulnerabilidades conhecidas, os seus relatórios nem sempre fornecem contexto suficiente para determinar se o código afetado é alcançável pela aplicação ou se uma atualização poderá introduzir incompatibilidades. Esta falta de informação pode deixar os programadores perante uma escolha entre atualizar e arriscar comprometer o funcionamento da aplicação, ou adiar a atualização e manter uma dependência potencialmente vulnerável.

Esta dissertação apresenta o *Contextualised Risk-Based Framework* (CRBF), implementado como um plugin Maven para apoiar decisões de atualização de dependências. O trabalho seguiu a metodologia *Design Science Research*, articulando a revisão de literatura com a conceção, implementação e avaliação da solução. A abordagem integra informação sobre vulnerabilidades, características do ecossistema e contexto específico do projeto, procurando conciliar a relevância das vulnerabilidades com o esforço necessário para as corrigir.

O CRBF obtém registos de vulnerabilidades através do OSV e complementa-os com probabilidades de exploração fornecidas pelo EPSS. A análise estática, realizada com WALA, permite construir um grafo de chamadas para avaliar a alcançabilidade do código potencialmente vulnerável. O JApiCmp identifica alterações de compatibilidade entre versões, enquanto o Goblin Weaver fornece métricas do ecossistema e informações sobre dependências transitivas. O modelo combina a severidade das vulnerabilidades, a probabilidade de exploração e a desatualização das dependências, ajustando o risco segundo a alcançabilidade e a qualidade do grafo de atualização proposto. A análise de compatibilidade contribui para estimar o esforço associado às atualizações propostas. Um modelo de otimização, resolvido com Z3, seleciona as atualizações que maximizam a redução de risco estimada dentro de um orçamento de esforço configurável.

A avaliação comparou o CRBF com o OSV-Scanner, com e sem análise de chamadas, em seis projetos Maven de código aberto. Os resultados demonstraram que o CRBF disponibiliza classificações de alcançabilidade mais detalhadas e produz planos de atualização que consideram o risco contextual e o esforço estimado, respeitando os orçamentos definidos. A integração destas análises implicou tempos de execução superiores aos do OSV-Scanner. A disponibilidade dos dados externos e as limitações da análise estática também condicionaram a informação produzida. Em geral, os resultados demonstraram a viabilidade de integrar a avaliação contextual de risco e planeamento de atualizações sujeitos a um orçamento de esforço na gestão de dependências Maven.

Palavras-chave: Maven, Gestão de Dependências, Segurança da Cadeia de Fornecimento de Software, Análise de Alcançabilidade, Framework Baseado em Risco.

Contents

1 Introduction 22

1.1 Context and Motivation 22

1.2 Problem Statement 23

1.3 Objectives and Research Questions 23

1.4 Research Methodology 24

1.5 Ethical Considerations 25

1.5.1 Use of AI Tools 27

1.6 Document Structure 27

2 Background 28

2.1 Maven Ecosystem and Dependency Management 28

2.1.1 Artifacts and Coordinates 28

2.1.2 Dependency Mechanism 29

2.1.3 Semantic Versioning and Breaking Changes 29

2.2 Software Supply Chain Security and Vulnerability Management 30

2.2.1 Software Bill of Materials and Software Composition Analysis 30

2.2.2 Risk-Based Vulnerability Management 31

2.2.3 Vulnerability Identification and Classification 31

2.2.4 Vulnerability Scoring 32

2.2.5 Vulnerability Databases and Aggregators 33

2.3 Risk Scoring and Decision Models 33

2.3.1 OWASP Risk Rating Methodology 33

2.3.2 Multi-Criteria Decision-Making Techniques 34

2.3.3 Runtime Calibration of Scoring Models 35

2.4 Ecosystem Stability and Compatibility 35

2.4.1 Technical Lag and Adoption Metrics 35

2.4.2 Compatibility Analysis 36

2.4.3 Ecosystem Graph Infrastructure 37

2.5 Contextual and Static Analysis 38

2.5.1 Static Application Security Testing 38

2.5.2 Static Analysis Frameworks for Java Bytecode 38

2.5.3 Formal Methods for Dependency Resolution 39

2.6 Evaluation Metrics 39

3 State of the Art 41

3.1 Research Question 1 42

3.1.1 Review Process 42

3.1.2 Findings and Discussion 44

3.1.3 Threats to Validity and Limitations 52

3.2 Research Question 2 52

3.2.1 Review Process 53
3.2.2 Findings and Discussion 54
3.2.3 Empirical Validation Strategies 59
3.2.4 Threats to Validity and Limitations 59

4 Analysis and Design 61

4.1 Requirements 61
4.1.1 Functional Requirements 61
4.1.2 Quality Requirements 64

4.2 Selection of tools 65
4.2.1 Vulnerability Data Source 65
4.2.2 Static Reachability Analysis 65
4.2.3 Compatibility Detection 65
4.2.4 Ecosystem Stability Metrics 66
4.2.5 Remediation Optimisation Engine 66

4.3 Architectural Design 66
4.3.1 System Context 66
4.3.2 Logical Organisation 67
4.3.3 Implementation View 69
4.3.4 Risk Scoring and Remediation Optimisation 70
4.3.5 Analysis Pipeline 77

5 Implementation 81

5.1 Development Environment and Technology Stack 81

5.2 Maven Plugin Integration and Pipeline Orchestration 81

5.3 Vulnerability and Reachability Analysis 82

5.4 Ecosystem Analysis and Upgrade Path Validation 83

5.5 Risk Scoring and Remediation Optimisation 84

5.6 Risk Report and VEX Generation 85

5.7 Testing and Requirements Verification 87

6 Evaluation 91

6.1 Evaluation Design 91

6.2 Experimental Setup and Dataset Selection 92

6.3 Results and Analysis 93
6.3.1 Impact of Java Call Analysis (C1 vs. C2) 93
6.3.2 Dependency Inventory and Vulnerability Detection Control 94
6.3.3 Contextual Reachability and Prioritisation (C2 vs. C3) 95
6.3.4 Remediation Recommendations and Validation 96
6.3.5 Execution Time Comparison 97

6.4 Threats to Validity 98

7 Conclusions 99

7.1 Achievements and Contributions 99
7.2 Difficulties 100
7.3 Future Work 100
7.4 Final Considerations 101

References 103

Appendix A 111

A.1 Identification of Required Skills 111
A.2 Assessment of Current Skills 113
A.3 Strategies for Skill Development 113
A.4 Evaluation and Control Mechanisms 114

Appendix B 115

B.1 Project Charter 115
B.2 Scope of Work 116
B.3 Integrated Project Schedule 121
B.4 Costs 125
B.5 Monitor and Control 125
B.6 Risk Identification and Management 125

List of Figures

Figure 1 - DSR process applied to this dissertation (adapted from [7]) 24
Figure 2 – PRISMA search process for RQ1 44
Figure 3 - PRISMA Search Process for RQ2 Adapted from [62] 55
Figure 4 - Use case diagram of the Contextualised Risk-Based Framework 62
Figure 5 - The CRBF and its external environment 67
Figure 6 - Logical organisation of the CRBF 68
Figure 7 - Implementation view of the reachability analysis port and adapter 69
Figure 8 – The CRBF analysis pipeline. 77
Figure 9 - Reachability classification flow. 78
Figure 10 - Overview and Remediation Plan in the generated risk report. 86
Figure 11 - Vulnerability analysis and API compatibility assessment in the generated report. 87
Figure 12 - WBS Project Deliveries 117
Figure 13 - Gantt Project Timeline - Part 1 121
Figure 14 - Gantt Project Timeline - Part 2 122
Figure 15 - Detailed project schedule. 123
Figure 16 - Gantt chart providing the graphical timeline representation of the integrated project schedule 124

List of Tables

Table 1 - Comparison of binary compatibility analysis tools along criteria relevant to programmatic integration. 37
Table 2 - Search terms for RQ1. 42
Table 3 - Search string applied to each database for RQ1. 43
Table 4 - Selected Studies for RQ1. 45
Table 5 - Search Terms for RQ2 53
Table 6 - Search string applied to each database for RQ2. 53
Table 7 - Selected Studies for RQ2. 56
Table 8 - Functional Requirements. 63
Table 9 - Quality Requirements. 64
Table 10 - Base risk score weights rationale. 71
Table 11 – Reachability weights. 73
Table 12 – Compatibility effort costs 75
Table 13 - Functional requirements testing. 88
Table 14 - Quality requirements testing. 89
Table 15 – Experimental configurations used in evaluation. 91
Table 16 - Project selection criteria. 92
Table 17 - Selected projects and their resolved dependency counts. 93
Table 18 - Vulnerability occurrences and call analysis results for C1 and C2. 94
Table 19 - Dependency inventories and vulnerability occurrences by input type. 94
Table 20 - Comparison of C2 call-analysis annotations and C3 reachability classifications. 95
Table 21 - Example of contextual vulnerability prioritisation in My-Blog. 96
Table 22 - Remediation decisions and estimated effort by project. 96
Table 23 - Execution times by project and configuration in seconds. 97
Table 24 - Required Skills. 112
Table 25 - WBS Dictionary 117

List of Codes

Code 1 - WALA configuration and call graph construction. 83
Code 2 - Z3 objective and constraint configuration for remediation optimisation. 84
Code 3 - ArchUnit test verifying application-layer independence from external analysis libraries. 89

List of Acronyms

AHP	Analytic Hierarchy Process
AI	Artificial Intelligence
API	Application Programming Interface
CG	Call Graph
CHA	Class Hierarchy Analysis
CPE	Common Platform Enumeration
CRA	Cyber Resilience Act
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DSR	Design Science Research
EC	Exclusion Criteria
EPSS	Exploit Prediction Scoring System
GDPR	General Data Protection Regulation
GHSA	GitHub Security Advisory
HTML	HyperText Markup Language
IC	Inclusion Criteria
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
JAR	Java Archive
JDK	Java Development Kit
MADM	Multi-Attribute Decision-Making
MCDM	Multi-Criteria Decision-Making
MODM	Multi-Objective Decision-Making
MTTR	Mean Time to Remediate

MVN	Maven Repository
MVP	Minimum Viable Product
NVD	National Vulnerability Database
ODC	OWASP Dependency-Check
OSI	Open Source Insights
OSV	Open Source Vulnerabilities
OWASP	Open Worldwide Application Security Project
POM	Project Object Model
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PRs	Pull Requests
RBVM	Risk-Based Vulnerability Management
RTA	Rapid Type Analysis
SAST	Static Application Security Testing
SAW	Simple Additive Weighting
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SDLC	Software Development Life Cycle
SMT	Satisfiability Modulo Theories
SRA	Static Reachability Analysis
TOPSIS	Technique for Order of Preference by Similarity to Ideal Solution
UML	Unified Modeling Language
VM	Vulnerability Management
WBS	Work Breakdown Structure

1 Introduction

This chapter contextualises the work described in this thesis, presents the context and motivation, defines the problem statement, and outlines the objectives. Additionally, it details the adopted research methodology and addresses the relevant ethical considerations guiding the work.

1.1 Context and Motivation

Modern software systems rely heavily on open-source dependencies, distributed via package managers such as Maven Central. These dependencies evolve rapidly, and vulnerabilities in widely used libraries can propagate across thousands of projects. Despite tools such as OWASP Dependency-Check (ODC) being effective at identifying known vulnerabilities in project dependencies, they operate strictly at the project level and provide no information about whether an upgrade is stable or widely adopted in the broader ecosystem [1].

On the other hand, ecosystem-level insights, including dependency prevalence, version adoption trends, and propagation chains, are also important, as not all vulnerabilities are equally relevant in all contexts. If a project relies exclusively on JSON, a critical vulnerability in the library´s XML deserialisation logic is irrelevant because it is not actually exploitable. Blindly updating in such cases can introduce unnecessary risks (e.g., instability, breaking changes) or wasted effort.

Thus, there is a need for a risk-based approach that combines vulnerability detection with ecosystem-wide stability metrics, while also considering project-specific usage contexts. Such an approach can guide updates that are both necessary (truly relevant vulnerabilities) and safe (ecosystem-proven stable).

The research focuses specifically on the Maven ecosystem for Java as it remains one of the most widely adopted programming languages in the industry [2], and Maven Central serves as the primary distribution infrastructure for Java artefacts [3], encompassing millions of releases and

packages. The longevity and immense scale of Java's open-source community have made third-party libraries foundational to modern enterprise software development. Because Maven automates complex dependency resolution through the Project Object Model (POM), vulnerabilities introduced within these packages can have a widespread, cascading impact across critical systems [1]. Consequently, addressing the security-stability trade-off within this specific ecosystem stands as a highly relevant necessity for the modern software industry.

1.2 Problem Statement

Updating dependencies can incur significant costs [1], particularly when migrating due to vulnerable dependencies, which is often considered "a practice that requires extra migration effort and added responsibility" [4]. Increased security often leads to higher operational risks due to the potential for breaking changes, which can manifest as compilation errors, test failures, dependency resolution failures, or lock failures [5].

In dependency management, security is directly linked to stability, where optimisation for one often negatively affects the other. Blindly upgrading to the latest version to fix a vulnerability can cause instability, while prioritising stability might leave the system vulnerable to security threats. Current approaches often force a binary choice: upgrade immediately and risk breaking the build or delay the upgrade and accept the security risk.

A recent study highlights that while standard tools identify vulnerable versions effectively, they fail to quantify the "stability confidence" of the proposed upgrade [6]. Additionally, ecosystem-level insights, such as adoption rates, are rarely integrated into decision-making processes.

Thus, there is a lack of integrated approaches that combine vulnerability detection with ecosystem-wide stability metrics while accounting for project-specific context. This gap highlights the urgency of developing a risk-based framework to address these issues.

1.3 Objectives and Research Questions

The objective of this research is to design, implement, and evaluate a risk-based framework to support dependency update decisions for Java projects in the Maven ecosystem. This solution aims to automate the integration of vulnerability data, ecosystem stability metrics, and project-specific usage context.

Four main activities were defined:

- Investigate the state of the art to establish a theoretical baseline.
- Design a novel risk-based architectural framework that synthesises multi-dimensional data (security, stability, and context) into actionable update recommendations.

- Implement a Minimum Viable Product (MVP) of the framework capable of parsing Maven dependency trees, calculating stability scores, and performing static reachability analysis.
- Evaluate the effectiveness of the developed solution through comparative experiments.

To achieve these objectives and validate the proposed solution, the research addresses the following research questions:

- RQ1: How can a novel risk-based framework be designed and implemented to automate dependency update decisions for Maven projects by integrating vulnerability data, ecosystem stability metrics, and project-specific usage context?
- RQ2: To what extent does the proposed framework's context-aware guidance improve the effectiveness of dependency management compared to non-contextual vulnerability reports?

RQ1 was addressed through the literature review and the design and implementation of the framework. RQ2 was investigated through a comparative experiment on open-source Maven projects using OSV-Scanner with and without call analysis as baselines. The evaluation examined dependency inventory coverage, reachability classifications, contextual vulnerability prioritisation, remediation recommendations, and execution time.

1.4 Research Methodology

The development of this work follows the Design Science Research (DSR) methodology [7] to achieve the objectives detailed in Section 1.3 and ensure a rigorous scientific process. DSR solves real-world problems by developing practical solutions, referred to as artefacts, while producing knowledge on how such solutions should be designed. This process comprises six activities, as illustrated in Figure 1.

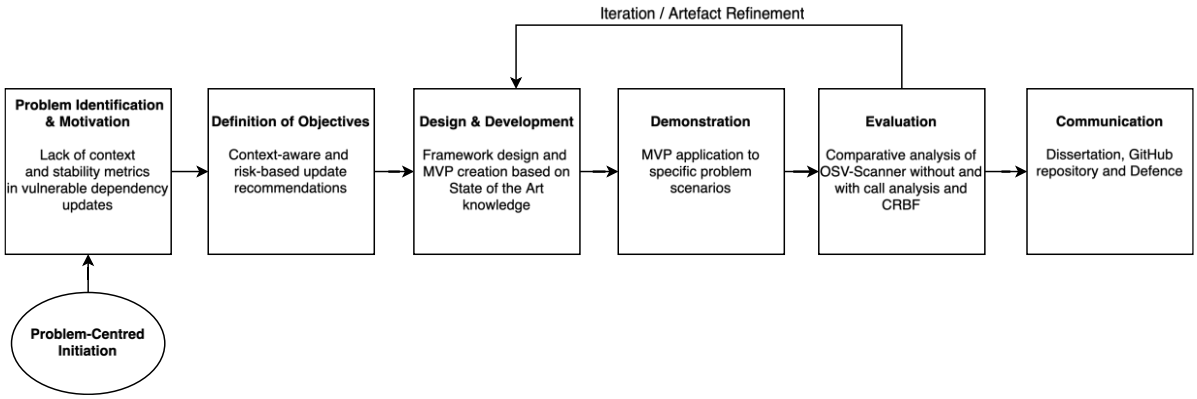


Figure 1 - DSR process applied to this dissertation (adapted from [7])

The following paragraphs describe each activity and how it was applied in this research [7]:

1. Problem Identification and Motivation: This stage details the research problem and justifies the value of a solution (detailed in Sections 1.1 and 1.2).
2. Definition of Objectives for a Solution: Establishes the framework's requirements and goals based on the problem definition, driving the design of the artefact (Section 1.3).
3. Design and Development: The artefact is created by determining its architecture and functionality. Development follows the architectural requirements identified in the State of the Art chapter. The project plan (Appendix B, WBS 1.2 and 1.3) covers tool selection and the design and implementation of a framework that combines security, stability, and contextual data to generate update recommendations.
4. Demonstration: The MVP built during Implementation is tested on a real project before the evaluation, to verify that the pipeline behaves as intended (Appendix B, WBS 1.3).
5. Evaluation: This phase evaluates how the developed artefact performs in comparison with existing tools. In the Experiment Realisation stage (Appendix B, WBS 1.5), the framework is evaluated across a sample of real-world open-source Maven projects using three configurations: OSV-Scanner without call-graph analysis, OSV-Scanner with call-graph analysis, and the proposed framework. The comparison between the first two configurations isolates the contribution of static reachability analysis, while the comparison with the proposed framework examines the additional contextual information provided to support dependency update decisions.
6. Communication: The final activity shares the research findings, the proposed framework, and its evaluation with the relevant stakeholders. This is accomplished through the present dissertation, the open-source release of the tool on a public GitHub repository, and the final academic defence (WBS 1.6), thereby ensuring transparency and enabling future work to build upon these results.

Although presented sequentially, these activities are iterative. Feedback from Evaluation informs subsequent design cycles, allowing the framework to be refined based on experimental results.

1.5 Ethical Considerations

This dissertation and its associated research activities were conducted in strict adherence to the ethical principles established by the IEEE/ACM Software Engineering Code of Ethics [8]. Furthermore, the work fully complies with the Code of Ethical Conduct of the Polytechnic Institute of Porto (P.PORTO) [9], ensuring integrity, responsibility, and transparency throughout the development process.

In accordance with Principle 3 of ACM/IEEE-CS, a comprehensive assessment of the technical skills required for the project was conducted (detailed in Appendix A). This process aimed to fulfil Clause 3.04: "ensure that they are qualified for any project on which they work by an

appropriate combination of education and training, and experience" [8]. Identified gaps were addressed through the learning plan defined in Appendix A.3 Strategies for Skill Development. Additionally, the work was subject to supervision and validation by academic advisors to guarantee technical accuracy, methodological rigour, and alignment with the high standards of the engineering profession.

The execution of the project also adhered to Principle 3 of the ACM/IEEE-CS, specifically to Clause 3.01, which states for "*high quality, acceptable cost and a reasonable schedule, ensuring significant tradeoffs are clear…*" [8]. This principle was applied in Project Planning (detailed in Appendix B) by explicitly defining the engineering trade-offs between the precision of the vulnerability analysis (quality) and the constraints of processing the Maven ecosystem within the academic semester (Schedule / Cost). These trade-offs were managed through specific risk mitigation strategies.

As stated in Clause 6.06 of the ACM/IEEE-CS, professionals must "Obey all laws governing their work…"[8]. Accordingly, this project strictly adheres to applicable regulations, including the General Data Protection Regulation (GDPR) [10]. The research primarily uses public data sources, specifically open-source software artefacts distributed via the Maven Central repository and public vulnerability metadata. Since the data is publicly available, the framework operates outside the scope of GDPR, avoiding the handling of personal or confidential information. Regarding security, any previously undisclosed vulnerabilities identified during this work would be reported privately to the maintainers to reduce the risk of exploitation.

Regarding the development of the software artefact, this project acknowledges the regulatory landscape defined by the European Cyber Resilience Act (CRA) [11]. The development of the framework aligns with the CRA essential requirements for products with digital elements, promoting security by design. It facilitates the timely remediation of vulnerabilities and supports the requirement for accurate vulnerability reporting.

The execution of this project aligns with Point 2.3 of the European Code of Conduct for Research Integrity [12], which states that "researchers report their results and methods, including the use of external services or AI and automated tools, in a way that is compatible with the accepted norms of the discipline and facilitates verification or replication". The thesis ensures transparency regarding the methodology, including the detailed documentation of AI tool usage (Section 1.5.1) and the publication of source code [13] and datasets. This approach guarantees technical accuracy and enables the academic community to verify and replicate the research findings.

To support visual accessibility, the hyperlinks included in the Analysis and Design and Implementation chapters are presented using both a distinct colour and underlining, ensuring that they remain identifiable without relying on colour alone, in accordance with *WCAG 2.2 Success Criterion 1.4.1 (Use of Color)* [14].

1.5.1 Use of AI Tools

Artificial Intelligence (AI) tools were used during the preparation of this thesis without compromising academic integrity. Tools such as Gemini (Google) assisted in improving textual clarity, correcting grammar, and structuring arguments. For some complex topics, they also helped synthesise previously acquired knowledge.

These tools were not used to produce the principal scientific contributions or conclusions of this dissertation. Moreover, all outputs generated by AI tools were subjected to rigorous human verification to ensure their accuracy, relevance, and consistency with the cited literature.

1.6 Document Structure

The remainder of this document is organised into six chapters and two appendices, providing a logical progression from the theoretical background to the implementation and evaluation of the proposed framework, followed by the conclusions.

The Background chapter establishes the fundamental concepts and terminology necessary to comprehend the research.

The State of the Art chapter presents an analysis of existing research on the topic to identify the architectural requirements for the proposed framework and the strategies to evaluate its effectiveness.

The Analysis and Design chapter presents the proposed framework, detailing its requirements, architectural decisions, and internal components.

The Implementation chapter describes how the framework was implemented in practice and the challenges encountered during development.

The Evaluation chapter presents the experimental comparison with OSV-Scanner, analyses the results, and discusses the threats to validity.

The Conclusions chapter summarises the achievements and contributions, discusses the difficulties encountered, outlines future work, and presents the final considerations.

Appendix A details the skills management strategy adopted during the dissertation, including the identification of required competencies, the plan established to address identified gaps, and the mechanisms in place to monitor skill development.

Appendix B presents the project planning artefacts that supported the execution of the dissertation, including the project charter, scope of work, integrated schedule, cost estimation, and risk management plan.

2 Background

This chapter outlines the fundamental concepts and terminology necessary to understand the State of the Art presented in Chapter 3. It first introduces the Maven ecosystem and its dependency management mechanisms (Section 2.1), followed by the main concepts used in software supply chain security and vulnerability management (Section 2.2). It then presents risk scoring and decision models (Section 2.3), ecosystem stability and compatibility concepts (Section 2.4), and contextual and static analysis techniques used to assess vulnerability reachability (Section 2.5). Finally, Section 2.6 introduces the metrics used to evaluate the effectiveness of dependency management approaches.

2.1 Maven Ecosystem and Dependency Management

Apache Maven [3] is a build automation and project management tool primarily used for Java-based applications. It relies on a Project Object Model (POM): an XML file that declares the project's configuration, build plugins, and dependencies. From this declaration, Maven standardises the build lifecycle across environments and automates dependency resolution by fetching the required external libraries from central repositories.

2.1.1 Artifacts and Coordinates

In the Maven ecosystem, a project or library is called an *artifact*[1]. Each artifact is uniquely addressed by the GAV coordinate system, composed of three mandatory fields [15]:

- **GroupId:** An identifier for the group that owns the project, typically a reversed domain name (e.g., `org.apache.maven`).

[1] This dissertation retains the American spelling *artifact* when referring to a Maven artifact, as this is the term used by Apache Maven [15]. Elsewhere, it uses the British spelling *artefact*.

- **ArtifactId:** The name of the artifact within that group, distinguishing it from other artifacts owned by the same group.
- **Version**: An identifier for a specific artifact release (e.g., `1.0.0`).

2.1.2 Dependency Mechanism

Maven manages external libraries through a dependency mechanism. These are classified into two categories:

- **Direct Dependencies:** Libraries explicitly declared in the project's POM.
- **Transitive Dependencies:** Libraries required by the direct dependencies, which Maven resolves recursively to build the full dependency tree.

At the ecosystem scale, these dependencies form a graph in which nodes represent packages and edges represent dependency relationships [16]. As this graph grows, it becomes increasingly difficult to track, a problem known as "dependency hell". Consequently, projects inherit extensive library networks that significantly expand the attack surface and allow vulnerabilities to propagate unnoticed [17]. This growth also leads to version conflicts when multiple versions of the same artifact are present in the tree. Maven resolves them through dependency mediation, using a "nearest-definition" strategy that selects the version closest to the project root. Although deterministic, this strategy can select a version that is incompatible with deeper dependencies or that contains known vulnerabilities [16].

2.1.3 Semantic Versioning and Breaking Changes

Maven follows the Semantic Versioning (SemVer) specification. Thus, a version number is structured as `MAJOR.MINOR.PATCH`, with each component indicating specific compatibility guarantees [16]:

- **MAJOR:** Incompatible API changes.
- **MINOR:** Backwards-compatible functionality additions.
- **PATCH:** Backwards-compatible bug fixes.

However, adherence to this specification is not strictly enforced across the ecosystem, and version numbers alone are unreliable indicators of stability. Bug fixes intended as patches frequently introduce breaking changes: updates that cause compilation failures or runtime errors [18]. Analysis of the Maven repository indicates that about one-third of all releases include at least one such breaking change [18]. This contributes to dependency staleness, in which developers delay updates to avoid the risk of unexpected failures.

2.2 Software Supply Chain Security and Vulnerability Management

The software supply chain comprises all systems, technologies, personnel, and processes involved in developing an application, at any stage of the Software Development Life Cycle (SDLC) [19]. Many of its elements are managed or distributed by third parties whose security posture cannot be directly verified. Securing the modern software supply chain requires analysing these third-party elements, as a vulnerability in any one of them can compromise the entire system [19].

This section describes how vulnerabilities in third-party components are found and prioritised. It first explains how the components of a project are inventoried (Section 2.2.1) and how risk-based management decides which vulnerabilities to remediate first (Section 2.2.2). It then presents the standards that identify, classify and score vulnerabilities (Sections 2.2.3 and 2.2.4), and the databases from which tools retrieve this data (Section 2.2.5).

2.2.1 Software Bill of Materials and Software Composition Analysis

Mitigating the risk posed by third-party components requires a comprehensive understanding of a system's composition. A Software Bill of Materials (SBOM) fulfils this need by providing a detailed inventory of every component, dependency, and hierarchical relationship within a piece of software [19]. Unlike a simple package list, an SBOM details version numbers, licensing data, and authorship for every direct and transitive dependency [20], allowing organisations to rapidly identify affected systems when a vulnerability is disclosed in a widely used library. To ensure cross-tool interoperability and compliance with government regulations, SBOMs adhere to standardised formats [19]. A leading international standard is CycloneDX [20], developed by the Open Worldwide Security Project (OWASP) to support application security and supply chain analysis.

Component inventories alone do not indicate whether a known vulnerability affecting a component is exploitable in a particular product. Vulnerability Exploitability eXchange (VEX) complements SBOM information by recording the status of the vulnerability for a specific product or component. The VEX minimum requirements define four possible statuses: not affected, affected, fixed, and under investigation. VEX is not tied to a single representation format and is implemented by specifications such as CSAF, OpenVEX and CycloneDX [21]. This allows exploitability information to be exchanged in a standardised, machine-readable format.

While an SBOM describes what a project contains, Software Composition Analysis (SCA) [22] is the process of producing and leveraging that inventory. SCA automates the identification and analysis of open-source components within a codebase, including direct and transitive dependencies, to evaluate their security, licence compliance, and code quality. SCA tools typically parse build manifests, such as Maven's `pom.xml`, to generate a complete dependency inventory, which is then mapped against known vulnerability databases to identify and assess risk.

2.2.2 Risk-Based Vulnerability Management

Traditional Vulnerability Management (VM) approaches typically focus on identifying and patching reported vulnerabilities based on technical severity scores alone. However, as the number of open-source vulnerabilities grows exponentially, this approach becomes unsustainable, and teams struggle to identify which ones pose a real threat [23].

Unlike traditional VM, Risk-Based Vulnerability Management (RBVM) prioritises remediation by contextualising vulnerabilities against the specific threat landscape and the asset's criticality. This approach focuses on reducing the likelihood of exploitation and the business impact rather than simply reducing the number of unpatched vulnerabilities [23].

In the context of software dependency management, RBVM relies on three dimensions to assess a component's actual risk [23]:

- **Threat Intelligence:** Whether a vulnerability is actively being exploited in the wild.
- **Vulnerability Reachability:** Whether the vulnerable code within a dependency is executed by the application, reducing false positives.
- **Contextual Impact:** How the role of a dependency within an application dictates the urgency of the fix.

Threat intelligence relies on standards for identifying, classifying, and scoring vulnerabilities, and on the databases that publish this information, which are presented in the remainder of this section.

2.2.3 Vulnerability Identification and Classification

The Common Vulnerabilities and Exposures (CVE) [24] is a standardised system of unique identifiers for publicly disclosed vulnerabilities, maintained by the MITRE Corporation. Each entry is assigned an identifier formatted as `CVE-YEAR-ID` (e.g., `CVE-2021-44228`, commonly known as Log4Shell), and includes a brief description, at least one public reference (such as a security advisory or vulnerability report), and the date the record was created. CVE serves as a foundational reference for vulnerability databases and SCA tools, enabling consistent communication across the security community.

However, this information is not always sufficient for dependency analysis. A CVE identifier may be reserved before its details are made public [25], creating a period in which tools that rely on published CVE data cannot detect the vulnerability. In addition, CVE records do not necessarily identify the vulnerable code itself: information about affected products and versions is required, whereas functions or methods are optional [25]. A record may therefore identify a vulnerable library version without showing which part of the library is affected.

While a CVE describes a specific instance of a security flaw, the Common Weakness Enumeration (CWE) [26] classifies its root causes, that is, the software defects that lead to them.

A weakness is a flaw in software architecture, design, or implementation that can be exploited to create a vulnerability. The relationship between weaknesses and vulnerabilities is many-to-many: a single weakness can be the root cause behind numerous CVEs, just as a single CVE can stem from more than one weakness. `CVE-2021-44228` (Log4Shell in Apache Log4j) illustrates the latter case, deriving from both `CWE-400` (Uncontrolled Resource Consumption) and `CWE-502` (Deserialisation of Untrusted Data). In the Maven ecosystem, these two weaknesses are among the most dangerous [17]. CWE complements CVE by describing the type of weakness behind a vulnerability, but it does not quantify its severity.

2.2.4 Vulnerability Scoring

The Common Vulnerability Scoring System (CVSS) [27] quantifies the severity of a vulnerability by assigning it a numerical score from 0.0 to 10.0. The score is derived from three metric groups [27]:

- **Base Metrics:** Represent the intrinsic qualities of a vulnerability (e.g., exploitability, impact on confidentiality), remaining constant over time and across different environments.
- **Threat Metrics:** Reflect attributes of a vulnerability that are subject to change, for instance, whether proof-of-concept exploits have been published or if the flaw is being actively exploited in the wild.
- **Environmental Metrics:** Capture characteristics unique to a specific user's environment, such as the relative importance of an affected system within the organisation's infrastructure.

CVSS v4.0 [28] also introduces a fourth group called Supplemental Metrics. This group captures additional context, notably safety implications in critical systems, without affecting the numerical score itself. However, only the Base Metrics group is assigned and distributed at scale by NIST [29]. Because these metrics remain constant over time, the CVSS Base Score describes how severe a vulnerability could be if exploited, but not how likely exploitation is. Only about 5% of known vulnerabilities are exploited in the wild, and CVSS-based prioritisation performs poorly at identifying them [29].

The Exploit Prediction Scoring System (EPSS) [30] addresses this limitation. EPSS assigns each published CVE a probability between 0 and 1 (0% to 100%) that the vulnerability will be exploited within the next 30 days, with scores updated daily as new information becomes available [30]. A vulnerability may therefore carry a high CVSS score but a low EPSS score, meaning that although the vulnerability is theoretically severe, it is not currently a target for attackers [30]. Together, CVSS and EPSS provide complementary measures of severity and exploitation likelihood that can support vulnerability prioritisation. EPSS only covers vulnerabilities with a published CVE record, so vulnerabilities that have not yet been publicly disclosed do not have a score [29].

2.2.5 Vulnerability Databases and Aggregators

CVE, CWE, CVSS, and EPSS define how vulnerabilities are identified, classified, and scored, but applying them in practice requires a source that stores this information in a way that tools can query programmatically.

The National Vulnerability Database (NVD) [31], maintained by NIST, serves as the standard repository for vulnerability information, enriching published CVE records with impact metrics (CVSS) and vulnerability types (CWE). However, it relies on the Common Platform Enumeration (CPE) framework rather than ecosystem-specific package identifiers. Therefore, it does not offer structured package-to-version mapping, meaning that correlating a CVE with a specific software artefact requires additional post-processing.

The GitHub Security Advisory (GHSA) [32] compiles and categorises security flaws within the open-source ecosystem, aggregating data from multiple sources and allowing project maintainers to report and fix vulnerabilities directly on the platform. Each advisory has a unique identifier, the GHSA ID, formatted as `GHSA-xxxx-xxxx-xxxx`, and is usually mapped to a corresponding CVE record and CVSS score [32]. GHSA provides ecosystem-specific advisories structured directly at the package level, which are accessible via the GitHub GraphQL API [33], though access requires authentication.

The Open Source Vulnerabilities (OSV) [34] database, maintained by Google, aggregates vulnerability data from GHSA, NVD, and other sources into a single service, enforcing a schema that remains consistent across all underlying data sources. OSV exposes a public API that requires no authentication and allows querying for vulnerabilities by ecosystem and package name, returning the corresponding affected version ranges in a structured format.

2.3 Risk Scoring and Decision Models

The metrics presented so far capture distinct aspects of a vulnerability, but no single one supports a remediation decision on its own. Two complementary domains address the problem of combining them: established security risk frameworks and the discipline of multi-criteria decision-making (MCDM).

2.3.1 OWASP Risk Rating Methodology

The OWASP Risk Rating Methodology [35] is a widely referenced framework for estimating the risk associated with a vulnerability in an application security context. It models risk as the product of likelihood and impact, where each term is itself a weighted average of qualitative sub-factors. Likelihood combines threat-agent attributes (skill, motive, opportunity) with vulnerability characteristics (ease of discovery, ease of exploit, awareness, intrusion detection). Impact combines technical consequences (loss of confidentiality, integrity, availability, accountability) with business consequences (financial damage, reputation damage, non-

compliance, privacy violation). Each sub-factor is scored on an ordinal scale, and the resulting risk is mapped to qualitative classifications (None, Low, Medium, High, Critical). Its original scope is for application-level threat modelling.

2.3.2 Multi-Criteria Decision-Making Techniques

MCDM is the discipline of decision theory concerned with selecting or ranking alternatives evaluated against multiple, often conflicting, criteria [36]. The field divides into two broad categories. Multi-objective decision-making (MODM) addresses continuous decision spaces with an infinite number of alternatives and is typically framed as an optimisation problem. Multi-attribute decision-making (MADM) addresses discrete decision spaces, ranking a finite set of explicitly alternatives against a fixed set of attributes [36]. Among the MADM techniques catalogued in recent surveys [36], the three methods detailed below serve as foundational representatives of distinct approaches to criteria aggregation.

The Simple Additive Weighting (SAW) [36], originally introduced by Churchman and Ackoff, computes the aggregate score of an alternative as the weighted sum of its normalised criterion values:

$$S = \sum_i w_i \times x_i \tag{1}$$

, where:

- S is the overall aggregate score of the evaluated alternative.
- w_i is the weight assigned to each individual criterion, reflecting its relative importance within the decision model.
- x_i is the normalised performance value of the alternative evaluated against that specific criterion.

The transparent nature of this method ensures that the contribution of each criterion to the final score is directly readable from the formula [36].

The Analytic Hierarchy Process (AHP) [36], developed by Saaty, decomposes the decision problem into a hierarchy and determines the relative importance of each criterion through pairwise comparisons. However, this method relies on subjective expert judgments to establish the weights, making it difficult to fully automate. Furthermore, it struggles to handle situations where criteria are interdependent, which can reduce the reliability of the final score.

Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) [36], authored by Hwang and Yoon, evaluates alternatives based on their distance to an ideal solution. It ranks options by measuring their proximity to a hypothetical positive ideal (constructed from the best values observed on every criterion) and a negative anti-ideal (constructed from the worst). Consequently, the output is a relative ranking rather than an absolute score.

The three techniques differ in their output formats and how weights are established: SAW calculates an absolute score from directly provided values; AHP derives its metrics from pairwise expert judgements over a hierarchy; and TOPSIS generates a relative ranking against a reference solution.

2.3.3 Runtime Calibration of Scoring Models

Weighted aggregation models such as the ones introduced in 2.3.2 typically fix the weights at design time and leave them unchanged during operation. This static configuration becomes fragile when new evidence about the evaluated criteria accumulates over time, or when the environment in which the model is deployed evolves in ways that were not anticipated during calibration. Self-adaptive systems address this fragility by embedding a feedback loop that adjusts internal parameters in response to observed outcomes.

The MAPE-K reference model, introduced by IBM in the context of Autonomic Computing [37], structures this feedback loop as four activities supported by a shared knowledge base. The Monitor activity collects data from the managed system; the Analyse activity interprets that data to identify divergences from expected behaviour; the Plan activity proposes adjustments; and the Execute activity applies them. The Knowledge component stores the information used throughout the loop. When applied to a scoring model, this process can be used to adjust the criterion weights according to observed outcomes. The weights must therefore remain configurable rather than being hard-coded constants.

2.4 Ecosystem Stability and Compatibility

A safe dependency update requires more than verifying the absence of known vulnerabilities: it must also be stable and compatible with the project that depends on it. This section examines three aspects of that evaluation: the metrics that quantify community adoption, the tools that detect binary incompatibilities, and the infrastructure that supports querying historical ecosystem data.

2.4.1 Technical Lag and Adoption Metrics

Open-source dependencies come with no guarantee that a new release is free of hidden defects or regressions. As a result, the behaviour of the broader ecosystem often serves as an indicator of a version's reliability [4], which is typically measured through technical lag and community adoption.

Technical lag represents the latency between the currently deployed version of a library and its latest available release, typically measured along two dimensions [38]:

- **Time Lag:** The number of days elapsed since the release of the current version, compared to the newest one.
- **Version Lag:** The number of releases (major, minor, or patch) missed between the deployed and available versions.

Community adoption complements technical lag by capturing how the wider ecosystem responds to new releases. Two complementary metrics are used to characterise this response [39]:

- **Adoption Lifespan:** The interval between the first and the last dependent adoption of a given version, indicating how long the release remains actively integrated in the ecosystem.
- **Adoption Reach:** The number of downstream dependents using a given version, indicating how widely it has been adopted across the ecosystem.

2.4.2 Compatibility Analysis

When a dependency is updated, semantic versioning alone does not guarantee that the new version is safe to integrate. As established in Section 2.1.3, even minor and patch releases can introduce breaking changes that violate the SemVer contract. Binary compatibility analysis detects such changes by comparing two compiled library versions, without requiring access to their source code.

A dependency update introduces a binary incompatibility when changes to the library's binary interface — such as removing a public method, changing a method signature, or deleting a class — cause code compiled against the previous version to fail when used with the new version [40]. Source incompatibility is different: it occurs when the dependent project no longer compiles against the new version. The two forms of incompatibility do not always coincide; a change can preserve source compatibility while breaking binary compatibility, and conversely, a source-incompatible change may preserve binary compatibility [40].

Several tools support automated binary compatibility analysis between Java Archive (JAR) files. Clirr [41] compares two JARs and reports removed or modified classes, methods, and fields, but has not been actively maintained since 2012 and lacks support for Java 9 and later. Revapi [42] is a modern alternative that supports binary, source, and semantic compatibility analysis and integrates into the Maven build lifecycle as a verification plugin. JApiCmp [43] is designed for programmatic, library-mode comparison: it accepts two JARs as input and returns a structured classification of each change as compatible, source-incompatible, or binary-incompatible. It has been adopted in ecosystem-level studies of breaking changes in Maven, including Jayasuriya et al. [44].

Table 1 summarises the three tools along three properties relevant to programmatic integration: whether the project is actively maintained, whether it supports modern Java releases, and how its API accommodates the comparison of externally resolved artefacts.

Table 1 - Comparison of binary compatibility analysis tools along criteria relevant to programmatic integration.

Tool	Maintained	Java 9+ support	Programmatic API for external JARs
Clirr [41]	No (inactive since 2012).	No	Yes.
Revapi [42]	Yes.	Yes.	Designed as a build-time plugin.
JApiCmp [43]	Yes	Yes	Yes

2.4.3 Ecosystem Graph Infrastructure

The community adoption metrics introduced in Section 2.4.1 – adoption lifespan and adoption reach – require ecosystem-wide historical data about releases and their dependents, together with timestamps that place adoption events in time. Ecosystem dependency graphs provide this information in a structure that can be queried programmatically.

A dependency graph represents artifacts, releases, and dependency relationships as nodes and edges. When release timestamps are included, the graph can be used to reconstruct the state of the ecosystem at different points in time and analyse how dependency adoption evolves across releases [45].

The Goblin framework [45] applies this dependency-graph representation for the Maven Central ecosystem through three components: a dependency graph metamodel (Goblin-DG), implemented as a Neo4j property graph; a miner (Goblin-Miner) that retrieves and incrementally updates the dependency graph from Maven Central; and an on-demand metrics weaver (Goblin-Weaver). The weaver exposes a REST API through which the graph can be queried and enriched with ecosystem-level metrics [45]. To avoid redundant computation, computed metrics are memoised as nodes in the graph and reused by subsequent queries [46].

Goblin-Weaver provides built-in indicators including freshness and release speed [45]. Freshness captures both the number of releases published after a given version and the time elapsed between that version and the most recent release [46], corresponding to the version-lag and time-lag dimensions introduced in Section 2.4.1. Release speed measures the average number of releases published per day by an artifact [45]. Although adoption lifespan and adoption reach are not built-in metrics, Weaver can be extended with additional metrics computed over the same dependency graph [45].

2.5 Contextual and Static Analysis

The mechanisms presented in Section 2.2 associate dependency versions with known vulnerabilities, but this association does not determine whether the affected code is reachable from the application. SCA tools that rely on version matching can therefore report vulnerabilities whose affected code is never reached, producing a high volume of false positives that developers must triage manually.

Contextual analysis addresses this gap by combining static analysis of the application's code with formal reasoning about its dependency structure. This section examines three aspects of that approach: the general principles of static analysis and reachability (Section 2.5.1), the frameworks that construct call graphs from Java bytecode (Section 2.5.2), and the formal methods that reason about dependency resolution as a constraint-satisfaction problem (Section 2.5.3).

2.5.1 Static Application Security Testing

Static Application Security Testing (SAST) [47] analyses source code or bytecode without executing the program. Within SAST, Static Reachability Analysis (SRA) evaluates whether a call path may exist from the application to a target method [47]. SRA relies on a Call Graph (CG), a directed graph in which nodes represent methods and edges represent possible method invocations [48]. When the graph includes both the application and its transitive dependencies, it can reveal whether vulnerable methods are reachable from the application. This is particularly relevant for transitive dependencies, where a vulnerable library may be present in the dependency tree even though no call path reaches the affected method [48].

2.5.2 Static Analysis Frameworks for Java Bytecode

Constructing a call graph from compiled Java bytecode requires a static analysis framework capable of resolving method invocations across an application and its transitive dependencies. This is challenging in Java because virtual method calls may target different implementations depending on the runtime type of an object. Since this runtime type cannot always be determined statically, call graph algorithms over-approximate the possible targets of each invocation to avoid missing feasible execution paths [49].

Call graph algorithms make this approximation with varying levels of precision and computational cost. Class Hierarchy Analysis (CHA) considers all compatible implementations in the class hierarchy, which can introduce call edges that are never exercised at runtime. Rapid Type Analysis (RTA) narrows this set by considering only types instantiated by the program [49].

Pointer analysis further narrows the set of possible call targets by estimating the objects to which individual variables may refer [49]. 0-CFA is a context-insensitive form of pointer analysis: a method is analysed without distinguishing among the call sites from which it is reached.

Context-sensitive variants analyse the same method under different calling contexts, potentially increasing precision at additional computational cost [50].

WALA [50] is a mature open-source static analysis framework developed at IBM Research. It supports Java bytecode and provides built-in call graph builders based on RTA, 0-CFA and context-sensitive variants. Its Java API allows call graphs to be constructed and queried directly in memory.

SootUp [49] is a rewrite of the Soot framework with a modular Java API that likewise builds and queries call graphs in memory. It provides CHA and RTA, and its extensible architecture permits custom virtual dispatch resolution strategies. More sophisticated pointer analyses are available through the integrated Qilin submodule [51].

Regardless of the framework or algorithm used, static call graphs approximate runtime behaviour. Runtime mechanisms such as reflection can introduce calls that static analysis does not capture, so the absence of a path does not guarantee that the target code is unreachable [52]. Call graph construction can also become computationally expensive as the analysis scope grows. WALA allows this scope to be restricted, for example by excluding selected classes or packages [50].

2.5.3 Formal Methods for Dependency Resolution

As detailed in Section 2.1.2, Maven's nearest-definition heuristic frequently fails to satisfy complex transitive constraints, leading to build failures or the selection of vulnerable versions. To overcome these limitations, advanced dependency management frames resolution as a constraint-satisfaction problem rather than a simple selection process [53].

Satisfiability Modulo Theories (SMT) [53] is a formal decision problem concerned with determining whether a set of logical constraints can be satisfied simultaneously. In dependency resolution, version requirements can be expressed as hard constraints over the dependency graph (e.g., Library A requires Library B > 2.0). Unlike local resolution heuristics, an SMT solver considers these constraints together to find a globally consistent assignment of dependency versions [53].

The Z3 Theorem Prover [54], developed by Microsoft Research, is an SMT solver that implements this form of constraint reasoning and also supports optimisation over weighted soft constraints. It is used by Depex [53], one of the constraint-based dependency management approaches discussed in Section 3.1.2.

2.6 Evaluation Metrics

To assess the effectiveness of vulnerability management tools, the industry relies on quantitative metrics that measure the accuracy and speed of remediation.

In the context of Static Analysis and SCA, precision measures the relevance of the findings, that is, the proportion of reported vulnerabilities that are actual threats. Recall measures the completeness of the analysis, the proportion of real vulnerabilities that the tool successfully identifies [55]. A low precision score indicates a high volume of false positives, which contributes to developer alert fatigue.

The F1-score is a single metric that combines Precision and Recall, providing a balanced measure of a tool's overall detection performance. An F1-score of 1.0 indicates perfect precision and recall, while lower values reflect deficiencies in either one or both metrics. This measure is particularly relevant for evaluating SCA tools, as it captures the trade-off between reporting all real vulnerabilities (Recall) and avoiding false alarms (Precision) [55].

Mean Time to Remediate (MTTR) tracks the average interval between the discovery of a vulnerability and the deployment of a fix. Empirical studies of dependency management tools use remediation time as a primary indicator of effectiveness, alongside merge rates and acceptance of automated update proposals [6].

3 State of the Art

To address the Research Questions (RQ1-RQ2), an investigation was conducted using the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) [56], a reporting standard designed to enhance transparency in systematic reviews. In accordance with this guideline, a systematic search was performed across established bibliographic databases, applying specific inclusion and exclusion criteria to screen the identified publications. The selected databases were: ScienceDirect [57], IEEE Xplore [58], ACM Digital Library [59], and b-on [60]. The research process followed:

- Identification: An initial search was performed in the chosen databases using a search string based on keywords relevant to the research area.
- Screening: The records from the initial search were reviewed through their titles and abstracts to filter out irrelevant studies. Duplicate entries across the various databases were also eliminated at this stage.
- Eligibility: The remaining articles were meticulously evaluated using a specific set of Inclusion Criteria (IC) and Exclusion Criteria (EC) to determine their relevance to the research questions.
- Included: The last set of papers that fulfilled all inclusion criteria was chosen for detailed analysis.

The primary literature search was conducted between October 2025 and March 2026. To address the rapid evolution of the field and capture the most recent advancements, a forward snowballing technique was applied to the selected primary studies up to May 2026. This process involved tracking citations to identify newer relevant literature.

This chapter presents the literature review conducted to address the research questions. The following sections describe the research process applied to each question, followed by a presentation and discussion of the findings.

3.1 Research Question 1

The goal of this research is to identify, through a systematic review of the literature, the technical requirements and architectural considerations necessary for a risk-based framework that supports dependency update decisions in Maven projects. The review examines how existing tools and research address vulnerability detection, update compatibility, ecosystem stability, and project-specific risk assessment, evaluating their capabilities and limitations. By establishing a theoretical and technical foundation, the review addresses the gap between isolated security scanning and the practical needs of developers who must balance security against stability and effort constraints when deciding whether to update a dependency.

3.1.1 Review Process

The search terms were constructed systematically to cover the project-specific usage context, ecosystem stability, vulnerability management, update compatibility, and effort components of RBVM. The goal was to include a broader scope of studies and tools addressing these topics individually and in combination. Table 2 lists the specific keywords defined for these scopes.

Table 2 - Search terms for RQ1.

Scope	Terms
Context	(Maven OR Java OR “Third-party libraries” OR “Open source software”)
Problem	(“Dependency update*” OR “Library migration” OR “Vulnerability management” OR “Vulnerability detection” OR “Breaking changes” OR “Update effort”)
Solution/Metrics	(“Risk assessment” OR “Software Composition Analysis” OR “Ecosystem stability” OR “Reachability Analysis”)

The keywords within each scope were combined with the OR operator, and the three scopes joined with the AND operator to form the full search string. As the complete search string could not be submitted through the ScienceDirect search interface, an adapted search string containing a reduced set of terms was used for that database:

(Maven OR Java) AND ("breaking changes" OR vulnerability OR dependency OR effort) AND (risk OR reachability OR update)

The query applied to each database is summarised in Table 3.

Table 3 - Search string applied to each database for RQ1.

Database	Search String
b-on	Complete search string.
IEEE Xplore	Complete search string.
ACM Digital Library	Complete search string.
ScienceDirect	Reduced search string.

To ensure a rigorous selection of relevant literature, specific inclusion and exclusion criteria were established. The inclusion criteria are as follows:

- Articles published from 2017 onwards.
- Studies addressing dependency management, vulnerability detection, or update decision-making in Maven/Java or directly applicable ecosystems.
- Research on reachability analysis, breaking changes, update effort/cost, risk assessment, or ecosystem stability metrics.
- Publications proposing frameworks, tools, concepts, or datasets supporting dependency updates, vulnerability remediation, or exploit prediction.
- Available in the English language.

The exclusion criteria are as follows:

- Articles without sufficient technical detail regarding ecosystem metrics, vulnerabilities, or implementation.
- Studies on vulnerability detection in source code without a dependency management context.
- Grey literature.

These criteria ensure that the reviewed literature provides insights relevant to the framework's design. The review prioritises the Maven/Java ecosystem given its scale and tooling maturity and extends to comparable ecosystems when findings are directly transferable. The temporal constraint of 2017 reflects the period when modern automated dependency management tools, such as *Dependabot* [61], became widely adopted, shifting the research focus from dependency management as a build problem to a security concern. Studies were included if they addressed at least one of the core topics of the research question. The exclusion criteria were designed to filter out literature lacking technical depth, research outside the problem domain, and grey literature. Duplicate studies were resolved by retaining the most recent or comprehensive version.

3.1.2 Findings and Discussion

For RQ1, the query retrieved 1195 records from the target databases. In the first processing step, 137 duplicate entries were removed. The remaining 1058 unique records underwent a preliminary screening based on the title and abstract, of which 985 were excluded for failing to meet the inclusion criteria. This resulted in 73 candidates for full-text assessment. After a review of the full content, 34 articles were excluded for insufficient technical depth, and a further 18 for lacking a dependency management context. Finally, 21 publications met all criteria and were selected for the review. Forward snowballing on these studies did not identify additional publications meeting the inclusion criteria within the defined period. The search process is illustrated in Figure 2.

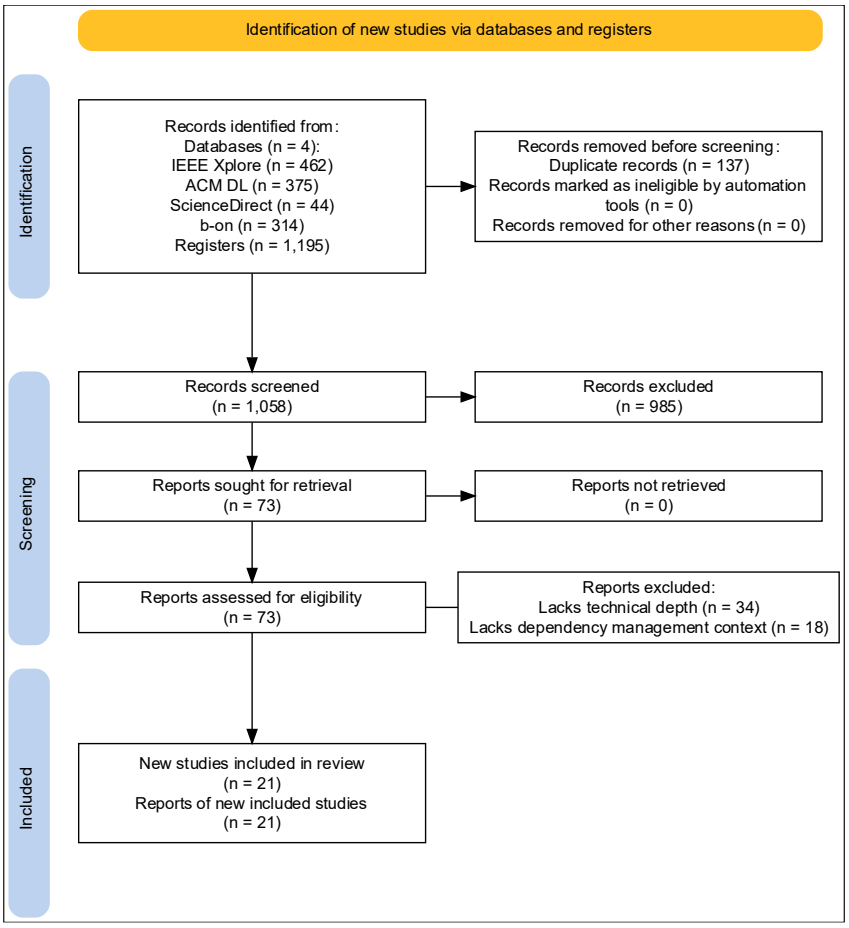


Figure 2 – PRISMA search process for RQ1
Adapted from [62]

The complete set of studies retained through this process is presented in Table 4.

Table 4 - Selected Studies for RQ1.

	Title	Reference	Database
1	Decoding Dependency Risks: A Quantitative Study of Vulnerabilities in the Maven Ecosystem	Nachuma et al. [17]	IEEE Xplore
2	Understanding the Threats of Upstream Vulnerabilities to Downstream Projects in the Maven Ecosystem	Wu et al. [63]	b-on
3	On the Security Blind Spots of Software Composition Analysis	Dietrich et al. [64]	ACM DL
4	VulNet: Towards improving vulnerability management in the maven ecosystem	Ma et al. [65]	b-on
5	Where is it? Tracing the Vulnerability-relevant Files from Vulnerability Reports	Sun et al [66]	ACM DL + IEEE Xplore + b-on
6	ReposVul: A Repository-Level High-Quality Vulnerability Dataset	Wang et al [67]	b-on
7	A manually-curated dataset of fixes to vulnerabilities of open-source software	Ponta et al. [68]	ACM DL + IEEE Xplore + b-on
8	Does the Vulnerability Threaten Our Projects? Automated Vulnerable API Detection for Third-Party Libraries	Zhang et al. [69]	IEEE Xplore + b-on
9	On the relation of method popularity to breaking changes in the Maven ecosystem	Keshani et al. [70]	ScienceDirect
10	CovSBOM: Enchancing Software Bill of Materials with Integrated Code Coverage Analysis	Zhao et al. [71]	IEEE Xplore
11	Depex: A Software for analysing and reasoning about vulnerabilities in software project dependencies	Márquez et al. [53]	b-on
12	Impact assessment of third-party library vulnerabilities through vulnerability reachability analysis	Jia et al. [72]	ACM DL
13	Dependency Update Adoption Patterns in the Maven Software Ecosystem	Berretta et al. [39]	IEEE Xplore
14	Semantic versioning and impact of breaking changes in the Maven repository	Raemaekers et al. [18]	ScienceDirect

	Title	Reference	Database
15	Goblin: A Framework for Enriching and Querying the Maven Central Dependency Graph	Jaime et al. [45]	IEEE Xplore
16	A Qualitative Study of Dependency Management and Its Security Implications	Pashchenko et al. [73]	ACM DL + IEEE Xplore + b-on
17	An extended study of syntactic breaking changes in the wild	Jayasuriya et al. [44]	ACM DL + b-on
18	Characterizing usages, updates and risks of third-party libraries in Java projects	Huang et al. [74]	ACM DL + IEEE Xplore + b-on
19	Compatible Remediation on Vulnerabilities from Third-Party Libraries for Java Projects	Zhang et al. [75]	ACM DL + IEEE Xplore + b-on
20	Enhancing Vulnerability Prioritization: Data-Driven Exploit Predictions with Community-Driven Insights	Jacobs et al. [29]	IEEE Xplore + b-on
21	Balancing the Quality and Cost of Updating Dependencies	Jaime et al. [1]	IEEE Xplore + b-on

The selected studies indicate that Maven dependency update decision-making is a multi-dimensional problem. Current practices often force a binary choice between security (upgrading to fix vulnerabilities) and stability (pinning versions to avoid breaking changes). Resolving this trade-off requires the framework to combine several complementary technical dimensions: deep graph resolution and vulnerable method identification to map the true exposure surface; contextual reachability analysis to filter irrelevant threats; and operational stability and update effort assessments to quantify practical risk and adaptation cost. These inputs must be aggregated into a risk score and interpreted by a decision engine capable of balancing security against stability under computational constraints.

Deep Graph Resolution

The first aspect identified in the literature is the need to model and traverse the full depth of the dependency graph, encompassing direct dependencies, all underlying transitive dependencies, and hidden dependency inclusions, such as shaded artefacts. Current tools frequently operate on a flat or superficial list of dependencies, an approach that systematically underestimates the scope of vulnerability exposure in the Maven ecosystem.

Nachuma et al. [17] conducted a quantitative study of over 14 million Maven releases, finding that while 31.39% of the latest releases are impacted by vulnerabilities in direct dependencies, a significantly higher proportion, specifically 62.89%, are affected by vulnerabilities present in transitive dependencies, a limitation corroborated by Wu et al. [63], who collected 44,450 instances of CVE-upstream-downstream relations and analysed approximately 50 million invocations across the Maven ecosystem. Their study, building upon the seminal findings of Zapata et al., revealed that up to 73.3% of projects depending on vulnerable libraries are actually safe, since the true exposure surface becomes visible only when the complete dependency graph is resolved. Standard approaches cannot distinguish these cases, inevitably

generating a high volume of false alerts and placing a significant cognitive burden on developers, which ultimately leads to alert fatigue.

The accuracy of dependency resolution is compromised by a second limitation known as Jar shading. In this practice, a dependency bytecode is embedded directly into the project artefact, bypassing standard POM declarations. Dietrich et al. [64] demonstrated that these shaded clones of vulnerable components are deployed in Maven Central but are not marked as vulnerable in standard databases, including the GHSA. Their study retrieved over 53,000 potentially vulnerable clones from Maven Central, confirming 727 vulnerable cases, and showed that existing SCA tools systematically miss these exposures. These findings support that complete graph resolution must operate at the bytecode level, instead of relying only on POM metadata.

A third limitation involves dependency scope awareness. Ma et al. [65] evaluated the Maven Repository (MVN) and Open Source Insights (OSI) and found that MVN omits dependencies detected by their tool *VulNet* in 9.5% of cases, while OSI omits them in 34.2%. A major reason for this gap is the failure to properly process Maven's dependency scope annotations. Vulnerabilities in test-scoped or provided-scoped dependencies carry a materially different risk profile compared to those in compile-scoped runtime dependencies. Therefore, accurate dependency exposure assessment requires not only resolving the entire transitive dependency tree, but also preserving scope annotations so that vulnerability data is propagated only for dependencies materialised in the deployed artefact.

Together, these findings identify three relevant considerations for dependency graph resolution: full transversal of transitive dependencies, bytecode-level analysis to detect shaded inclusions missing from POM metadata, and scope-aware processing to distinguish dependencies included in the deployed artefact.

Vulnerable Method Identification

Identifying vulnerable methods is critical because public vulnerability registries systematically lack method-level information. Standard vulnerability databases such as the NVD typically report only high-level metadata (e.g., the affected package or module name) without pointing to the exact location of the vulnerable code. Sun et al. [66] identified this omission as a critical gap that limits the effectiveness of downstream security tools, as static reachability analysis fundamentally requires a specific target to verify whether a vulnerability is exploitable within a given application.

Current research addresses this challenge through fix-commit analysis, extracting code diffs from version control systems to identify which methods were modified. Performing this extraction at runtime introduces another challenge, as developers often mix unrelated modifications, such as refactoring or feature additions, into the same commit. Wang et al. [67] demonstrated that relying on raw commit diffs without untangling these modifications introduces significant data noise, resulting in inaccurate method-level vulnerability mappings.

To overcome this challenge without the computational overhead of dynamic patch analysis, researchers have developed pre-computed knowledge bases. Ponta et al. [68] introduced an open-source vulnerability knowledge base for Eclipse Steady that maps CVEs to the specific source-code constructs of vulnerable methods, extracted from curated fix commits. However, Zhang et al. [69] identified a critical limitation in such tools. By restricting their extraction exclusively to the methods modified within patch commits, these tools frequently overlook hidden vulnerable root methods. If a patch introduces a new method to resolve a vulnerability, the original root methods that trigger the vulnerable execution path remain exploitable in unpatched versions, even though they were not directly modified in the fix commit.

To address this, Zhang et al. [69] proposed *VAScanner*, which applies a patch-unrelated sifting mechanism to isolate exact modifications and an augmentation mechanism to identify these hidden vulnerable root methods across library versions. Their evaluation demonstrated that *VAScanner* significantly outperforms *Eclipse Steady*, which exhibited 61.71% false negatives, while *VAScanner* yielded an accurate database of 90,749 vulnerable APIs with only a 1.45% false positive rate.

Together, these findings demonstrate that while method-level data is often missing from public registries, it can be recovered through different techniques, including querying pre-computed vulnerability knowledge bases when available. Method-level identification therefore provides a more precise basis for reachability analysis, while class or package-level detection remains necessary when such information is unavailable.

Contextual Reachability

Identifying a vulnerable dependency does not establish whether the affected code is reachable from the application. Keshani et al. [70] demonstrated that 87% of publicly documented methods in Maven libraries are never invoked by their dependents, indicating that package-level exposure alone provides limited information about application-specific risk. Reachability analysis addresses this limitation by determining whether a call path exists from the application's entry points to the vulnerable method across direct and transitive dependencies.

Zhao et al. [71] validated this distinction through *CovSBOM*, which embeds code coverage information into SBOMs as a proxy for runtime reachability. Their evaluation classified 105 of 145 reported vulnerabilities as false positives under this filter, corresponding to a 72% precision improvement over unfiltered package-level scanning.

Reachability analysis has also been integrated into dependency vulnerability scanners. OSV-Scanner v2 [76] extends package-level vulnerability detection with call analysis, allowing reported vulnerabilities to be separated based on whether the affected code is reachable from the analysed application. Java reachability analysis was added in version 2.2.2 for vulnerabilities affecting JAR files [76]. This capability allows OSV-Scanner to narrow package-level findings according to whether a call path to the affected code can be established. Among commercial scanners, Snyk [77] also provides reachability analysis for Java projects using Maven or Gradle, constructing a call graph to determine whether vulnerable functions are reachable.

However, the presence of a call path in the graph does not guarantee that the path can actually be exercised at runtime. A vulnerable method may appear reachable statically yet remain unreachable in practice because the conditions guarding the execution path may not be satisfiable at runtime. Wu et al. [63] found that up to 82.4% of paths to vulnerable functions were subject to constraints. Jia et al. [72] addressed this limitation in *CPVRA* by evaluating path constraints alongside call-graph reachability, reducing false alarms by 40.9% compared with conventional reachability tools such as *dep-scan*.

These findings collectively establish that determining whether a vulnerability poses a genuine risk to an application requires more than confirming its presence in the dependency graph. Method-level reachability provides application-specific evidence of whether vulnerable code can be reached, while path-constraint analysis further refines this assessment by considering whether the identified execution path is feasible at runtime.

Operational Stability

To address the tension between security and stability, the reviewed literature highlights the need to assess the likelihood that a given update will introduce a breaking change and warns against relying solely on SemVer. Raemaekers et al. [18] analysed over 100,000 jar files and found that 35.7% of minor releases and 23.8% of patch releases introduced binary incompatibilities, violating SemVer principles. Consequently, objective, ecosystem-level metrics are required to estimate upgrade safety. To extract these historical stability metrics, the literature demonstrates the utility of time-aware graph databases, such as the Goblin Framework [45], which can query the dependency graph at any point in the past to calculate temporal stability indicators.

While common assumptions associate the popularity of libraries with stability, Keshani et al. [70] empirically demonstrated the absence of this correlation, indicating that popular methods experience breaking changes as frequently as less popular ones. Instead of focusing on raw popularity, research points to adoption patterns as a more reliable indicator. Berretta et al. [39] demonstrated that adoption lifespan follows a log-normal distribution and correlates with semantic change size and maintenance practices. Specifically, releases with larger semantic changes have longer adoption lifespans, while highly maintained packages tend to have shorter ones, serving as strong indicators of stability.

Stringer et al. [38] show that technical lag — the accumulated time a dependency has remained outdated relative to newer available versions — is a measurable indicator of dependency management practices across major package managers. A high technical lag therefore signals a heightened likelihood of unaddressed security exposure, even when no CVE has been explicitly reported for the version in use.

Adoption patterns reflect long-term ecosystem trust, while temporal freshness captures the latent risk of inaction, and neither can be derived from the version number alone. Semantic versioning and library popularity are insufficient predictors of stability.

Update Effort

Knowing that a vulnerability is reachable and that a patched version exists does not resolve whether applying the update is practical. Pashchenko et al. [73] found that when effort is perceived as high, developers frequently opt to remain with the vulnerability rather than apply the fix. This pattern was observed consistently across multiple interviews, regardless of vulnerability severity. Effort estimation is therefore not merely useful contextual information but a decisive factor in whether a security update is adopted at all.

The complexity of a dependency update manifests in two ways: the likelihood that the new version breaks existing client code, and the extent to which the client's codebase must be adapted to accommodate the change. Jayasuriya et al. [44] analysed 18,415 Maven artefacts and found that 11.58% of dependency updates contained syntactic breaking changes affecting the client project, and that 71.60% of the studied dependencies were not up to date at the time of analysis. Almost half of the detected breaking changes violated semantic versioning by appearing in non-major version increments, confirming that the version number alone is insufficient to predict migration complexity [44]. The extent of the required adaptation is captured more precisely by the API surface delta between current and target versions, specifically, the number of public API methods deleted or modified in the target version that the client project currently invokes. Huang et al. [74] operationalised this in LIBSECURIFY, demonstrating that this metric directly captures how many call sites in the application code would require adaptation, making it a project-specific effort estimate rather than a library-level abstraction. The authors acknowledge, however, that these metrics represent a partial measure of updating effort, as the cognitive cost of adapting client code to API changes and the efforts of running test suites remain difficult to quantify automatically [74].

A further dimension specific to vulnerability remediation is the availability of a secure and non-breaking update path. In a large-scale study of 2,000 Maven projects, Zhang et al. [75] found that while 78.45% of vulnerabilities could be resolved via non-breaking updates, the remaining 21.55% required breaking changes or lacked a fix entirely. Without automated remediation support, developers could directly fix only 25.71% of vulnerabilities through non-major dependency updates. These constraints dictate that effort estimation must include a fixability classification – categorising issues into compatible fixes, breaking changes, and unresolvable cases – since each scenario fundamentally alters the update decision. This necessity is supported by the fact that over half (51.31%) of automated remediation pull requests on GitHub fail specifically due to incompatibility concerns [75].

Risk Score Formulation

Current vulnerability management tools present dependency risks but lack effective prioritisation mechanisms. In their evaluation of two vulnerability platforms (MVN and OSI), Ma et al. [65] found that neither could rank vulnerabilities based on their actual threat to a specific project. This limitation is attributed to the absence of project-specific context. Addressing it requires a composite risk score that aggregates multiple contextual signals to effectively guide update decisions.

A widely adopted prioritisation signal, CVSS, captures the theoretical severity of a vulnerability but not the likelihood of exploitation. Jacobs et al. [29] demonstrated that only 5% of known vulnerabilities are ever exploited in the wild, yet CVSS-based strategies achieve an efficiency of just 6.5% when prioritising for real-world exploitation risk. By contrast, EPSS v3, a machine learning model trained on observed exploitation data, addresses this gap by achieving 78.5% efficiency at its respective optimal threshold, an overall 82% improvement in classifier performance over its predecessor [29]. These findings demonstrate that threat severity and exploitation likelihood are distinct but complementary signals, and that relying only on CVSS leaves the likelihood of exploitation unaddressed.

Beyond the threat itself, the risk posed by a vulnerability is further shaped by whether the vulnerable code is reachable in the application and by the operational cost of the candidate update. Existing approaches consider these dimensions to different extents, but the reviewed literature does not identify an approach that combines them within a single prioritisation mechanism.

A further gap concerns the cost of remediation. The evaluation of LibSecurify by Huang et al. [74] showed that 24 of 33 alerted projects subsequently updated their vulnerable dependencies after receiving contextual information about both security risk and update effort. This result provides empirical evidence that combining security and update-effort information can support more informed dependency update decisions.

Finally, the reviewed approaches show that risk scoring often relies on weighted combinations of predefined signals. While this supports interpretability, fixed weighting schemes may limit the ability to represent more complex interactions between contextual factors or to adapt to different project characteristics.

Decision Engine

Having established the individual metrics for threat, reachability, stability, and effort, the framework requires a mechanism to synthesise these signals into actionable update recommendations. Standard Maven dependency resolution relies on a *nearest-wins* strategy to handle version conflicts [16]. This mechanism can be efficient. However, it does not guarantee that the chosen versions satisfy the constraints of other transitive dependencies, often leading to runtime errors or the selection of vulnerable versions.

Márquez et al. [53] addressed this by encoding the entire dependency graph and its version constraints as an SMT problem, solved via the Z3 solver. This transforms dependency management from a simple selection process into a constraint-satisfaction problem, enabling the solver to pinpoint mutually compatible versions that satisfy all semantic rules. The efficacy of this method is evident in their tool, *Depex*, which identified 152% more vulnerabilities than *GitHub Dependabot* and 204% more than *Snyk*, showing that constraint-aware selection uncovers risks missed by scanners that do not consider version resolution [53].

A critical limitation of static SMT formulations is their assumption of a fixed dependency graph. As Zhang et al. [75] showed with *CORAL*, updating a single library alters the graph structure,

potentially shifting the version constraints and vulnerability exposure of other dependencies. To address this phenomenon – termed *ripple effects* – their approach evaluates the affected subgraphs to capture these cascading changes.

Jaime et al. [1] demonstrated with *GoblinUpdater* that encoding the dependency update problem as a multi-objective linear program produces updates significantly better than naive approaches while maintaining compilation correctness. The tool allows developers to explicitly balance multiple quality metrics – freshness, popularity, and a vulnerability score – against the adaptation cost, measured through API breaking changes. This flexibility enables developers to reflect their own priorities in the update decision. However, the vulnerability signal is still derived at package level, using CVE counts and CVSS scores without considering whether the vulnerable code is reachable in the analysed project.

Execution Strategy

The components described in the reviewed literature exhibit different computational profiles. A vulnerability check against an advisory database completes in milliseconds, while constructing a complete call graph across all transitive dependencies can incur substantial execution times, depending on the project's complexity. Applying all the components uniformly on every build would therefore impose an unacceptable overhead. The definition of an optimised execution strategy – determining which components should run on each build, on a scheduled basis, or only upon specific events such as a manifest change or a new vulnerability disclosure – is an important practical concern. However, addressing this falls outside the scope of the current investigation and is identified as a direction for future work.

3.1.3 Threats to Validity and Limitations

A first threat concerns the adaptation of the search string for ScienceDirect. As the complete search string could not be submitted through the platform's search interface, a reduced search string was used for this database. This adaptation removed or generalised terms from the complete search string, including "Reachability Analysis", "Ecosystem Stability", and "Software Composition Analysis", and may therefore have excluded relevant studies. However, the complete search string was successfully executed on IEEE Xplore, ACM Digital Library, and the b-on aggregator, whose coverage overlaps with content from major publishers in this domain and therefore provides additional coverage beyond ScienceDirect.

A second threat concerns language. The review considered only English-language publications, as multilingual coverage was beyond the scope of this work. Consequently, relevant contributions published solely in other languages may be absent from the selected dataset.

3.2 Research Question 2

While RQ1 focused on the framework's architectural design, RQ2 focuses on its empirical validation. It investigates the extent to which context-aware guidance improves the

effectiveness of dependency management compared to non-contextual vulnerability reports. To address this question, the systematic review identified empirical studies and established metrics used to evaluate dependency management tools, with particular attention to research examining how contextual information influences developer decision-making and tool adoption.

3.2.1 Review Process

The search strategy was designed to capture empirical studies and comparative evaluations of dependency management tools, approaches, and developer practices, prioritising research that provides concrete data on tool performance and developer behaviour. To ensure consistency, the same databases used for RQ1 were queried: ScienceDirect, IEEE Xplore, ACM Digital Library, and b-on.

To comprehensively evaluate the effectiveness of context-aware approaches, the search query was constructed using Boolean logic to combine keywords across three core categories: Domain, Outcome/Metrics, and Methodology. By exploring a broader conceptual space focused on observable outcomes and decision-making, the query encompasses specific quantitative indicators, such as false-positive and merge rates, alongside analytical concepts such as reachability-based vulnerability assessment. The key terms are detailed in Table 5.

Table 5 - Search Terms for RQ2.

Scope	Terms
Domain	("vulnerable dependenc*" OR "dependency update*" OR "dependency management")
Outcome / Metrics	("merge rate" OR "time-to-merge" OR "false positive*" OR "alert fatigue" OR "developer responsiveness" OR "reachability" OR "vulnerability detection")
Methodology	("empirical" OR "case study" OR "comparative analysis" OR "qualitative")

The keywords were combined into the full query by joining the three scopes with the AND operator. As in RQ1, a reduced search string was used for ScienceDirect:

("vulnerable dependencies" OR "dependency updates" OR "dependency management") AND ("false positives" OR "reachability") AND ("empirical" OR "case study" OR "qualitative")

Table 6 summarises the query submitted to each database.

Table 6 - Search string applied to each database for RQ2.

Database	Search String
b-on	Complete search string.
IEEE Xplore	Complete search string.

ACM Digital Library	Complete search string.
ScienceDirect	Reduced search string.

Justification for the specific terms retained and removed from the reduced ScienceDirect query is presented in the Threats to Validity and Limitations section.

Following the same methodology applied in RQ1, inclusion and exclusion criteria were established and applied to select relevant studies. The inclusion criteria are as follows:

- Articles published from 2017 onwards.
- Empirical studies that evaluate, compare, or examine dependency management tools, approaches, or developer practices in the context of vulnerability remediation or dependency updates.
- Studies that evaluate the effectiveness of tools using quantitative metrics or qualitative assessments.

The exclusion criteria are:

- Publications not available in English.
- Purely theoretical or lacking an empirical evaluation.
- Address unrelated domains.
- Studies focused exclusively on vulnerability discovery without evaluating remediation or update outcomes.

3.2.2 Findings and Discussion

For RQ2, the search query retrieved a total of 305 records across the four databases. In the first processing step, 79 duplicate entries were identified and removed. The remaining 226 unique records were excluded for failing to meet the inclusion criteria. This resulted in 28 candidates for full-text assessment, of which 10 met all criteria and were selected for review. Forward snowballing on these studies did not identify additional publications meeting the inclusion criteria within the defined period. The search process is illustrated in Figure 3.

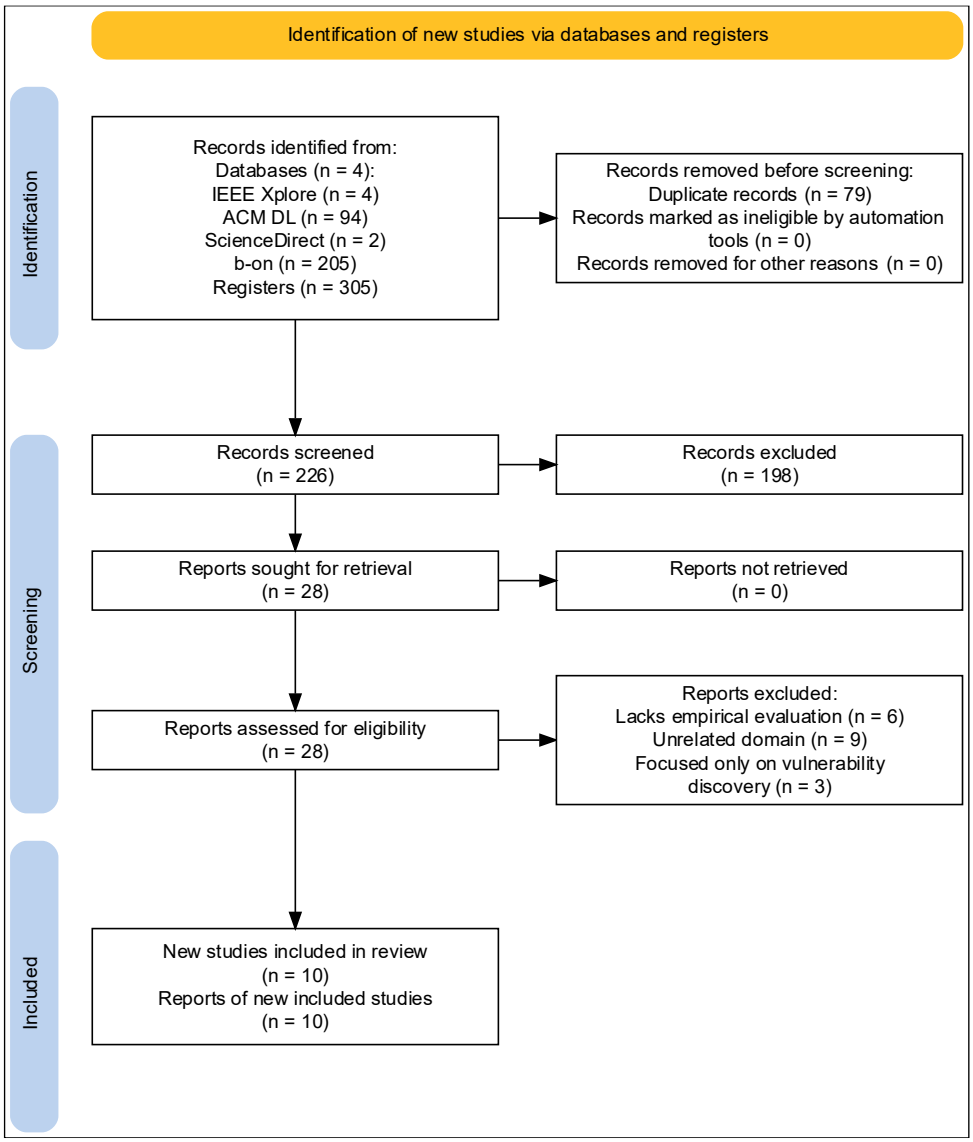


Figure 3 - PRISMA Search Process for RQ2
Adapted from [62]

The complete set of studies retained through this process is presented in Table 7.

Table 7 - Selected Studies for RQ2.

ID	Title	Reference	Database
1	Detection, assessment and mitigation of vulnerabilities in open-source dependencies	Ponta et al. [78]	ACM DL, b-on
2	Vulnerable open source dependencies: counting those that matter	Pashchenko et al. [79]	ACM DL, b-on
3	Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies	Pashchenko et al. [80]	IEEE Xplore, ACM DL, b-on
4	Software Composition Analysis for Vulnerability Detection: An Empirical Study on Java Projects	Zhao et al. [55]	ACM DL, b-on
5	Do Developers Update Their Library Dependencies? An Empirical Study on the Impact of Security Advisories on Library Migration	Kula et al. [4]	ACM DL, b-on
6	Dependabot and security pull requests: large empirical study	Rebatchi et al. [6]	ACM DL, b-on
7	Securing dependencies: A comprehensive study of Dependabot's impact on vulnerability mitigation	Mohayeji et al. [81]	ACM DL, b-on
8	Upgradvisor: Early Adopting Dependency Updates Using Hybrid Program Analysis and Hardware Tracing	David et al. [82]	b-on
9	A Qualitative Study of Dependency Management and Its Security Implications	Pashchenko et al. [73]	ACM DL, b-on
10	Characterizing usages, updates and risks of third-party libraries in Java projects	Huang et al. [74]	b-on

The systematic review of the selected literature reveals consistent empirical evidence supporting the hypothesis that context-aware approaches improve the effectiveness of dependency management compared to non-contextual vulnerability reports. The findings are organised into three categories: the impact of contextual analysis on false positive reduction, developer behaviour and tool adoption, and direct empirical evidence of context-aware effectiveness.

False Positive Reduction

A recurring finding across the reviewed studies is that non-contextual tools generate a substantial number of false positives, thereby reducing their practical value. Ponta et al. [78] conducted a direct comparison between Eclipse Steady, a code-centric tool that analyses whether vulnerable code is actually executed, and OWASP Dependency-Check, a non-contextual vulnerability scanner. Their evaluation on a dataset of industrial Java applications revealed that 88.8% of the findings reported by OWASP Dependency-Check were false positives. This result provides empirical evidence that context-aware analysis, specifically usage-based reachability, significantly reduces noise in vulnerability reports.

This finding is reinforced by two complementary studies from the same research group. Pashchenko et al. [79] proposed a precise counting methodology that distinguishes deployed from non-deployed dependencies. Their empirical study on 10,905 library instances of the 200

most popular Java libraries used by SAP revealed that 20% of the dependencies affected by a known vulnerability are not deployed and, therefore, do not represent a risk to the analysed applications. In a subsequent study, the same authors proposed the Vuln4Real methodology [80], which determines whether a dependency is vulnerable through code-level matching rather than conventional approaches. Their evaluation demonstrated that incorporating dependency-resolution context and usage analysis significantly reduced the number of actionable vulnerabilities compared to those reported by non-contextual tools. Together, these findings indicate that non-contextual reporting induces alert fatigue, leading to an inefficient allocation of development and audit resources.

Complementing these findings, Zhao et al. [55] evaluated six SCA tools on a large-scale dataset of 21,130 Maven modules with 73,499 unique dependencies and found that the average F1-score for vulnerability detection accuracy was only 0.475. However, they demonstrated that proper support for dependency-resolution semantics reduced false positives in vulnerability reports by 18.28% and false negatives by 8.72%, confirming that even incremental improvements in contextual awareness led to measurable gains in detection accuracy.

Developer Behaviour and Tool Adoption

A second group of evidence examines developer responses to automated update recommendations, revealing the practical limitations of current non-contextual tools. Kula et al. [4] conducted a large-scale empirical study across 4,600 GitHub projects, finding that 81.5% of projects retained outdated dependencies and that 69% of surveyed developers were unaware of security advisories affecting their codebase. This finding establishes that non-contextual vulnerability reports, even when available, frequently fail to reach or motivate developers.

In another study, Rebatchi et al. [6] analysed 9.9 million pull request issues across 1.7 million projects, revealing that while Dependabot dominates dependency management activity, only 25% of its security pull requests (PRs) are ultimately merged. Although most closed PRs were replaced by newer updates rather than explicitly rejected, a significant portion (31%) remained open and unresolved. The authors identified several factors correlating with PR acceptance: project activity level, vulnerability severity, the nature of the proposed version bump, and the extent of the introduced code changes. A qualitative survey conducted as part of the study revealed that 17% of developers rejected security PRs due to insufficient descriptive information, with one participant noting that a CVSS score alone is insufficient to justify a fix. Furthermore, 78% of developers left security PRs unaddressed due to low priority, and 50% perceived the impact of retaining the vulnerable version as low severity. These findings suggest that developers are implicitly performing a cost-benefit analysis without the contextual information needed to do so accurately, a limitation that conventional dependency-management tools do not consistently address.

Mohayeji et al. [81] complemented these findings with an analysis of 4,195 security updates across 978 JavaScript projects and found that 57% of bot-initiated security updates were merged. Their analysis of the vulnerability lifecycle revealed that the type of version bump (major, minor, or patch) strongly influenced developer decisions. Major version bumps, which

carry a higher risk of breaking changes, were less likely to be merged. This suggests that developers instinctively weigh the adoption cost against the security benefit, even without explicit guidance from tools.

The consequences of this friction were captured in a study by David et al. [82]. They noted that the typical delay in applying dependency updates across Python-related ecosystems increased from around 20 days in 2019 to over 200 days in 2021, showing that increasing the volume of security alerts does not accelerate remediation efforts.

To understand why this growing volume of alerts fails to drive decision updates, Pashchenko et al. [73] conducted 25 semi-structured interviews with developers, identifying the main barriers to effective dependency management. Developers reported that irrelevant or low-priority alerts are the main barriers to tool adoption. The volume of false positives and low-level warnings led some developers to abandon dependency analysis tools altogether and return to manual review. On encountering a vulnerability, developers reported that their immediate concern was whether it actually affected their project and how much effort a fix would require, two questions the available tooling did not address. Developers explicitly recommended that tools should report only vulnerabilities that affect the analysed project, identify the specific components exposed to the vulnerability, and indicate whether a fixed version introduces breaking changes. The findings suggest that developers do not reject updates because they are unaware of vulnerabilities, but because the tools available do not give them enough information to act.

Context-Aware Effectiveness

The literature reviewed in the previous sections consistently demonstrated the limitations of non-contextual dependency management tools. Huang et al. [74] provided empirical evidence that contextual information can mitigate some of these limitations. Their large-scale study on 806 open-source Java projects and 13,565 third-party libraries quantified both the prevalence of outdated dependencies and the risks associated with their use and update, finding that developers frequently rely on outdated library versions while unaware of the security bugs they introduce.

Building on these findings, the authors proposed *LIBSECURIFY* [74], an alerting system for Java that flags only security bugs reachable through the project's execution paths, directly addressing the false positive problem identified across the reviewed literature. Beyond reachability, the tool provides developers with contextual information, specifically, the number of library API calls affected by security bugs (risk quantification) and the number of API calls deleted or modified in the target version (effort quantification). Together, these signals give developers the information needed to assess both the risk of staying on the current version and the cost of updating.

To validate their approach, Huang et al. [74] applied *LIBSECURIFY* to 33 open-source Java projects that confirmed the presence of security bugs after receiving contextualised alerts. Of these, 24 (72.7%) subsequently updated their vulnerable dependencies. These findings provide

empirical evidence that contextual information can influence how developers respond to vulnerable reports.

3.2.3 Empirical Validation Strategies

Based on the reviewed literature, the empirical validation of dependency management tools typically relies on three complementary strategies to evaluate effectiveness against existing non-contextual baselines.

Detection Accuracy and False Positive Reduction
A standard evaluation approach in the literature involves comparing novel approaches with established industry tools (e.g., OWASP Dependency-Check and Dependabot) using datasets of real-world projects. Following the methodology established by Ponta et al. [78] and Zhao et al. [55], the primary evaluation metrics are precision, recall, and false positive rate. Complementing these, the percentage of actionable alerts, defined as the proportion of reported vulnerabilities affecting dependencies that are actually deployed and reachable, is frequently tracked, as demonstrated by Pashchenko et al. [80]. Together, these metrics capture whether context-aware analysis reduces the alert noise identified in earlier studies as a barrier to tool adoption. Additionally, MTTR is often used as a benchmark to measure improvements in remediation time, especially since remediation delays have been shown to increase over the years across multiple ecosystems [82].

Pull Request Acceptance Rate and Time-To-Remediation
To evaluate the practical impact of automated recommendations, empirical studies frequently rely on tracking pull request acceptance rates and time-to-merge as key indicators of developer receptiveness. Rebatchi et al. [6] and Mohayeji et al. [81] adopt this approach precisely because widely adopted tools such as Dependabot achieve low merge rates, leaving a significant portion of security PRs open and unaddressed.

Developer Perception and Decision-Making
Finally, quantitative metrics can be complemented by qualitative evidence on developer decision-making. Pashchenko et al. [73], for example, conducted 25 semi-structured interviews to investigate barriers to dependency management, including the relevance of vulnerability information and the expected effort of an update. Such qualitative evidence captures factors influencing dependency update decisions that are not reflected by detection or remediation metrics alone.

3.2.4 Threats to Validity and Limitations

Several limitations of the review process for RQ2 should be acknowledged. As with RQ1, a reduced search string was used for this database, retaining the full Domain scope but reducing the Outcome/Metrics and Methodology terms. This adaptation may have excluded relevant studies. However, the complete search string was applied to IEEE Xplore, ACM Digital Library,

and the b-on aggregator, providing additional coverage beyond ScienceDirect. The review also considered only English-language publications, which may exclude relevant work reported in other languages.

4 Analysis and Design

This chapter presents the analysis and design of the proposed Contextualised Risk-Based Framework. The analysis begins with the definition of the functional and quality requirements (Section 4.1), followed by the evaluation of the tools and external services required to satisfy them (Section 4.2). The chapter then presents the architectural design of the framework, including its high-level structure, logical organisation, and implementation view, together with the contextual risk-scoring model and the constraint-based optimisation formulation used to support dependency update recommendations (Section 4.3). Finally, the analysis pipeline describes how these elements operate together during a complete dependency risk assessment (Section 4.3.5).

The system views presented throughout this chapter use Unified Modeling Language (UML) notation, as defined in version 2.5.1 of the specification [83].

4.1 Requirements

The framework requirements derive from the gaps identified in the State of the Art and from the constraints of the Maven ecosystem discussed in the Background. They are divided into functional requirements, which define the behaviour expected from the framework, and quality requirements, which constrain how that behaviour must be provided. The quality requirements are grounded in the ISO/IEC 25010:2023 product quality model [84].

4.1.1 Functional Requirements

The framework is intended to support developers making risk-informed decisions about Maven dependency updates. The Developer is therefore the primary actor, initiating the assessment over a given project and receiving the resulting recommendations.

The assessment also depends on information supplied by external services. The Vulnerability Advisory Service reports known vulnerabilities affecting project dependencies, the Exploit

Prediction Service estimates the likelihood that each vulnerability will be exploited, and the Ecosystem Metrics Service provides stability-related metrics for dependency versions and candidate updates.

Figure 4 presents the CRBF use case diagram, providing a high-level view of the interactions between these actors and the use cases involved in the assessment.

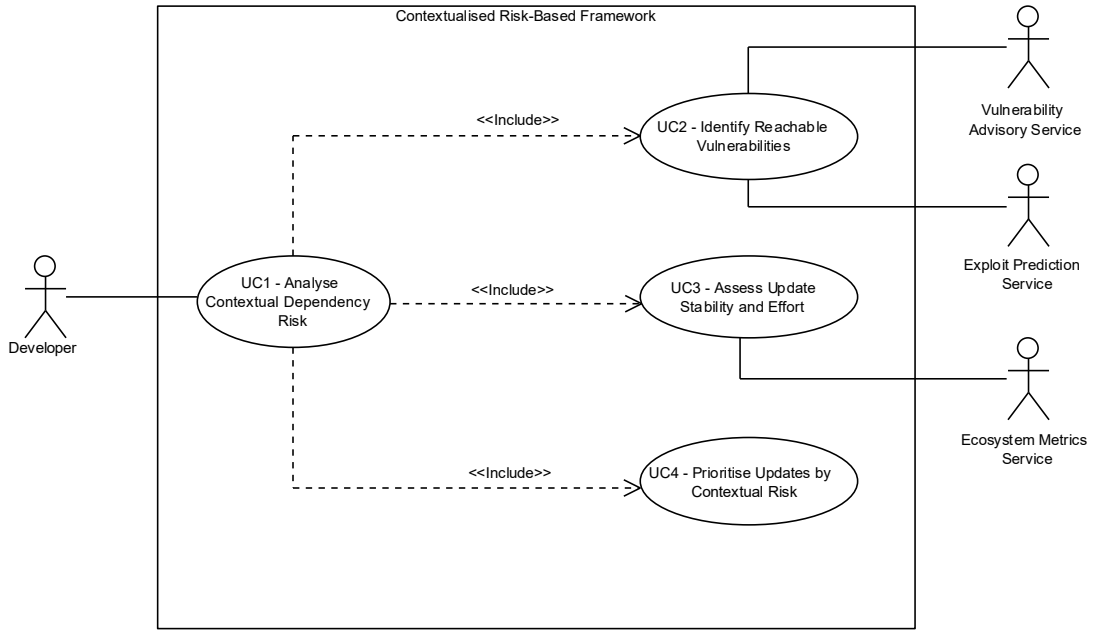


Figure 4 - Use case diagram of the Contextualised Risk-Based Framework

Analyse Contextual Dependency Risk (UC1) represents the overall assessment initiated by the Developer. It includes three high-level capabilities that together support dependency update decisions. *Identify Reachable Vulnerabilities* (UC2) determines which known vulnerabilities are relevant to the analysed project by considering whether the vulnerable code can be reached from the application. *Assess Update Stability and Effort* (UC3) evaluates candidate updates according to their ecosystem stability and the effort associated with compatibility changes. *Prioritise Updates by Contextual Risk* (UC4) combines the contextual evidence gathered during the assessment to rank candidate updates by remediation priority.

The use cases are refined into verifiable functional requirements presented in Table 8. These requirements cover the stages of the assessment process, including dependency resolution, vulnerability analysis, contextual risk assessment, update selection, and communication of results.

Table 8 - Functional Requirements.

ID	Requirement	Acceptance Criteria
FR1	The framework shall resolve the full transitive dependency graph of a Maven project, preserving the GAV coordinates and scope of each resolved dependency.	The dependency graph contains the resolved direct and transitive dependencies, each with defined GAV coordinates and scope.
FR2	The framework shall retrieve the known vulnerabilities affecting each resolved dependency version from a vulnerability data source.	Each resolved dependency is associated with the vulnerabilities retrieved for its version.
FR3	The framework shall obtain the EPSS probability for each identified vulnerability when a score is available and explicitly represent its absence otherwise.	Each identified vulnerability has either an EPSS probability or an explicit indication that the score is unavailable.
FR4	The framework shall determine the project-specific reachability of each identified vulnerability and assign a reachability classification.	Each identified vulnerability receives exactly one reachability classification.
FR5	The framework shall assess the compatibility of each applicable dependency update between the current and proposed versions and assign a compatibility classification.	Each applicable dependency update receives exactly one compatibility classification.
FR6	The framework shall obtain ecosystem stability indicators for each applicable dependency update.	Each applicable dependency update has either ecosystem stability indicators or an explicit indication that the indicators are unavailable.
FR7	The framework shall calculate a contextual risk score for each identified vulnerable dependency using the information gathered during the analysis.	Each identified vulnerable dependency has a contextual risk score.
FR8	The framework shall select dependency updates that maximise contextual risk reduction within a configurable effort budget and the applicable update constraints.	The selected dependency updates satisfy all applicable constraints, remain within the configured effort budget, and achieve the maximum contextual risk reduction among feasible selections.
FR9	The framework shall export a VEX document containing the exploitability status of each identified vulnerability.	The exported VEX document is valid against the selected VEX schema and includes an exploitability status for each identified vulnerability consistent with its reachability classification.
FR10	The framework shall present the prioritised dependency updates and their supporting contextual risk information to the developer.	The exported report includes each recommended dependency update together with its contextual risk score and the information supporting that score.

4.1.2 Quality Requirements

Table 9 presents the quality requirements and their corresponding rationale. Each requirement is associated with the relevant characteristic and sub-characteristic of the ISO/IEC 25010:2023 product quality model [84].

Table 9 - Quality Requirements.

ID	Characteristic (sub-characteristic)	Requirement	Rationale
QR1	Functional Suitability (Functional Completeness).	The framework shall retrieve every vulnerability reported by the selected vulnerability data source for the resolved dependency versions.	Verifies FR2 against the selected vulnerability data source. A vulnerability missed during retrieval propagates as a false negative through the subsequent analysis.
QR2	Performance Efficiency (Time Behaviour).	For a Maven project with up to 100 direct and transitive dependencies, the full analysis shall complete in under 5 minutes on the defined reference environment.	Computational cost is identified as a practical barrier to the adoption of context-aware analysis in Section 3.1.2. Limiting execution time keeps the analysis usable within a standard build cycle.
QR3	Maintainability (Modularity).	Each external service or analysis component shall be replaceable without requiring any changes to the framework's core.	External services and analysis components evolve independently of the framework, and the selections made in Section 4.2 may become outdated. Reducing coupling to these components lowers the cost of replacing them.
QR4	Interaction Capability (Self-Descriptiveness)	For each dependency included in the risk report, the framework shall expose all individual factors used to calculate the contextual risk score, allowing the developer to trace its derivation.	Automated dependency tools often provide insufficient context to support update decisions (Section 3.1.2). Exposing each contributing factor allows developers to audit the recommendation before acting on it.

The reference environment for QR2 is an Apple MacBook Air with an M5 processor, 16 GB of unified memory, running macOS 26.6.2, OpenJDK 17, and Apache Maven 3.9.16.

The tests and verification procedures used to assess the functional and quality requirements are presented in Chapter 5.

4.2 Selection of tools

The selection of tools was guided by the functional and quality requirements defined in the previous section and by the limitations identified in the State of the Art. Five capabilities require external tools: vulnerability data retrieval, static reachability analysis, compatibility analysis, ecosystem stability metrics, and dependency-update optimisation. The following subsections justify the selected tool or service for each capability.

4.2.1 Vulnerability Data Source

To satisfy FR2 (Table 8) and QR1 (Table 9), the framework requires a vulnerability data source that can identify vulnerabilities affecting specific versions of Maven packages without per-user authentication. Section 2.2.5 presents three candidates. The NVD [31] indexes vulnerabilities by CPE, so every Maven query would require a CPE-to-package translation step. The GHSA [32] supports package-level queries, but its API requires authentication. OSV [34] meets both conditions and aggregates advisories from the other two candidates and additional sources, allowing the framework to access these sources through a single service.

To satisfy FR3, CVE records retrieved from OSV are enriched with exploitation probabilities obtained from the EPSS API [30], as described in Section 2.2.4.

4.2.2 Static Reachability Analysis

To satisfy FR4, the framework requires a static analysis engine capable of constructing call graphs across the project and its direct and transitive dependencies. Since reachability classification depends on the precision of the resulting call graph, a pointer-analysis-based approach is preferred to CHA and RTA, as discussed in Section 2.5.2.

Both frameworks presented in Section 2.5.2 expose Java APIs and construct call graphs in memory, so the selection depends primarily on their pointer-analysis support. WALA [50] provides built-in pointer-analysis-based call graph builders. In SootUp, more advanced pointer analysis is provided through the Qilin integration, which in version 2.0.0 was documented as work in progress, with an API subject to change [51]. WALA is therefore selected.

Among the available builders, 0-CFA is selected as a compromise between precision and computational cost. Its context-insensitive analysis avoids the additional cost of distinguishing calling contexts. More precise builders are available, but their additional computational cost makes 0-CFA more suitable under the execution-time constraint defined in QR2.

4.2.3 Compatibility Detection

Satisfying FR5 requires a tool capable of detecting changes that break source or binary compatibility between two versions of a compiled library. Of the three tools compared in

Section 2.4.2 (see Table 1), JApiCmp [43] is selected. It exposes a Java API that accepts two JAR archives and returns a structured classification of the compatibility changes between them. JApiCmp has also been used in ecosystem-level studies of breaking changes in Maven [18].

4.2.4 Ecosystem Stability Metrics

Satisfying FR6 requires ecosystem-level metrics that characterise the stability and adoption of Maven dependency versions. As established in Section 2.4.3, the Goblin Framework [45] maintains a historical dependency graph of Maven Central, which Goblin Weaver exposes through a REST API and enriches with ecosystem-level metrics. Goblin Weaver is therefore selected as the source of these metrics.

Weaver's extensibility and memoisation mechanisms further support this selection. Its extension mechanism allows the adoption indicators required by FR6 to be implemented over the same dependency graph, while memoisation enables repeated queries to reuse previously computed results, reducing processing time and supporting the execution-time constraint defined in QR2 [45].

4.2.5 Remediation Optimisation Engine

Satisfying FR8 requires an optimisation mechanism capable of selecting the subset of dependency upgrades that maximises risk reduction within a configurable effort budget and the applicable update constraints. As discussed in Section 3.1.2, this can be formulated as a constraint-based optimisation problem. Z3 [54], introduced in Section 2.5.3, is selected because it supports optimisation under logical constraints and provides a Java API for direct integration into the framework.

4.3 Architectural Design

This section describes the architectural design of the proposed framework. It begins by describing the system context, then presents the logical structure of the system, followed by the risk scoring model and concludes with the conceptual analysis pipeline that defines how data flows through the system.

4.3.1 System Context

The Contextualised Risk-Based Framework (CRBF) is a Maven plugin that analyses a project's dependency graph to assess dependency risk and recommend updates. Figure 5 shows CRBF and its external environment.

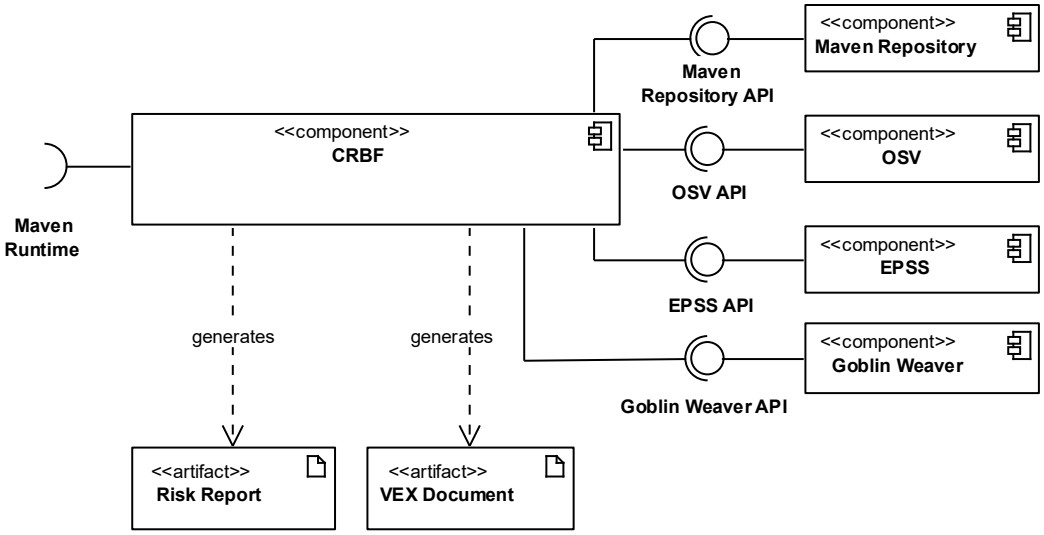


Figure 5 - The CRBF and its external environment

The CRBF retrieves data from four external sources. Maven repositories provide the dependency artefacts under analysis, OSV supplies vulnerability advisories, EPSS provides exploitation-likelihood scores, and Goblin Weaver exposes ecosystem metrics and dependency-graph data.

The analysis produces two outputs. The risk report contains the contextualised assessment and the recommended dependency updates, while the VEX document describes the exploitability status of the identified vulnerabilities.

4.3.2 Logical Organisation

The CRBF follows the Hexagonal Architecture, also known as Ports and Adapters [85]. The style was selected because the framework interacts with several heterogeneous external systems while its core logic should remain independently testable and decoupled from them. External integrations are isolated behind port interfaces, allowing adapters to be replaced without changes to the core. Figure 6 shows this organisation.

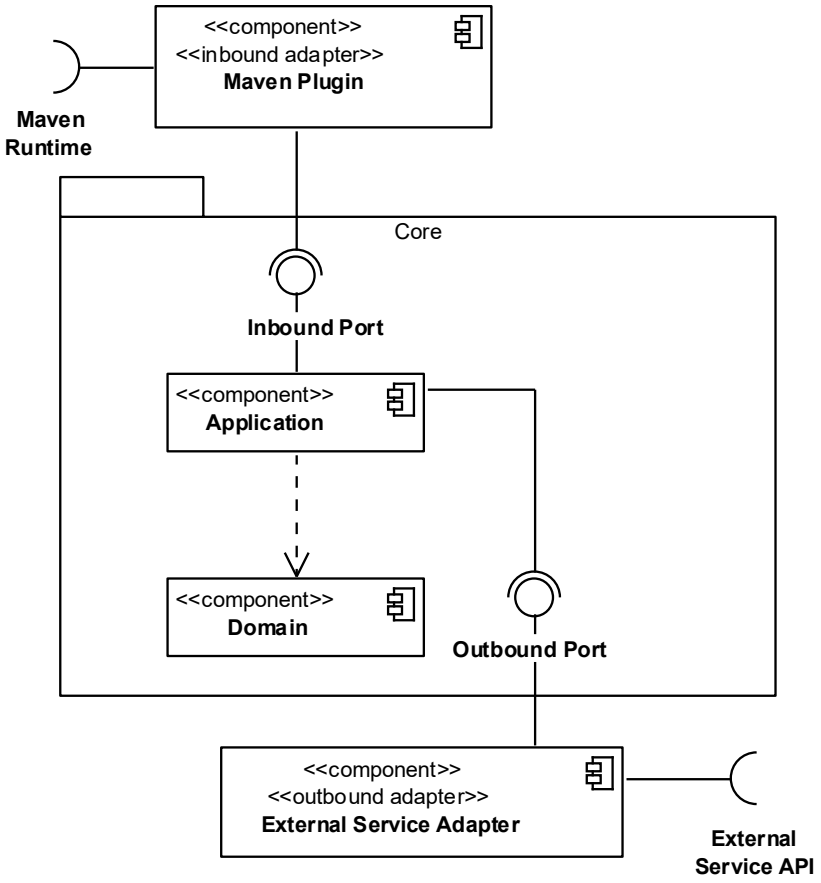


Figure 6 - Logical organisation of the CRBF

The core comprises a domain layer and an application layer. The domain layer contains the entities and value objects that model dependency risk and remediation. These types do not depend on external libraries and form the innermost part of the architecture.

The application layer defines the framework's behaviour through inbound ports, outbound ports, and application services. Inbound ports define the use cases that external actors can invoke, while outbound ports define the capabilities required from external systems without specifying their implementation. Application services implement the use cases by coordinating these outbound ports. In the diagrams above, these architectural ports are represented as UML interfaces.

The Maven plugin hosts the inbound adapter through which the Maven runtime invokes the framework. Outbound adapters implement the outbound ports and translate between the framework's internal model and the external systems or libraries they encapsulate. Adapters depend inward on the core, never the reverse, preserving the separation between the core and external integrations.

4.3.3 Implementation View

The implementation view in Figure 7 refines the logical organisation introduced in Section 4.3.2 by mapping the architectural roles to concrete implementation elements. To preserve readability, the figure details the reachability analysis path as a representative example of the interaction between an application service, an outbound port, and its adapter.

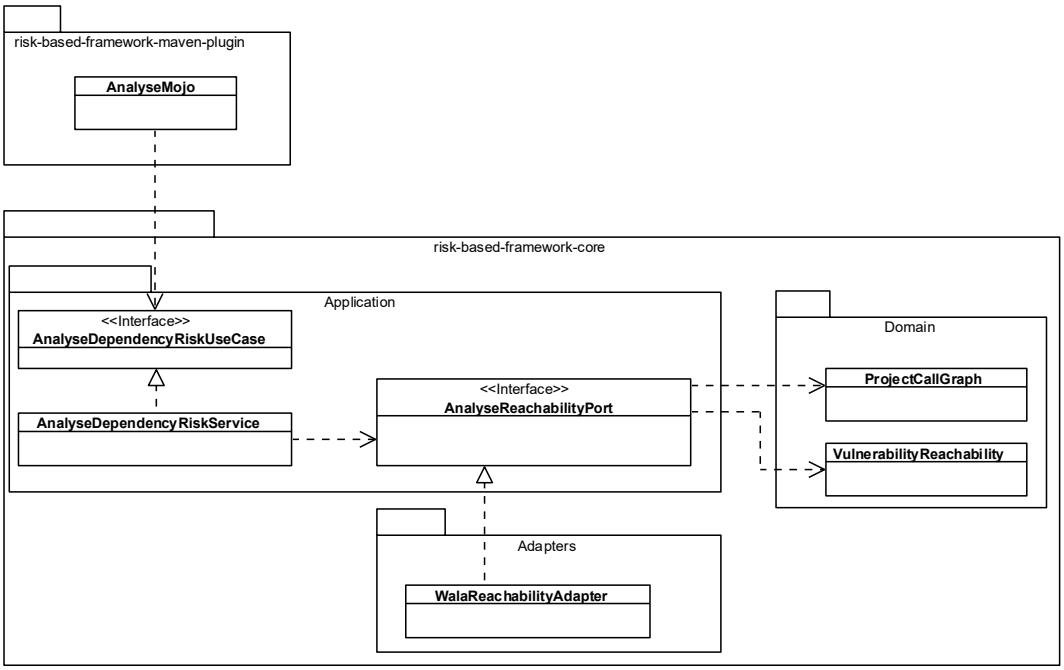


Figure 7 - Implementation view of the reachability analysis port and adapter

The framework is organised into two Maven modules. `risk-based-framework-core` contains the domain, application, and outbound adapter packages, while `risk-based-framework-maven-plugin` contains the Maven entry point and Maven-specific artefact resolution. The plugin module acts as the composition root.

Within this structure, `AnalyseMojo` provides the framework entry point and delegates the analysis to `AnalyseDependencyRiskUseCase`, which is implemented by `AnalyseDependencyRiskService`. The service accesses the reachability analysis through `AnalyseReachabilityPort`, implemented by `WalaReachabilityAdapter`. The port exposes framework types rather than WALA-specific types: `ProjectCallGraph` represents the call graph constructed for the analysed project, while `VulnerabilityReachability` represents the reachability classification assigned to a vulnerability. WALA therefore remains confined to the adapter and can be replaced without changing the application service or domain model.

The same structure is applied to the remaining outbound capabilities: each outbound adapter implements a port defined by the application layer, keeping dependencies on external systems

and libraries outside the domain and application layers. Their concrete implementations are described in Chapter 5.

4.3.4 Risk Scoring and Remediation Optimisation

This section defines the contextual risk model used by the framework to quantify dependency risk and the remediation optimisation model used to select dependency updates. The risk model combines vulnerability severity, exploitation likelihood, dependency staleness, project-specific reachability, and the quality of the proposed upgrade path. Compatibility is considered by the remediation model to estimate the effort required to apply an update.

The contextual risk score is calculated independently for each vulnerability. CVSS severity, EPSS exploitation likelihood, and dependency staleness form a weighted base score, which is then adjusted according to reachability and upgrade-path quality. All inputs and adjustment factors are bounded within the interval [0,1], ensuring that the resulting score also remains in this range.

To combine the base-risk signals into a single score, the framework adopts a Multi-Criteria Decision-Making (MCDM) approach. Among the techniques compared in 2.3.2, SAW [36] was selected because it computes a weighted sum of normalised criteria, producing a score determined by the input values and configured weights. Its linear and transparent structure also supports the interpretability required by QR4 (Self-Descriptiveness, see Table 9). AHP was rejected because it requires pairwise expert comparisons to determine the criteria weights, making it unsuitable for automated risk assessment. TOPSIS was also rejected because it produces a relative ranking based on the alternatives under consideration, meaning that the resulting score can change when the set of alternatives changes. This conflicts with the framework's requirement that the same input values produce the same score under a fixed scoring configuration.

Staleness Urgency

The staleness urgency quantifies the degree to which a dependency version used by the project is outdated. It combines two dimensions of technical lag introduced in Section 2.4.1: the time lag between the version in use and the latest available version, and the number of missed releases. The resulting staleness urgency is expressed as a value between 0 and 1:

$$stalenessUrgency = timeLagNorm \times 0.60 + versionLagNorm \times 0.40 \quad (2)$$

, where:

- *timeLagNorm = min (timeLagDays / 365, 1.0)* represents the temporal distance, in days, between the release date of the dependency version used by the project and the latest available release, normalised against a 365-day saturation threshold.
- *versionLagNorm = min (versionLag / 10, 1.0)* represents the number of missed releases, normalised against a saturation threshold of 10 releases.

The two dimensions are combined using initial weights of 0.60 for time lag and 0.40 for version lag. Time lag receives greater weight because it provides a consistent measure of how long the dependency has remained outdated, independently of the library's release cadence. Version lag complements this measure by capturing the number of missed update opportunities independently of elapsed time, including releases that may contain security or maintenance fixes. However, its interpretation depends on the release cadence of each project: ten missed releases in a rapidly evolving library do not represent the same degree of outdatedness as ten releases in a slowly evolving one. The 0.60/0.40 split therefore represents an initial design choice rather than an empirically calibrated weight.

The 365-day and 10-release thresholds define the point at which each normalised measure reaches its maximum value of 1.0. A time lag greater than 365 days is therefore treated the same as 365 days, and a version lag greater than 10 releases is treated the same as 10 releases. This prevents extreme lag values from disproportionately influencing the staleness urgency.

Base Risk Score

The base risk score is calculated independently for each vulnerability and combines three signals: CVSS severity, EPSS exploitation likelihood, and the staleness urgency of the dependency version containing the vulnerability. CVSS is normalised from [0,10] to [0,1], while EPSS and staleness are already defined on [0,1]. The resulting inputs can therefore be combined directly using SAW:

$$B_v = C_v \times w_C + E_v \times w_E + S_a \times w_S \quad (3)$$

, where:

- B_v is the base risk score of vulnerability v.
- $C_v = \mathrm{CVSS}_v \,/\, 10$ is the normalised CVSS Base Score of vulnerability v.
- E_v is the EPSS exploitation probability associated with vulnerability v.
- S_a is the staleness of artifact a, calculated using Equation (2).
- w_C, w_E, w_S are the weights assigned to CVSS, EPSS, and staleness urgency, respectively.

The default base weights assigned to the three base-risk signals are presented in Table 10, together with the rationale for their relative importance.

Table 10 - Base risk score weights rationale.

Weight	Value
$\boldsymbol{w_{cvss}}$	0.50 (a)
$\boldsymbol{w_{epss}}$	0.30 (b)
$\boldsymbol{w_{staleness}}$	0.20 (c)

Notes:

- (a) CVSS [28] anchors the score on the intrinsic severity of a vulnerability and is a widely adopted signal for vulnerability prioritisation. The largest weight is therefore assigned to CVSS as the reference severity signal.
- (b) EPSS complements CVSS by providing an empirical probability of exploitation. Jacobs et al. [29] report that EPSS v3 achieves 78.5% efficiency at prioritising vulnerabilities exploited in the wild, compared with 6.5% for CVSS-based only prioritisation. This evidence supports assigning EPSS a substantial weight, although lower than CVSS because EPSS estimates exploitation likelihood rather than impact. The two signals are complementary and capture different aspects of vulnerability risk.
- (c) Staleness is an indirect proxy for accumulated risk rather than a direct measure of a specific threat. Its role is to increase the priority of outdated dependencies that might otherwise receive a lower score based only on severity and exploitation likelihood. It therefore receives the smallest weight.

The 0.50/0.30/0.20 weighting represents an initial design choice rather than an empirically calibrated configuration.

When an EPSS score is unavailable for a vulnerability, its weight is redistributed proportionally between CVSS and staleness rather than assigning a zero exploitation likelihood. This preserves the total weight of 1.0 while ensuring the absence of EPSS data does not artificially reduce the base risk score.

Contextual Risk Score

The base risk score captures the severity, exploitation likelihood, and staleness associated with a vulnerability, but does not account for how that vulnerability affects the analysed project or the quality of the proposed remediation path. The contextual risk score adjusts the base risk using two multiplicative factors: project-specific reachability and upgrade-path quality. It is defined as:

$$R_v = B_v \times RW_v \times PQ_a \quad (4)$$

, where:

- R_v is the contextual risk score of vulnerability v.
- B_v is the base risk score of vulnerability v, calculated using Equation (3).
- RW_v is the reachability weight assigned to vulnerability v.
- PQ_a is the path-quality factor associated with the proposed update for artifact a containing vulnerability v.

Since the base risk, reachability weight, and path-quality factors are all defined on [0,1], the contextual risk score also ranges from 0 to 1. The two contextual factors are non-amplifying: they can preserve or reduce the base risk but cannot increase it. The resulting score is dimensionless and should be interpreted as a prioritisation measure rather than as a probability or percentage of exploitation.

The multiplicative adjustment is conceptually similar to the use of Environmental Metrics in CVSS v4.0 [28], which modifies a base severity score according to the context or the affected system. In CRBF, the contextual factors capture concerns specific to dependency management: whether the vulnerable code is reachable from the analysed project and whether the proposed update introduces additional transitive vulnerability risks. These two factors are defined below.

1. **Reachability Weight**

   The reachability weight adjusts the base risk according to the project-specific reachability of each vulnerability. The four reachability classifications and their corresponding weights are presented in Table 11.

   Table 11 – Reachability weights.

Reachability Status	Weight (Rationale)
Reachable	1.0 (a)
Probably Reachable	1.0 (b)
Unknown	0.5 (c)
Unreachable	0.1 (d)

   Notes:

   - (a) The vulnerable class or method is confirmed as reachable in the call graph, so the full base risk is retained.
   - (b) The dependency is reachable, but the specific vulnerable element cannot be identified. The full weight is therefore retained conservatively.
   - (c) Reachability cannot be determined, so an intermediate weight of 0.5 is applied. This avoids treating the vulnerability as safe when static call-graph analysis may miss behaviour involving reflection or path constraints.
   - (d) The vulnerable code is not present in the call graph, so the risk is substantially reduced. The weight is kept above zero because static call-graph analysis may fail to capture all runtime call edges and therefore cannot guarantee the vulnerable code is unreachable at runtime.

2. **Path Quality Factor**

   The path-quality factor adjusts the contextual risk according to the security quality of the proposed update path. It considers whether the proposed fix version remains affected by known vulnerabilities and whether the update introduces new vulnerable transitive dependencies. The factor is defined as:

$$PQ_a = \max\left(0.3, \frac{S_a + 1}{2}\right) \tag{5}$$

   , where:

   - PQ_a is the path-quality factor associated with the proposed update for artifact a.
   - $S_a \in [-1,1]$ is the upgrade-path security signal for artifact a, reflecting whether the proposed fix version contains known vulnerabilities and whether the update introduces new vulnerable transitive dependencies.

The upgrade-path security signal is calculated as:

$$S_a = F_a - \frac{V_a}{N_a} \quad (6)$$

, where:

- F_a is 1 when the proposed fix version has no known vulnerabilities and 0 otherwise.
- V_a is the number of vulnerable transitive dependencies introduced by the update.
- N_a is the total number of transitive dependencies introduced by the update.

Since S_a ranges from -1 to 1, it is normalised to the interval [0,1] before the minimum path-quality factor is applied. A value of 1 represents the most favourable update path, while lower values indicate progressively less favourable paths. A minimum factor of 0.3 prevents upgrade-path quality from reducing the contextual risk to zero.

When the upgrade-path security signal cannot be computed because vulnerability data for the proposed fix version or transitive dependency data are unavailable, PQ_a defaults to 1.0. This preserves the base risk rather than reducing it based on missing evidence and does not imply that the upgrade path is clean.

Ecosystem Stability Score

The stability score characterises a proposed fix version based on its ecosystem adoption and maintenance history. It is defined as a continuous value in the interval [0.0, 1.0], where higher values indicate stronger evidence of ecosystem adoption and active maintenance, while lower values indicate weaker evidence. The score is derived from a weighted aggregation of three ecosystem metrics:

$$ES_a = adoptionRate_a \times 0.55 + maintenanceNorm_a \times 0.30 + lifespanNorm_a \times 0.15 \quad (7)$$

, where:

- ES_a is the ecosystem stability score of the proposed fix version for artifact a.
- $adoptionRate_a$ is the adoption rate of the proposed fix version, expressed as a value in [0,1].
- $maintenanceNorm_a = \min(maintenanceRate_a \,/\, 0.10, 1)$ is the maintenance rate normalised against a threshold of 0.10 releases per day, above which the maintenance component reaches its maximum value.
- $lifespanNorm_a = \min(adoptionLifespan_a \,/\, 730, 1)$ is the adoption lifespan, in days, normalised against a threshold of 730 days, above which additional lifespan does not increase the normalised value.

The three metrics capture complementary dimensions of ecosystem stability, and the relative weights reflect a deliberate ordering of their importance. Adoption rate receives the largest weight (0.55) because it directly reflects the current use of the proposed version across dependent projects. The normalised maintenance rate receives a weight of 0.30 because adoption alone does not indicate whether the artifact continues to be actively maintained; a widely adopted version may still belong to a project with limited ongoing maintenance. The normalised lifespan receives the lowest weight (0.15) because recently released versions naturally have shorter lifespans, which makes this signal less reliable when comparing versions of different ages.

The selected weights and normalisation thresholds are initial design values rather than empirically calibrated parameters.

When one or more ecosystem metrics are unavailable, the stability score is not computed and is reported as unavailable rather than interpreting missing data as low ecosystem stability.

The ecosystem stability score complements the prioritisation result by providing evidence about the adoption and maintenance context of each recommended update. A high-risk dependency may have a widely adopted and actively maintained fix available, whereas another may have only a fix belonging to an abandoned project; this distinction is relevant to the final update decision.

Upgrade Effort Cost

The upgrade effort cost estimates the relative effort required to adopt a proposed fix version. Instead of treating all updates equally, the cost is derived from the compatibility classification between the current dependency version and the proposed fix version. The possible compatibility classifications and their corresponding effort costs are presented in Table 12.

Table 12 – Compatibility effort costs

Compatibility Classification	Effort Cost	Rationale
Compatible	1.0	No source or binary incompatibilities detected; lowest relative effort cost.
Unknown	2.0	Compatibility could not be determined; an intermediate conservative cost is assigned.
Source Incompatible	3.0	Source compatibility is broken, potentially requiring changes before recompilation.
Binary Incompatible	4.0	Binary compatibility is broken; highest relative effort cost.
No fix available	N/A	The dependency is excluded from remediation optimisation.

These effort costs are expressed as relative units and are used by the optimiser to represent the cost of each candidate update.

Remediation Optimisation Formulation

The core challenge of the remediation phase is that updating all vulnerable dependencies is not always feasible. Each upgrade consumes developer effort, including resolving compilation issues and running tests to detect regressions, while teams operate within limited effort budgets. The framework therefore selects the subset of updates that maximises the cumulative contextual risk addressed without exceeding the configured budget. Each remediation candidate has an associated contextual risk score and upgrade effort cost. The problem is formulated as a constrained optimisation model and solved using Z3. Given *n* remediation candidates, the formulation is defined below.

For each remediation candidate i, a binary decision variable $upgrade_i$ indicates whether the corresponding update is selected:

$$upgrade_i \in \{0,1\}, \quad \forall i \in \{1, \dots, n\} \tag{8}$$

, a value of 1 indicates that the update is selected, while 0 indicates it is not selected.

The objective is to maximise the cumulative contextual risk addressed by the selected updates:

$$Maximise \sum_{i=1}^{n} (upgrade_i \times contextualRisk_i) \tag{9}$$

, where $contextualRisk_i$ represents the contextual risk score associated with the remediation candidate i. Only selected updates contribute to the objective function.

The objective is subject to the following constraints:

- **(C1) Effort Budget Constraint:** The cumulative effort cost of the selected updates must not exceed the configured effort budget.

  $$M \sum_{i=1}^{n} (upgrade_i \times effortCost_i) \leq effortBudget \tag{10}$$

  , where $effortCost_i$ represents the estimated effort required to apply an update i, while $effortBudget$ represents the maximum effort allocated to remediation.

- **(C2) Fix Availability Constraint:** An update can only be selected if a valid fix version is available.

  $$\forall i: \neg hasFixAvailable_i \Rightarrow \neg upgrade_i = 0 \tag{11}$$

- **(C3) Mandatory Upgrade Constraint:** Updates classified as mandatory must be selected.

  $$\forall i: isMandatory_i \Rightarrow upgrade_i = 1 \tag{12}$$

  An update is classified as mandatory when it addresses a critical vulnerability that is reachable from the analysed application and has an available fix. If the effort required

for mandatory updates exceeds the configured budget, the optimiser cannot satisfy both constraints. In this case, the framework produces an infeasible remediation plan, identifying the mandatory updates and the additional effort required without selecting any updates. A greedy fallback is used when the solver encounters a technical failure, providing a feasible selection without guaranteeing optimality.

The weights used in the scoring model are initial design choices informed by the evidence discussed in the State of the Art, rather than empirically calibrated values. Although some weights are configurable, the framework does not currently support automatic calibration. A future extension could introduce an adaptive calibration mechanism, such as the MAPE-K model discussed in Section 2.3.3, to adjust them based on observed developer decisions.

4.3.5 Analysis Pipeline

The framework analyses a Maven project's dependencies through six sequential phases to produce a contextualised risk report and a VEX document. Figure 8 illustrates the analysis pipeline.

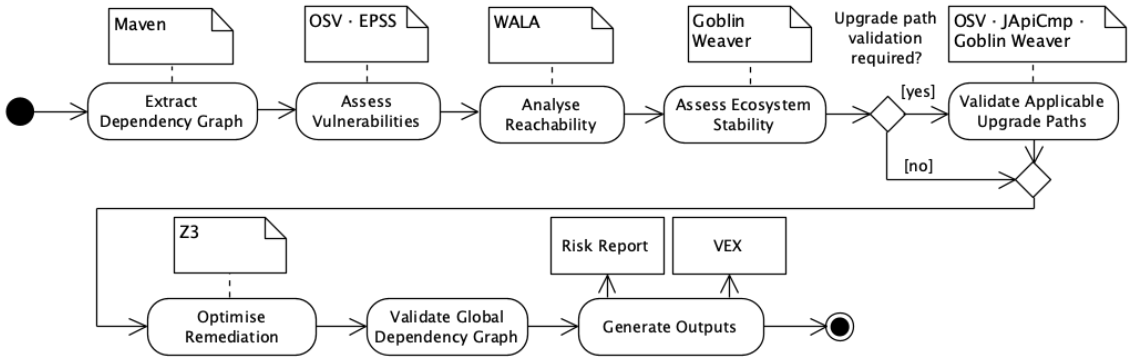


Figure 8 – The CRBF analysis pipeline.

Phase 1 – Dependency Graph Extraction

The framework receives the target project's dependency graph, including direct and transitive dependencies. Each dependency is identified by its Maven coordinates and scope. The resulting graph serves as input for the subsequent analysis phases.

Phase 2 – Vulnerability Assessment

For each dependency in the graph, the framework queries OSV for known vulnerabilities affecting its resolved version. Dependencies without recorded vulnerabilities are excluded from further vulnerability-specific analysis. The identified vulnerabilities are then enriched with EPSS exploitation probabilities, when available.

Phase 3 – Reachability Analysis

For each vulnerable dependency, the framework uses a call graph generated with WALA to assess whether the affected code is reachable from the project. Each vulnerability is classified as reachable if an affected class or method is identified in the call graph, probably reachable if the dependency is reached but the affected code cannot be identified, unreachable if no relevant path is found, or unknown if the available evidence is insufficient. Figure 9 presents the rationale for the reachability classification decision.

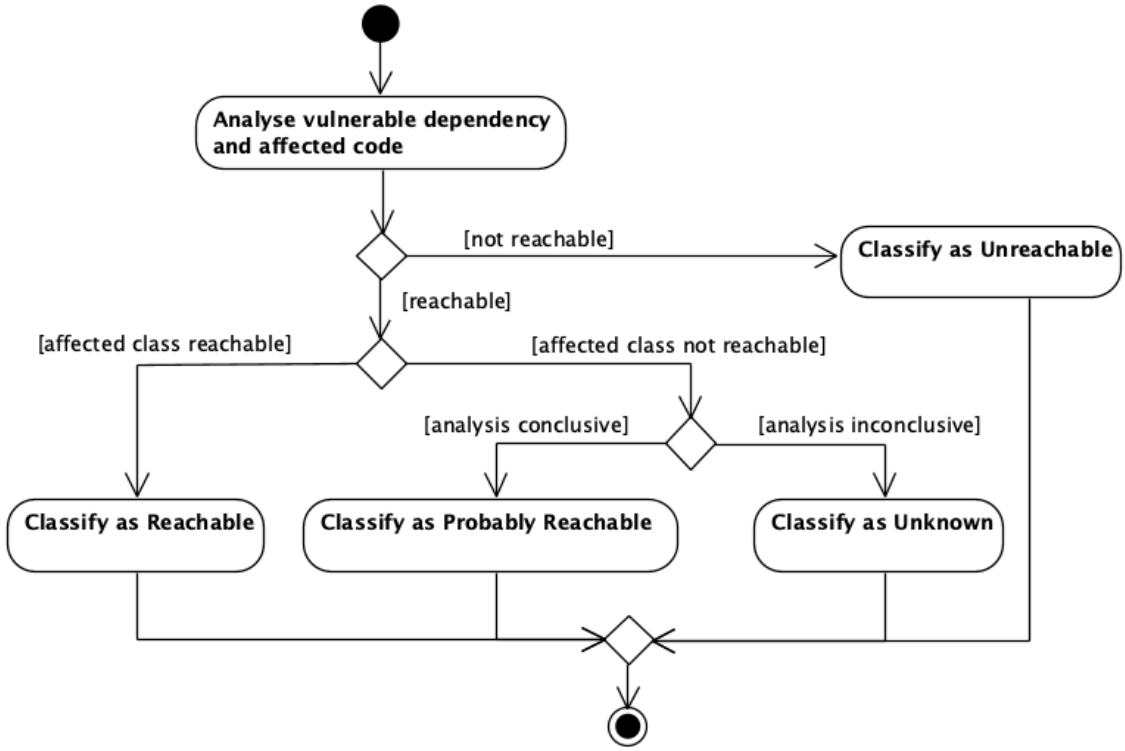


Figure 9 - Reachability classification flow.

Phase 4 – Ecosystem Stability Assessment

For each vulnerable dependency, the framework retrieves ecosystem data from Goblin Weaver to assess how outdated its resolved version is. The time lag and number of missed releases are used to calculate the staleness urgency defined in section 4.3.4.

Phase 5 – Upgrade Path Validation

For each vulnerable dependency with an available fix that is not confirmed as unreachable, the framework validates the proposed upgrade path. It selects a candidate fix version using OSV and Goblin Weaver data, checks whether the proposed version contains known vulnerabilities, and assesses compatibility using JApiCmp when the required JAR files are available. The framework also compares the transitive dependencies of the current and proposed versions to identify newly introduced vulnerable dependencies. Finally, ecosystem stability metrics are retrieved for the proposed version.

Phase 6 – Optimised Remediation

The remediation candidates resulting from previous phases are submitted to the Z3 optimiser, which selects dependency updates according to the objective and constraints defined in Section 4.3.4. The selected updates are then validated together using Goblin Weaver. This step checks the resulting dependency graph for vulnerable transitive dependencies. Finally, the framework generates a report containing the risk assessment and recommended updates, alongside a VEX document describing the exploitability status of the identified vulnerabilities.

5 Implementation

This chapter presents the implementation of the CRBF as a Maven plugin, following the architecture and analysis pipeline defined in Chapter 4. It describes the integration with the Maven runtime, the implementation of dependency analysis and remediation mechanisms, and the generation of the resulting reports. The chapter also presents the tests and verification procedures used to assess the functional and quality requirements defined in Section 4.1. The source code is publicly available in the project repository.

5.1 Development Environment and Technology Stack

The framework was implemented in Java 17 and organised as a multi-module Maven project. The project consists of two modules: `risk-based-framework-core`, which contains the domain, application, and adapter layers, and the `risk-based-framework-maven-plugin`, which integrates the framework with Maven and acts as the composition root. The separation isolates the application logic from the Maven plugin implementation, following the architecture described in Section 4.3.3.

The implementation relies on several third-party libraries: WALA for static call graph analysis, JApiCmp for compatibility detection, Z3 for remediation optimisation, Jackson for JSON processing, and CycloneDX Core Java for VEX generation. External data is retrieved through the OSV, EPSS, and Goblin Weaver APIs. JUnit 5, Mockito, and ArchUnit support implementation testing. The project uses a parent POM to centralise dependency versions and build configurations.

5.2 Maven Plugin Integration and Pipeline Orchestration

The AnalyseMojo class implements the Maven plugin's `analyse` goal, which initiates the dependency analysis. Before starting the analysis, the plugin validates the configured effort

budget and risk weights. It also checks whether the project's compiled classes are available for reachability analysis. If they are missing, the plugin issues a warning and continues execution.

Dependency resolution uses Maven's DependencyGraphBuilder to obtain the project's resolved dependency tree. The `MavenDependencyResolver` then traverses this tree using a depth-first search, producing a `DependencyPath` for each dependency. Each path preserves the Maven coordinates, scope, and sequence of dependencies from the project root.

The AnalyseMojo also acts as the composition root, instantiating the concrete adapters and injecting them into the `AnalyseDependencyRiskService` through their corresponding port interfaces. The service receives the project directory, compiled classes directory, and resolved dependency paths, and orchestrates the analysis pipeline described in Section 4.3.5. The plugin also provides configurable parameters for the risk weights, effort budget, call graph analysis, and external service endpoints.

5.3 Vulnerability and Reachability Analysis

The OsvVulnerabilityAdapter queries the OSV API using each dependency's Maven coordinates and resolved version. The OsvVulnerabilityMapper converts the returned advisories into domain objects containing vulnerability identifiers, severity information, and available fixes. It calculates CVSS base scores from the reported vectors, prioritising CVSS v4 over v3 when available. To identify applicable fix versions, it considers only versions newer than the currently resolved version and belonging to the corresponding Maven package.

The EpssAdapter enriches the identified vulnerabilities with exploitation probabilities. It collects their CVE identifiers and retrieves the corresponding EPSS scores in a batched request for each vulnerable dependency. Missing scores remain unavailable, while API failures do not interrupt the analysis. In contrast, failed OSV queries are recorded as lookup failures, since the absence of vulnerability data cannot be interpreted as evidence that a dependency is secure.

The WalaReachabilityAdapter constructs a call graph using WALA's 0-CFA algorithm. The analysis includes the project's compiled classes and the JAR files of its resolved dependencies. Instead of relying exclusively on the `main` method, the adapter considers all public and protected non-abstract application methods as entry points. This approach reduces the risk of missing execution paths in applications that use dependency injection or other forms of dynamic method invocation.

To reduce the computational cost, the adapter limits reflection modelling and applies configurable exclusions during class-hierarchy construction. These exclusions primarily target JDK internal and graphical user interface packages, such as `java.awt.*` and `javax.swing.*`. The adapter also uses a configurable timeout for call graph construction. The graph is built once per analysis and reused for all vulnerable dependencies. If its construction fails or exceeds the timeout, the framework assigns an unknown reachability status rather than classifying the

vulnerabilities as unreachable. Code 1 illustrates the WALA configuration and call graph construction.

```
...
options.setReflectionOptions(
 AnalysisOptions.ReflectionOptions.NO_FLOW_TO_CASTS_NO_METHOD_INVOKE);

IAnalysisCacheView cache = new AnalysisCacheImpl();

CallGraphBuilder<InstanceKey> builder = Util.makeZeroCFABuilder(
 Language.JAVA, options, cache, cha);

CallGraph callGraph = makeCallGraphWithTimeout(builder, options);
…
```

Code 1 - WALA configuration and call graph construction.

For each vulnerable dependency, the adapter compares the classes contained in its JAR with the reachable methods in the call graph. It then uses Java class names mentioned in the vulnerability advisory to refine the classification. A vulnerability is classified as reachable when a matching class contains a reachable method, or probably reachable when other methods from the dependency are reachable. If neither condition is met, the vulnerability is classified as unreachable, unless the JAR declares `ServiceLoader` providers. In that case, the status is unknown because these providers may be loaded dynamically.

5.4 Ecosystem Analysis and Upgrade Path Validation

The GoblinWeaverStabilityAdapter retrieves ecosystem stability metrics from Goblin Weaver for vulnerable dependencies and their proposed fix versions. It combines release-level and artifact-level data to obtain time and version lag, adoption rate, adoption lifespan, maintenance rate, and the last available version. If the required data is unavailable or an API request fails, the metrics remain unavailable rather than being treated as zero values.

The UpgradePathAnalyser validates potential updates for vulnerable dependencies with an available fix and a reachability status other than unreachable. It uses GoblinWeaverGraphAdapter to retrieve newer versions and identify candidates without reported vulnerabilities. The adapter selects the earliest suitable version that is at least as recent as the fix identified by OSV, after confirming that no vulnerabilities are reported for that version by OSV. If no suitable candidate is found, the original OSV fix version is used.

The JapicmpCompatibilityAdapter compares the current and proposed JAR files using JApiCmp to identify changes to their public APIs. These changes are classified according to their source and binary compatibility. If either JAR is unavailable or the comparison fails, compatibility is classified as unknown. The resulting report is also contextualised using the project's call graph, allowing the framework to determine whether the methods affected by breaking changes are reachable.

Finally, the framework compares the transitive dependency graphs of the current and proposed versions using Goblin Weaver. This comparison identifies added and removed dependencies, including newly introduced dependencies with known vulnerabilities. Together with the vulnerability assessment of the proposed version, these results determine whether the upgrade path is clean, introduces risks, or cannot be fully assessed. The resulting information contributes to the upgrade-path quality factor defined in Section 4.3.4.

5.5 Risk Scoring and Remediation Optimisation

The risk-scoring logic is implemented in the domain model, primarily through the RemediationCandidate and ContextualRisk classes, following the model defined in Section 4.3.4. Each vulnerability is scored independently using the configured weights and contextual factors. When EPSS is unavailable, its weight is redistributed proportionally between CVSS and staleness. The highest contextual risk score is then used to represent the dependency during remediation optimisation.

The StabilityScore class calculates staleness urgency from the time and version lag of the current dependency. When ecosystem data is unavailable, the framework uses a default urgency value of 0.5. Similarly, when the upgrade path cannot be assessed, its quality factor defaults to 1.0. The RemediationCandidate also assigns an estimated effort cost to each proposed update based on its compatibility classification, using the relative costs defined in Table 12.

The Z3RemediationAdapter translates the optimisation model defined in Section 4.3.4 into Z3 expressions. It represents each upgrade decision as a Boolean variable and calculates the total risk reduction and effort associated with the selected upgrades. The adapter then applies the budget, feasibility, and mandatory-upgrade constraints before invoking the solver. Code 2 illustrates how these elements are configured using the Z3 Java API.

```
...
Optimize optimizer = ctx.mkOptimize();
int n = candidates.size();

BoolExpr[] upgradeVars = declareDecisionVariables(ctx, n);

applyHardConstraints(ctx, optimizer, candidates, upgradeVars);

ArithExpr<RealSort> totalRiskReduction =
        buildRiskObjective(ctx, candidates, upgradeVars);
ArithExpr<RealSort> totalEffort =
        buildEffortSum(ctx, candidates, upgradeVars);

optimizer.MkMaximize(totalRiskReduction);
BoolExpr budgetConstraint =
        ctx.mkLe(totalEffort, ctx.mkReal(Double.toString(effortBudget)));
optimizer.Add(budgetConstraint);

Status status = optimizer.Check();
```

Code 2 - Z3 objective and constraint configuration for remediation optimisation.

When the effort budget is insufficient to accommodate all mandatory upgrades, the adapter returns an infeasible remediation plan. The report still identifies the mandatory updates, their estimated effort costs, and the additional effort required to address them. This allows developers to understand which updates remain necessary, even though no feasible selection can be made within the available budget. If Z3 fails due to a technical error, the adapter uses a greedy fallback that respects the constraints but does not guarantee optimality. For feasible plans, the selected updates are subsequently validated together through Goblin Weaver, and the results are included in the report.

5.6 Risk Report and VEX Generation

The RiskReportAssembler consolidates the analysis results and remediation decisions into a structured `RiskReport` object. For each vulnerable dependency, the report includes the identified vulnerabilities, contextual risk scores, reachability evidence, compatibility assessment and recommended action. It also records residual risk, vulnerability lookup failures, and the results of global dependency graph validation.

The ReportGeneratorAdapter exports the report into two files within the project's target directory. The `risk-report.json` file contains the structured results, serialised using Jackson. The `risk-report.html` file provides a visual dashboard generated from an embedded HTML template, into which the report data is inserted.

Figure 10 illustrates the dashboard overview and remediation plan. The overview summarises the identified vulnerabilities, reachability results, and risk reduction, while the remediation plan presents the selected dependency updates, their risk scores, and associated effort costs.

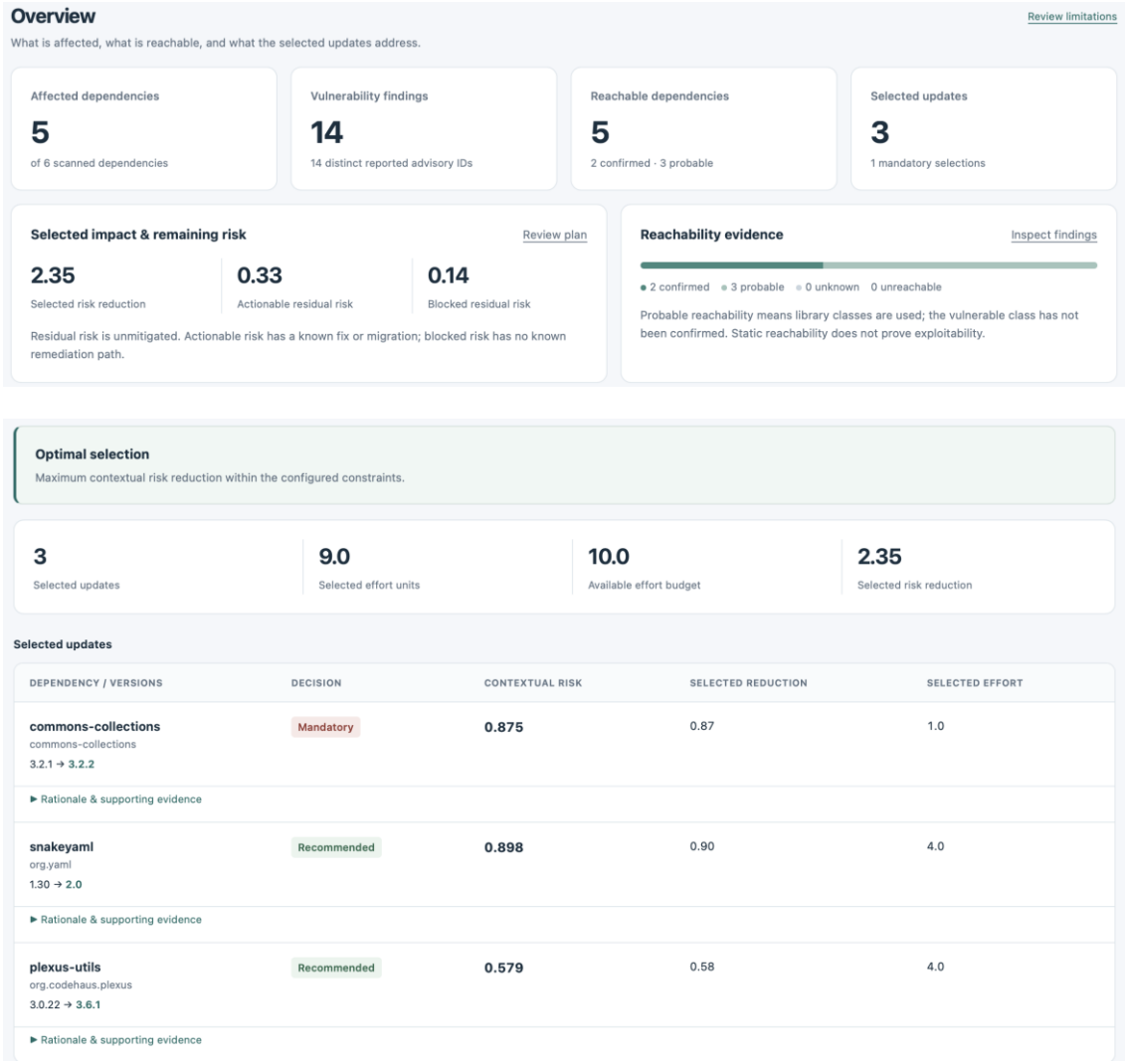


Figure 10 - Overview and Remediation Plan in the generated risk report.

Figure 11 provides a more detailed view of the evidence supporting the analysis results. It presents the identified vulnerabilities and the individual factors used to calculate their risk scores, alongside the compatibility assessments and API changes associated with proposed updates.

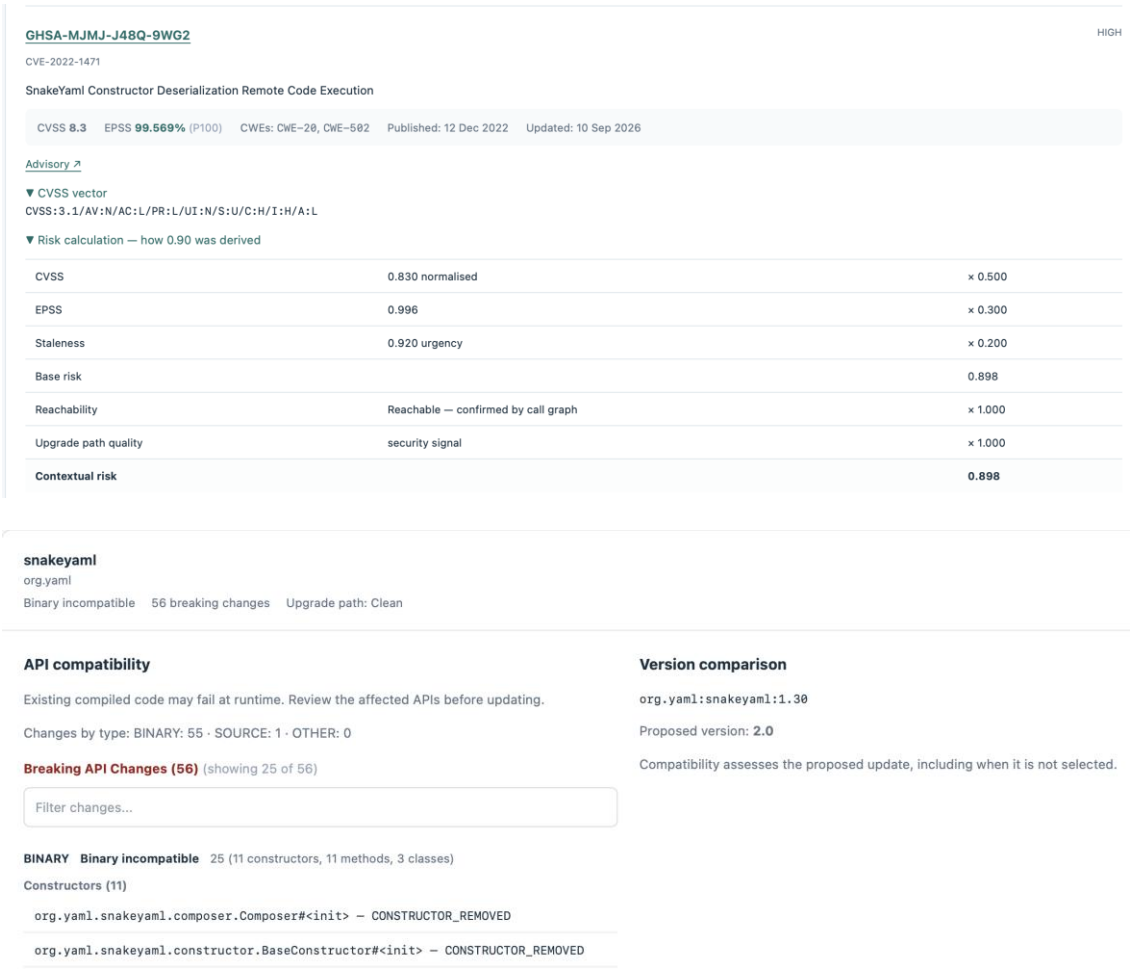


Figure 11 - Vulnerability analysis and API compatibility assessment in the generated report.

The CycloneDxVexAdapter also generates a `vex.json` file following the CycloneDX 1.7 specification. It represents the analysed dependencies as components identified by Maven Package URLs and associates each vulnerability with an exploitability assessment derived from its reachability classification. Confirmed reachability is mapped to `exploitable`, while unreachable vulnerabilities are marked as `not_affected` with the justification `code_not_reachable`. Probably reachable and unknown cases are assigned `in_triage`, reflecting the absence of sufficient evidence for a definitive classification.

5.7 Testing and Requirements Verification

CRBF was tested throughout development to verify the functional and quality requirements defined in Section 4.1 and identify implementation deficiencies. The test suite uses JUnit 5 and Mockito to examine domain logic, application services, and external adapters, while ArchUnit checks architectural constraints. Table 13 and Table 14 summarise the tests associated with each functional and quality requirement, respectively.

Table 13 - Functional requirements testing.

Requirement	Test classes	Tests and results
FR1	MavenDependencyGraphResolverTest	Verified dependency path extraction, preserving Maven coordinates and dependency scopes.
FR2	OsvVulnerabilityAdapterTest	Verified vulnerability retrieval for the correct Maven coordinates and resolved dependency versions.
FR3	EpssAdapterTest	Verified EPSS score association by CVE, including aliases, unavailable scores, invalid values, and service failures.
FR4	WalaReachabilityAdapterTest, WalaServiceLoaderGuardTest, AnalyseDependencyRiskServiceTest	Verified reachability classifications using WALA call graphs built from compiled Java applications and libraries, including handling of unavailable and service-provider declarations.
FR5	JapicmpCompatibilityAdapterTest	Verified API compatibility classifications between compiled JAR versions, including source and binary incompatibilities and unavailable artifacts.
FR6	GoblinWeaverStabilityAdapterTest GoblinWeaverStabilityMapperTest StabilityUnavailabilityTest	Verified ecosystem stability retrieval and metric mapping, including the handling of unavailable, invalid, and zero-value indicators.
FR7	RemediationCandidateTest ContextualRiskFactorsTest RiskWeightsTest	Verified contextual risk calculation, including signal weighting, reachability, staleness, path-quality factors, and aggregation of vulnerability scores.
FR8	Z3RemediationAdapterTest RemediationCandidateTest	Verified optimal remediation selection under effort budget and mandatory-upgrade constraints, including infeasible solutions and solver failures.
FR9	CycloneDxVexAdapterTest	Verified CycloneDX 1.7 schema compliance, vulnerability coverage, exploitability status mapping, and component associations.
FR10	RiskReportAssemblerTest ReportGeneratorAdapterTest	Verified that exported reports include recommended updates, contextual risk scores, and supporting information.

The functional tests covered the main analysis stages and the treatment of unavailable or invalid information. The OSV and EPSS adapters were tested using predefined API responses. For reachability analysis, compatibility assessment, and remediation optimisation, the tests used WALA, JApiCmp, and Z3, respectively. These tests verified expected behaviour under controlled conditions.

Table 14 - Quality requirements testing.

Requirement	Test classes	Tests and results
QR1	OsvVulnerabilityAdapterTest	Verified 100% recall against deterministic OSV responses, including paginated results.
QR2	Evaluation execution records	Three Maven projects with 52-74 dependencies recorded execution times ranging from 50.2 to 72.6 seconds, below the 300-second threshold.
QR3	ModularityArchitectureTest	Four ArchUnit tests verified compliance with the dependency constraints between architectural layers and selected external libraries. All four passed.
QR4	RiskReportAssemblerTest ReportGeneratorAdapterTest	Verified that reported risk scores can be reconstructed from individual factors and effective weights.

As an example, Code 3 presents an architectural test developed to assess QR3. The test uses ArchUnit to verify that the application layer does not depend directly on WALA, JApiCmp, or Z3.

```
@Test
void applicationShouldNotDependOnConcreteAnalysisLibraries() {
    noClasses()
            .that().resideInAPackage(APPLICATION_PACKAGE)
            .should().dependOnClassesThat()
            .resideInAnyPackage(
                    WALA_PACKAGE,
                    JAPICMP_PACKAGE,
                    Z3_PACKAGE)
            .check(PROJECT_CLASSES);
}
```

Code 3 - ArchUnit test verifying application-layer independence from external analysis libraries.

The test passed, confirming that the application layer has no direct dependencies on WALA, JApiCmp, or Z3. Together with the other architectural tests, it provides evidence of the hexagonal architecture described in Section 4.3.2. However, the replacement of individual external components was not tested directly.

Testing also supported the iterative refinement of the framework. Initial tests of the OSV adapter revealed that it did not account for pagination, causing vulnerabilities returned in subsequent API responses to be missed. The tests also identified issues with duplicate vulnerability records. The adapter was subsequently modified to retrieve all available pages and consolidate duplicate records. Regression tests verified that these issues were resolved and that retrieval failures were handled appropriately.

Further tests led to improvements in EPSS enrichment and compatibility classification, particularly in the validation of scores and the classification of compatibility changes. Remediation optimisation tests also revealed situations where mandatory upgrades could exceed the configured effort budget. The implementation was revised to identify infeasible cases rather than recommend updates that violate the budget constraint.

The HTML report was also reorganised to improve the presentation of remediation recommendations and supporting information. Automated tests verified that the exported reports retained relevant information. However, no formal usability evaluation was conducted.

Overall, testing contributed to improvements in vulnerability retrieval, data validation, compatibility classification, and remediation optimisation. Chapter 6 complements these tests by evaluating the framework on real-world Maven projects.

6 Evaluation

This chapter presents the experimental evaluation conducted to investigate RQ2 (described in Section 1.3). The evaluation was structured as a comparative experiment on real-world Maven projects, as detailed in Section 6.1. The experimental scripts, project definitions, and raw outputs are publicly available in the project repository.

6.1 Evaluation Design

The evaluation considered three experimental configurations, summarised in Table 15: OSV-Scanner without Java call analysis (C1), OSV-Scanner with Java call analysis enabled (C2), and the proposed framework (C3).

Table 15 – Experimental configurations used in evaluation.

Configuration	Description	Role
C1	OSV-Scanner without Java call analysis	Vulnerability-reporting baseline
C2	OSV-Scanner with Java call analysis	Reachability-aware baseline
C3	CRBF	Proposed framework

The comparison between C1 and C2 focuses on the effect of introducing static reachability analysis. Both configurations use the same OSV-Scanner version and input artefact, with C2 additionally enabling Java call analysis. The comparison examines the vulnerabilities reported by both configurations, the proportion for which C2 provides reachability information, and the distribution of the resulting classifications.

C2 is subsequently compared with C3, as both configurations provide reachability information. This comparison identifies vulnerabilities reported by both approaches and those reported exclusively by either configuration. For common vulnerabilities, their reachability classifications are compared to identify agreements and differences. The CRBF output is then examined to assess the additional information available to support dependency updates, including exploitation likelihood, update compatibility, ecosystem information, contextual risk, and remediation recommendations.

The OSV-Scanner configurations analyse the packaged JAR, whereas C3 resolves the Maven dependency graph, potentially resulting in different dependency inventories. To account for this, an additional control was performed using a CycloneDX SBOM generated for each project. Each SBOM was scanned with OSV-Scanner, and the results were compared with those of C3 to determine whether both approaches identified the same vulnerabilities.

6.2 Experimental Setup and Dataset Selection

The experiments were conducted on an Apple MacBook Air with an M5 processor and 16 GB of unified memory, running macOS 26.6.2. The evaluation used OpenJDK 17, Apache Maven 3.9.16, and OSV-Scanner 2.7.0. CycloneDX Maven Plugin 2.9.1 was used to generate SBOMs for the additional control described in the previous section.

For C3, the effort budget was fixed at 30 units across all projects. Dependency resolution included compile and runtime scopes, excluding test, provided, and system dependencies. The call graph analysis was configured with a timeout of 600 seconds, while each execution step had a timeout of 1800 seconds.

The evaluation relied on external data from OSV for vulnerability records, EPSS for exploitation likelihood, and Goblin Weaver for ecosystem metrics and dependency graph information. The experiments were conducted in October 2026. Changes to these external data sources may affect the results of subsequent executions.

The dataset comprised six open-source Java projects hosted on GitHub. Candidate projects were assessed based on their build configuration, availability of packaged Java bytecode, and reproducibility. The selection also considered projects with different numbers of resolved dependencies to assess how the three configurations behave across varying dependency graph complexity. The selection criteria are presented in Table 16.

Table 16 - Project selection criteria.

ID	Criterion
IC1	Built with Apache Maven using a valid `Pom.xml`.
IC2	Produces a JAR containing compiled Java bytecode suitable for analysis.
IC3	Can be built successfully with Maven without requiring changes to the project's source code.
IC4	Publicly available at a fixed Git commit.

IC5 Has a dependency graph with a maximum of 100 direct and transitive dependencies.

These criteria ensure that the selected projects can be built and analysed by all three configurations, while allowing the experiments to be reproduced. The limit of 100 dependencies was established to facilitate manual examination of the results when necessary.

The selection process involved manually examining candidate repositories, their Maven build configurations, and the resulting packaged artefacts. A total of 22 candidate projects were examined during the selection process. Sixteen were excluded: six used Gradle or Ant instead of Maven, three failed to compile, one had no runtime dependencies, and six were excluded due to additional build requirements. The six projects selected for the evaluation are presented in Table 17.

Table 17 - Selected projects and their resolved dependency counts.

Project	Resolved Dependencies
Spring Security JWT	74
My-Blog	52
WebSocket Demo	63
SpecialSource	18
Checkstyle	35
Heros	7

Each project was evaluated at a fixed Git commit using all three experimental configurations. The experiments were automated through scripts that recorded the execution commands, tool versions, execution times, and raw outputs. The results were subsequently processed to support comparisons between configurations.

6.3 Results and Analysis

This section presents the results obtained from the three experimental configurations. The analysis examines the contribution of Java call analysis, differences in dependency coverage and reachability classifications, and the additional contextual information provided by CRBF for vulnerability prioritisation and remediation planning. Execution times are also compared across configurations.

6.3.1 Impact of Java Call Analysis (C1 vs. C2)

Across the six selected projects, C1 and C2 reported identical sets of vulnerability occurrences, totalling 237. Enabling Java call analysis did not change the vulnerabilities reported by OSV-Scanner but added reachability information. As shown in Table 18, C2 classified 203 occurrences as `called:true` and 34 as `called:false`.

Table 18 - Vulnerability occurrences and call analysis results for C1 and C2.

Project	C1/C2 Occurrences	Called: True	Called: False
Spring Security JWT	39	38	1
My-Blog	59	52	7
WebSocket Demo	125	102	23
SpecialSource	5	3	2
Checkstyle	6	5	1
Heros	3	3	0
Total	237	203	34

Most occurrences classified as `called:false` were identified in WebSocket Demo (23) and My-Blog (7). OSV-Scanner's call analysis determines whether classes within an affected dependency are reachable but does not establish whether the specific code responsible for a vulnerability can be reached. The annotations therefore provide an initial indication of reachability without confirming that the vulnerable code itself is reachable.

6.3.2 Dependency Inventory and Vulnerability Detection Control

C3 reported 423 vulnerability occurrences, including all 237 identified by C2 and 186 additional occurrences. A vulnerability occurrence represents a vulnerability associated with a dependency in a particular project. Most of the additional occurrences were identified in Spring Security JWT, My-Blog, and WebSocket Demo.

The two configurations use different dependency inventories: C2 analyses the packaged JAR, while C3 resolves dependencies through Maven. To determine whether these differences originated from dependency identification or vulnerability detection, the SBOM-based control described in Section 6.1 was used.

The SBOM-based analysis produced the same dependency inventories and vulnerability occurrences as C3 across all six projects. Table 19 compares the results obtained from JAR-based analysis (C1/C2) with those obtained from Maven-based analysis (C3 and the SBOM control). The final column indicates the additional vulnerability occurrences identified through the Maven-based inventory.

Table 19 - Dependency inventories and vulnerability occurrences by input type.

Project	JAR Dependencies	Maven Dependencies	Jar Occurrences	Maven Occurrences	Difference
Spring Security JWT	66	74	39	93	+54
My-Blog	49	52	59	122	+63
WebSocket Demo	57	63	125	194	+69
SpecialSource	15	18	5	5	0
Checkstyle	32	35	6	6	0
Heros	6	7	3	3	0

Total	225	249	237	423	+186

The Maven-based inventories contained 249 dependency entries, compared with 225 identified through JAR analysis. All 186 additional vulnerability occurrences were concentrated in three projects. The remaining projects reported identical vulnerability counts despite differences in their dependency inventories.

The differences were not limited to the number of dependencies identified. Some dependencies were also represented using different Maven coordinates. For example, in My-Blog, OSV-Scanner identified MySQL Connector/J using `com.mysql.cj:mysql-connector-j`, whereas Maven resolved it as com.mysql:mysql-connector-j. Such differences can affect how dependencies are matched against vulnerability records.

The agreement between the C3 and SBOM-based analysis shows that both approaches retrieved the same vulnerabilities when using equivalent dependency inventories. The higher number of occurrences reported by C3 therefore reflects differences in dependency identification and coverage, rather than superior vulnerability detection accuracy.

6.3.3 Contextual Reachability and Prioritisation (C2 vs. C3)

The reachability comparison considered the 237 vulnerability occurrences reported by both C2 and C3. Table 20 presents the relationship between the call analysis annotations provided by C2 and the reachability classifications produced by C3.

Table 20 - Comparison of C2 call-analysis annotations and C3 reachability classifications.

C2 Classification	Reachable Confirmed	Reachable Probable	Unknown	Unreachable	Total
called: true	13	76	34	80	203
called: false	0	0	0	34	34
Total	13	76	34	114	237

The main differences between C2 and C3 lie in the granularity of their reachability assessments. OSV-Scanner's JAR analysis determines whether dependency classes are reachable from the application and applies this information to the associated vulnerabilities, without identifying whether specific vulnerable methods are reachable. CRBF, in contrast, constructs a method-level call graph and distinguishes between confirmed and probable reachability, unreachability, and cases where the analysis is inconclusive. Among the 203 occurrences annotated as `called:true` by C2, CRBF classified 80 (39.4%) as unreachable and 34 (16.7%) as unknown. The remaining 89 received confirmed or probable reachability classifications. These results show that CRBF provides more differentiated information for vulnerability triage, allowing developers to distinguish between vulnerabilities with different levels of reachability evidence.

CRBF also incorporates vulnerability severity, exploitation likelihood, ecosystem information, and project-specific factors into its contextual risk scores. To illustrate the impact of contextual risk assessment on prioritisation, Table 21 compares two vulnerabilities identified in My-Blog.

Table 21 - Example of contextual vulnerability prioritisation in My-Blog.

Vulnerability	CVSS	EPSS	Contextual Risk
CVE-2024-38819	7.5	0.56	0.74
CVE-2023-42276	9.8	0.01	0.69

Although CVE-2024-38819 has a lower CVSS score than CVE-2023-42276, it receives a higher risk score due to its greater exploitation likelihood. Both vulnerabilities have identical reachability, path-quality, and staleness factors. This example demonstrates how incorporating EPSS into the risk assessment can change the relative priority of vulnerabilities compared with CVSS alone.

6.3.4 Remediation Recommendations and Validation

While C2 reports identified vulnerabilities alongside severity and call-analysis information, CRBF extends this assessment by producing a remediation plan based on contextual risk and estimated upgrade effort. The plan selects a subset of dependency updates within a configurable effort budget.

Across the six projects, CRBF identified 83 vulnerable dependencies and selected 40 updates, comprising 11 classified as `Mandatory` and 29 as `Recommended`. The remaining 43 dependencies received a DEFER decision. Table 22 presents the distribution of these decisions and the estimated effort used by each project.

Table 22 - Remediation decisions and estimated effort by project.

Project	Mandatory	Recommended	Deferred	Effort Used / Budget
Spring Security JWT	3	7	9	30/30
My-Blog	4	5	10	30/30
WebSocket Demo	4	6	24	30/30
SpecialSource	0	4	0	11/30
Checkstyle	0	6	0	18/30
Heros	0	1	0	4/30
Total	11	29	43	-

The optimiser returned an optimal solution for all six projects. Spring Security JWT, My-Blog, and WebSocket Demo used their full effort budgets, leaving some vulnerable dependencies deferred. In contrast, the remaining projects selected updates for all their reported vulnerable dependencies while staying below the budget limit.

Compatibility analysis also influenced the estimated effort of individual updates. In My-Blog, JApiCmp identified binary-incompatible API changes when updating `jackson-core` from version 2.13.4 to 2.18.11. However, CRBF found that none of the affected methods appeared in the project's call graph. The update was therefore classified as compatible in the project context, reducing its estimated effort from four units to one.

After selecting the updates, CRBF validates the resulting dependency graph to check whether they introduce new vulnerable transitive dependencies. In Heros, the framework recommended updating Guava from 22.0 to 32.0.0-android. The validation found no known vulnerable dependencies, but JApiCmp identified binary-incompatible API changes between the two versions. This example illustrates that an update may resolve known vulnerabilities while still introducing compatibility concerns that developers need to consider.

Overall, the results show that CRBF extends vulnerability reporting by producing budget-constrained remediation plans, including selected updates, estimated effort cost, and dependency graph validation when sufficient data is available.

6.3.5 Execution Time Comparison

Table 23 presents the execution times recorded for each configuration across the six evaluated projects.

Table 23 - Execution times by project and configuration in seconds.

Project	C1	C2	C3
Spring Security JWT	1.36	7.53	68.60
My-Blog	1.37	4.90	55.69
WebSocket Demo	1.52	5.04	436.73
SpecialSource	0.88	2.02	13.40
Checkstyle	0.98	6.37	435.22
Heros	0.82	1.70	9.37

C1 consistently had the shortest execution times, followed by C2. CRBF required more time across all projects, as its execution involves retrieving ecosystem metrics, assessing update compatibility, and optimising remediation decisions.

Four projects completed C3 within the five-minute threshold defined by QR2 (Table 9). However, WebSocket Demo and Checkstyle exceed this limit. Execution time also varied considerably between projects, without a direct relationship to the number of dependencies analysed.

An important factor affecting execution time is Goblin Weaver's computation of ecosystem metrics. When metrics are requested for the first time, they must be calculated, whereas subsequent requests can reuse previously computed results through memoisation. This behaviour is consistent with the shorter execution times observed when the evaluation was repeated. However, the contribution of memoisation was not independently measured.

The results show that CRBF introduces additional execution overhead compared with OSV-Scanner and that its performance varies depending on the analysis conditions and the availability of cached ecosystem data.

6.4 Threats to Validity

The evaluation was conducted on six open-source Maven projects. Although these projects have different characteristics and dependency structures, the results may not generalise to other Java applications, particularly proprietary or enterprise systems. Additionally, the framework is designed for Maven, and the findings may not extend to package managers with different dependency resolution mechanisms.

The comparison between C2 and C3 is affected by differences in dependency inventory coverage, since OSV-Scanner analysed packaged JAR files, while CRBF resolved dependencies through Maven. The SBOM-based control showed that both tools identified the same vulnerabilities when using equivalent dependency inventories. This indicates that the differences in vulnerability counts resulted from dependency identification and coverage, rather than differences in vulnerability detection.

The reachability results are subject to the limitations of static analysis. Call graphs may not capture execution paths involving dynamic features, such as Java reflection, which can affect the accuracy of the resulting classifications.

Although the proposed updates were evaluated through compilation and testing, these checks may not detect all compatibility issues. The validation of updated dependency graphs was also constrained by the ecosystem data available through Goblin Weaver. Some newer dependency versions were missing from this dataset, preventing CRBF from resolving their transitive dependencies and determining whether the updated dependency graphs contained additional known vulnerabilities.

Execution times may be influenced by external service response times and the availability of previously computed ecosystem metrics. Although Goblin Weaver's memoisation mechanism allows previous results to be reused, its effect on execution time was not independently measured.

7 Conclusions

This chapter presents the main conclusions of the dissertation, summarising its findings and contributions in relation to the research objectives. It also discusses the difficulties encountered throughout the work and outlines possible directions for future work.

7.1 Achievements and Contributions

The main objective of this dissertation was to design, implement, and evaluate a risk-based framework to support dependency update decisions in Maven projects. To achieve this, a review of existing approaches was conducted, providing the foundation for the requirements and design of the proposed solution.

The resulting framework, CRBF, integrates vulnerability information, exploitation likelihood, static reachability analysis, ecosystem stability metrics, and binary compatibility assessment into a single analysis process. It also incorporates a constraint-based optimisation model to recommend dependency updates within a configurable effort budget. These contributions address RQ1 by demonstrating how security, stability, and project-specific information can be combined to support dependency update decisions.

To investigate RQ2, CRBF was evaluated against two configurations of OSV-Scanner across six open-source Maven projects. The results showed that CRBF provides more detailed reachability classifications and incorporates contextual factors that can influence vulnerability prioritisation. Of the 203 vulnerability occurrences classified as reachable by OSV-Scanner, CRBF classified 80 as unreachable, illustrating the finer distinctions provided by its reachability assessment. CRBF also produced remediation plans that prioritised dependency updates within a defined effort budget. The evaluation also revealed a performance trade-off, with CRBF requiring longer execution times than OSV-Scanner.

This work was also presented at the International Symposium on Ambient Intelligence and Embedded Systems (AmiEs 2026) and published in its proceedings [86].

Overall, the findings demonstrate the feasibility of integrating contextual risk assessment and budget-constrained remediation into Maven dependency management.

7.2 Difficulties

One of the main challenges was translating the findings of the literature review into a coherent and implementable framework. Combining vulnerability information, reachability, ecosystem stability, and compatibility required defining how each factor would contribute to risk assessment and remediation decisions. Selecting initial weights and thresholds was particularly challenging, as the literature did not provide values directly applicable to CRBF.

Implementing the framework required developing expertise in Maven dependency resolution, static analysis, and constraint-based optimisation. Reachability analysis was particularly demanding, as its configuration involved balancing analysis coverage against computational cost. Limited information about affected code in vulnerability advisories further complicated the classification of vulnerabilities.

Integrating external data sources introduced additional difficulties, particularly when vulnerability information or ecosystem metrics were incomplete or unavailable. Remediation optimisation also required addressing situations where mandatory updates exceeded the configured effort budget, which required explicitly identifying infeasible remediation plans.

Finally, the experimental comparison was complicated by differences between Maven's dependency resolution and OSV-Scanner's identification of packaged dependencies. An additional SBOM-based analysis was introduced to distinguish differences in dependency coverage from differences in vulnerability detection.

7.3 Future Work

Future work could investigate the calibration of the risk model's weights and thresholds using a broader set of projects and developer assessments. Sensitivity analysis could help establish how these parameters affect vulnerability prioritisation and remediation decisions. An adaptive calibration mechanism, potentially based on MAPE-K, could then use observed developer decisions to inform subsequent adjustments.

Reachability analysis could be improved by incorporating more precise mappings between vulnerabilities and affected methods, alongside better support for dynamically invoked code. Performance optimisation could also be investigated, particularly during call graph construction and ecosystem data retrieval, to reduce execution time while preserving analysis coverage.

The framework could also be extended to other build tools and package ecosystems, such as Gradle and npm. Its modular architecture could support alternative dependency resolvers and

data sources, although the analysis techniques and risk model would need to be reassessed for each ecosystem.

Finally, the evaluation could be extended to larger and more diverse Maven projects and include studies with developers. These studies could investigate whether contextualised reports improve the understanding of dependency risks, influence update decisions, and reduce remediation effort. This would complement the technical evaluation with evidence of CRBF's practical usefulness.

7.4 Final Considerations

This dissertation explored dependency updates as a decision problem involving security, project-specific exposure, and the effort required for remediation. The design, implementation, and evaluation of CRBF demonstrated the feasibility of bringing these dimensions together within a Maven plugin. The results illustrate how contextual information can support vulnerability prioritisation and update planning, while highlighting the additional computational cost and the limitations imposed by incomplete data and static analysis. Further evaluation is needed to establish its impact on developers' decisions in practice. CRBF provides a basis for this investigation and for the continued development of contextualised dependency management.

References

[1] D. Jaime, P. Poizat, J. El Haddad, and T. Degueule, “Balancing the Quality and Cost of Updating Dependencies,” vol. 1, Accessed: Oct. 01, 2025. [Online]. Available: https://hal.science/hal-04684254v1

[2] “TIOBE Index - TIOBE.” Accessed: May 10, 2026. [Online]. Available: https://www.tiobe.com/tiobe-index/

[3] “Welcome to Apache Maven – Maven.” Accessed: Dec. 04, 2025. [Online]. Available: https://maven.apache.org/

[4] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, “Do developers update their library dependencies?,” *Empir. Softw. Eng.*, vol. 23, no. 1, pp. 384–417, Feb. 2018, doi: 10.1007/s10664-017-9521-5.

[5] F. Reyes, “BUMP: A Benchmark of Reproducible Breaking Dependency Updates”, Accessed: Oct. 01, 2025. [Online]. Available: https://github.com/chains-project/bump.

[6] H. Rebatchi, T. F. Bissyandé, and N. Moha, “Dependabot and security pull requests: large empirical study,” *Empirical Software Engineering, 29 (5) (2024-09*, vol. 29, no. 5, Sep. 2024, doi: 10.1007/S10664-024-10523-Y.

[7] J. vom Brocke, A. Hevner, and A. Maedche, “Introduction to Design Science Research,” 2020, pp. 1–13. doi: 10.1007/978-3-030-46781-4_1.

[8] D. Gotterbarn, K. Miller, and S. Rogerson, “Software engineering code of ethics,” *Commun. ACM*, vol. 40, no. 11, 1997, doi: 10.1145/265684.265699.

[9] Instituto Politécnico do Porto, “Regulamento do Código de Boas Práticas e de Conduta do Instituto Politécnico do Porto (Despacho n.º 11171/2020),” 2020.

[10] “General Data Protection Regulation (GDPR) – Legal Text.” Accessed: Dec. 11, 2025. [Online]. Available: https://gdpr-info.eu/

[11] P. Office of the European Union L- and L. Luxembourg, “REGULATION (EU) 2024/2847 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) No 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act) (Text with EEA relevance)”, Accessed: Jan. 07, 2026. [Online]. Available: http://data.europa.eu/eli/reg/2024/2847/oj

[12] “The European Code of Conduct for Research Integrity REVISED EDITION 2023”, doi: 10.26356/ECOC.

[13] D. Almeida, “CRBF-Project repositories.” Accessed: Jul. 25, 2026. [Online]. Available: https://github.com/orgs/CRBF-Project/repositories

[14] “Web Content Accessibility Guidelines (WCAG) 2.2.” Accessed: Oct. 02, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/#use-of-color

[15] “Maven Artifacts – Maven.” Accessed: Dec. 04, 2025. [Online]. Available: https://maven.apache.org/repositories/artifacts.html

[16] “Introduction to the Dependency Mechanism – Maven.” Accessed: Dec. 05, 2025. [Online]. Available: https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html

[17] C. Nachuma, M. M. Hossan, A. K. Turzo, and M. Zibran, “Decoding Dependency Risks: A Quantitative Study of Vulnerabilities in the Maven Ecosystem,” May 2025, pp. 270–274. doi: 10.1109/MSR66628.2025.00048.

[18] S. Raemaekers, A. van Deursen, and J. Visser, “Semantic versioning and impact of breaking changes in the Maven repository,” *Journal of Systems and Software*, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008.

[19] “Technical Paper: Software Supply Chain Security - ECSO.” Accessed: Dec. 05, 2025. [Online]. Available: https://ecs-org.eu/?publications=technical-paper-software-supply-chain-security

[20] “CycloneDX Bill of Materials Standard | CycloneDX.” Accessed: Dec. 05, 2025. [Online]. Available: https://cyclonedx.org/

[21] “Minimum Requirements for Vulnerability Exploitability eXchange (VEX)”, Accessed: Aug. 02, 2026. [Online]. Available: http://www.cisa.gov/tlp/.

[22] “What Is Software Composition Analysis (SCA)? - Palo Alto Networks.” Accessed: Dec. 05, 2025. [Online]. Available: https://www.paloaltonetworks.com/cyberpedia/what-is-sca

[23] “What Is Risk-Based Vulnerability Management? - Palo Alto Networks.” Accessed: Jan. 07, 2026. [Online]. Available: https://www.paloaltonetworks.com/cyberpedia/risk-based-vulnerability-management

[24] “CVE: Common Vulnerabilities and Exposures.” Accessed: Dec. 05, 2025. [Online]. Available: https://www.cve.org/

[25] “CVE JSON record format.” Accessed: Mar. 10, 2026. [Online]. Available: https://cveproject.github.io/cve-schema/schema/docs/

[26] “CWE - Common Weakness Enumeration.” Accessed: Dec. 05, 2025. [Online]. Available: https://cwe.mitre.org/

[27] “Common Vulnerability Scoring System SIG.” Accessed: Dec. 05, 2025. [Online]. Available: https://www.first.org/cvss/

[28] “Common Vulnerability Scoring System.” Accessed: Jan. 07, 2026. [Online]. Available: https://www.first.org/cvss/v4-0/

[29] J. Jacobs, S. Romanosky, O. Suciu, B. Edwards, and A. Sarabi, “Enhancing Vulnerability Prioritization: Data-Driven Exploit Predictions with Community-Driven Insights,” in *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2023, pp. 194–206. doi: 10.1109/EuroSPW59978.2023.00027.

[30] “Exploit Prediction Scoring System (EPSS).” Accessed: Jan. 07, 2026. [Online]. Available: https://www.first.org/epss/

[31] “NVD - Home.” Accessed: May 03, 2026. [Online]. Available: https://nvd.nist.gov/

[32] “About the GitHub Advisory database - GitHub Docs.” Accessed: Jan. 07, 2026. [Online]. Available: https://docs.github.com/en/code-security/security-advisories/working-with-global-security-advisories-from-the-github-advisory-database/about-the-github-advisory-database?learn=security_advisories&learnProduct=code-security

[33] “GitHub GraphQL API documentation - GitHub Docs.” Accessed: May 03, 2026. [Online]. Available: https://docs.github.com/en/graphql

[34] “OSV - Open Source Vulnerabilities.” Accessed: May 03, 2026. [Online]. Available: https://osv.dev/

[35] “OWASP Risk Rating Methodology | OWASP Foundation.” Accessed: Jul. 20, 2026. [Online]. Available: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[36] H. Taherdoost and M. Madanchian, “Multi-Criteria Decision Making (MCDM) Methods and Concepts,” *Encyclopedia*, vol. 3, no. 1, pp. 77–87, 2023, doi: 10.3390/encyclopedia3010006.

[37] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *Computer (Long. Beach. Calif).*, vol. 36, no. 1, pp. 41–50, Jan. 2003, doi: 10.1109/MC.2003.1160055.

[38] J. Stringer, A. Tahir, K. Blincoe, and J. Dietrich, “Technical lag of dependencies in major package managers,” *Proceedings - Asia-Pacific Software Engineering Conference, APSEC*, vol. 2020-December, pp. 228–237, Dec. 2020, doi: 10.1109/APSEC51365.2020.00031.

[39] B. Berretta, A. Thomas, and H. Guarnera, “Dependency Update Adoption Patterns in the Maven Software Ecosystem,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, IEEE, Apr. 2025, pp. 295–299. doi: 10.1109/MSR66628.2025.00053.

[40] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, “The Java ® Language Specification Java SE 8 Edition,” 1997.

[41] “Clirr - Clirr.” Accessed: May 03, 2026. [Online]. Available: https://clirr.sourceforge.net/

[42] “Revapi :: Revapi.” Accessed: May 03, 2026. [Online]. Available: https://revapi.org/revapi-site/main/index.html

[43] “japicmp – japicmp-base.” Accessed: May 03, 2026. [Online]. Available: https://siom79.github.io/japicmp/

[44] D. Jayasuriya, S. Ou, S. Hegde, V. Terragni, J. Dietrich, and K. Blincoe, “An extended study of syntactic breaking changes in the wild,” *Empir. Softw. Eng.*, vol. 30, no. 2, p. 42, Mar. 2025, doi: 10.1007/s10664-024-10563-4.

[45] D. Jaime, J. El Haddad, and P. Poizat, “Goblin: A Framework for Enriching and Querying the Maven Central Dependency Graph,” in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 2024, pp. 37–41.

[46] D. Jaime, J. El Haddad, and P. Poizat, “Goblin-Weaver,” Dec. 2023. doi: 10.5281/zenodo.10291589.

[47] “Static reachability analysis | GitLab Docs.” Accessed: Dec. 05, 2025. [Online]. Available: https://docs.gitlab.com/user/application_security/dependency_scanning/static_reachability/

[48] A. R. F. J. Hejderup, A. Van Deursen, and G. Gousios, “Software Ecosystem Call Graph for Dependency Management,” 2018, doi: 10.1145/3183399.3183417.

[49] K. Karakaya *et al.*, “SootUp: A Redesign of the Soot Static Analysis Framework,” in *Tools and Algorithms for the Construction and Analysis of Systems*, 2024, pp. 229–247. doi: 10.1007/978-3-031-57246-3_13.

[50] “wala/WALA: T.J. Watson Libraries for Analysis, with front ends for Java, Android, and JavaScript, and many common static program analyses.” Accessed: May 03, 2026. [Online]. Available: https://github.com/wala/wala

[51] “Incorporate Qilin Pointer Analysis.” Accessed: Aug. 12, 2026. [Online]. Available: https://soot-oss.github.io/SootUp/latest/qilin/

[52] M. Reif, F. Kübler, M. Eichberg, D. Helm, and M. Mezini, “Judge: identifying, understanding, and evaluating sources of unsoundness in call graphs,” in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, in ISSTA 2019. New York, NY, USA: Association for Computing Machinery, 2019, pp. 251–261. doi: 10.1145/3293882.3330555.

[53] A. G. Márquez, Á. J. Varela-Vaca, M. T. Gómez López, J. A. Galindo, and D. Benavides, “Depex: A software for analysing and reasoning about vulnerabilities in software projects dependencies,” *SoftwareX*, vol. 30, p. 102152, May 2025, doi: 10.1016/J.SOFTX.2025.102152.

[54] “Z3Prover/z3: The Z3 Theorem Prover.” Accessed: May 03, 2026. [Online]. Available: https://github.com/z3prover/z3

[55] L. Zhao *et al.*, “Software Composition Analysis for Vulnerability Detection: An Empirical Study on Java Projects,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, New York, NY, USA: ACM, Nov. 2023, pp. 960–972. doi: 10.1145/3611643.3616299.

[56] M. J. Page *et al.*, “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, Mar. 2021, doi: 10.1136/BMJ.N71.

[57] “ScienceDirect.com | Science, health and medical journals, full text articles and books.” Accessed: Nov. 22, 2025. [Online]. Available: https://www.sciencedirect.com/

[58] “IEEE Xplore.” Accessed: Nov. 22, 2025. [Online]. Available: https://ieeexplore.ieee.org/Xplore/guesthome.jsp

[59] “ACM Digital Library.” Accessed: Nov. 22, 2025. [Online]. Available: https://dl.acm.org/

[60] “b-on.” Accessed: Dec. 07, 2025. [Online]. Available: https://www.b-on.pt/

[61] “Dependabot.” Accessed: Jul. 22, 2026. [Online]. Available: https://github.com/dependabot

[62] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness, “PRISMA2020: An R package and Shiny app for producing PRISMA 2020-compliant flow diagrams, with interactivity for optimised digital transparency and Open Synthesis,” *Campbell Systematic Reviews*, vol. 18, no. 2, p. e1230, Jun. 2022, doi: https://doi.org/10.1002/cl2.1230.

[63] Y. Wu, Z. Yu, M. Wen, Q. Li, D. Zou, and H. Jin, “Understanding the Threats of Upstream Vulnerabilities to Downstream Projects in the Maven Ecosystem,” in *Proceedings of the 45th International Conference on Software Engineering*, in ICSE ’23. IEEE Press, 2023, pp. 1046–1058. doi: 10.1109/ICSE48619.2023.00095.

[64] J. Dietrich, S. Rasheed, A. Jordan, and T. White, “On the Security Blind Spots of Software Composition Analysis,” in *Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, in SCORED ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 77–87. doi: 10.1145/3689944.3696165.

[65] Z. Ma, S. Mondal, T.-H. (Peter) Chen, H. Zhang, and A. E. Hassan, “VulNet: Towards improving vulnerability management in the Maven ecosystem,” *Empir. Softw. Eng.*, vol. 29, no. 4, p. 83, 2024, doi: 10.1007/s10664-024-10448-6.

[66] J. Sun, J. Chen, Z. Xing, Q. Lu, X. Xu, and L. Zhu, “Where is it? Tracing the Vulnerability-relevant Files from Vulnerability Reports,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, in ICSE ’24. New York, NY, USA: Association for Computing Machinery, 2024. doi: 10.1145/3597503.3639202.

[67] X. Wang, R. Hu, C. Gao, X.-C. Wen, Y. Chen, and Q. Liao, “ReposVul: A Repository-Level High-Quality Vulnerability Dataset,” in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, in ICSE-Companion ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 472–483. doi: 10.1145/3639478.3647634.

[68] S. E. Ponta, H. Plate, A. Sabetta, M. Bezzi, and C. Dangremont, “A manually-curated dataset of fixes to vulnerabilities of open-source software,” in *Proceedings of the 16th International Conference on Mining Software Repositories*, in MSR ’19. IEEE Press, 2019, pp. 383–387. doi: 10.1109/MSR.2019.00064.

[69] F. Zhang, L. Fan, S. Chen, M. Cai, S. Xu, and L. Zhao, “Does the Vulnerability Threaten Our Projects? Automated Vulnerable API Detection for Third-Party Libraries,” *IEEE Transactions on Software Engineering*, vol. 50, no. 11, pp. 2906–2920, Nov. 2024, doi: 10.1109/TSE.2024.3454960.

[70] M. Keshani, S. Vos, and S. Proksch, “On the relation of method popularity to breaking changes in the Maven ecosystem,” *Journal of Systems and Software*, vol. 203, p. 111738, Sep. 2023, doi: 10.1016/J.JSS.2023.111738.

[71] Y. Zhao, Y. Zhang, D. Chacko, and J. Cappos, “CovSBOM: Enhancing Software Bill of Materials with Integrated Code Coverage Analysis,” *Proceedings - International Symposium on Software Reliability Engineering, ISSRE*, pp. 228–237, 2024, doi: 10.1109/ISSRE62328.2024.00031.

[72] Z. Jia, C. Yang, P. Feng, X. Zhao, X. Li, and J. Ma, “Impact assessment of third-party library vulnerabilities through vulnerability reachability analysis,” *Comput. Secur.*, vol. 157, p. 104546, Oct. 2025, doi: 10.1016/j.cose.2025.104546.

[73] I. Pashchenko, D.-L. Vu, and F. Massacci, “A Qualitative Study of Dependency Management and Its Security Implications,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, New York, NY, USA: ACM, Oct. 2020, pp. 1513–1531. doi: 10.1145/3372297.3417232.

[74] K. Huang *et al.*, “Characterizing usages, updates and risks of third-party libraries in Java projects,” *Empirical software engineering : an international journal*, vol. 27, no. 4, 2022.

[75] L. Zhang *et al.*, “Compatible Remediation on Vulnerabilities from Third-Party Libraries for Java Projects,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, in ICSE ’23. IEEE, May 2023, pp. 2540–2552. doi: 10.1109/ICSE48619.2023.00212.

[76] Google, “OSV-Scanner Changelog.” Accessed: Sep. 27, 2026. [Online]. Available: https://github.com/google/osv-scanner/blob/main/CHANGELOG.md

[77] “Reachability analysis | Scan, fix, and prevent | Snyk User Docs.” Accessed: Sep. 27, 2026. [Online]. Available: https://docs.snyk.io/scan-fix-and-prevent/fix/prioritize-issues-for-fixing/reachability-analysis

[78] S. E. Ponta, H. Plate, and A. Sabetta, “Detection, assessment and mitigation of vulnerabilities in open source dependencies,” *Empir. Softw. Eng.*, vol. 25, no. 5, pp. 3175–3215, Sep. 2020, doi: 10.1007/s10664-020-09830-x.

[79] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, “Vulnerable open source dependencies: counting those that matter,” in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, in ESEM ’18. New York, NY, USA: Association for Computing Machinery, 2018. doi: 10.1145/3239235.3268920.

[80] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, “Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies,” *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592–1609, May 2022, doi: 10.1109/TSE.2020.3025443.

[81] H. Mohayeji, A. Agaronian, E. Constantinou, N. Zannone, and A. Serebrenik, “Securing dependencies: A comprehensive study of Dependabot’s impact on vulnerability mitigation,” *Empir. Softw. Eng.*, vol. 30, no. 3, p. 89, May 2025, doi: 10.1007/s10664-025-10638-w.

[82] Y. David *et al.*, “UPGRADVISOR: Early Adopting Dependency Updates Using Hybrid Program Analysis and Hardware Tracing,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA: USENIX Association, Jul. 2022, pp. 751–767. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/david

[83] “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[84] “ISO/IEC 25010:2023(en), Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.” Accessed: Jul. 24, 2026. [Online]. Available: https://www.iso.org/obp/ui/en/#iso:std:iso-iec:25010:ed-2:v1:en

[85] T. Hombergs, *Get Your Hands Dirty on Clean Architecture*, 2nd ed. Packt Publishing, 2023.

[86] D. Rodrigues Almeida and I. Azevedo, “A Contextualised Risk-Based Framework for Maven Dependency Updates,” in *Proceedings of the International Symposium on Ambient Intelligence and Embedded Systems (AmiEs 2026)*, Porto, Sep. 2026. Accessed: Oct. 17, 2026. [Online]. Available: https://international-symposium.org/amies_2026/proceedings_2026/Almeida_Azevedo_AmiEs_2026_Paper.pdf

Appendix A

Skills Management

This chapter outlines the skills management strategy and the project plan established to ensure the feasibility and successful completion of the thesis. Section A.1 Identification of Required Skills details the identification and assessment of required skills, along with targeted strategies for developing the necessary competencies. Section A.2 Assessment of Current Skills defines the planning elements, including the project charter, scope of work, integrated schedule, costs, and risk management.

A.1 Identification of Required Skills

To achieve the objectives established in Section 1.3, particularly the design of a risk-based framework, specific technical and soft skills are required. Table 24 presents a comprehensive analysis of these requirements, specifying the proficiency levels necessary to execute the dissertation relative to the current skill baseline (rated on a scale of 1 to 5), and identifying the corresponding gaps that must be addressed.

Table 24 - Required Skills.

Skill	Required Proficiency (1-5)	Current Proficiency (1-5)	Gap	Comments
Java and Maven Ecosystem	5	3	Medium	Solid understanding of standard Maven and Java usage, though internal dependency resolution mechanisms and complex POM inheritance structures require further study.
Static Analysis	5	2	High	Basic theoretical knowledge, with limited practical application. Significant development is required in generating and traversing CGs for context analysis.
Formal Methods (SMT)	4	1	High	No prior experience with SMT. Acquisition of knowledge regarding the encoding of dependency constraints into logical formulas is necessary.
Data Mining and APIs	4	3	Low	Proficiency in API interaction. The programmatic processing of large-scale stability metrics requires additional practice.
Vulnerability Analysis and SCA	5	3	Medium	Good understanding of current SCA tools (e.g., OWASP Dependency-Check) with practical experience. Basic capability to interpret vulnerability standards, with limited exposure to complex CVSS vector analysis and the programmatic filtering of false positives.
Systematic Research	4	2	Medium	Competent in general research; the PRISMA methodology for the systematic review requires deep study.
Communication	4	2	Medium	Clear verbal expression; however, translating complex technical information into accessible concepts for non-technical stakeholders needs significant improvement.
Critical Thinking	5	4	Low	Strong analytical capability.
Scientific Writing	5	3	Medium	Functional writing competency; however, the level of technical precision and structural coherence required for a thesis needs considerable refinement.

A.2 Assessment of Current Skills

The evaluation of the competencies detailed in section A.1 Identification of Required Skills provides a clear understanding of the project´s feasibility. This assessment synthesises the identified proficiency levels to distinguish between strengths and gaps requiring intervention:

- **Strengths:** A solid proficiency in the Java and Maven ecosystem provides the necessary starting point for the project's development, complemented by a good understanding of current SCA tools and vulnerability standards. In terms of behavioural skills, the project benefits from strong critical thinking capabilities, particularly in technical analysis.
- **Gaps and Areas for Development:** The limited experience with SMT and Static Reachability Analysis is a notable barrier, as these are vital for the intended decision engine. Moreover, while there is competence in interacting with APIs, a significant gap remains in the ability to programmatically process large-scale stability metrics from ecosystem repositories. Additionally, there is a need to deepen the understanding of Maven's internal resolution mechanisms and the interpretation of CVSS vectors for false-positive filtering. Finally, adherence to the PRISMA research methodology, refinement of formal scientific writing, and translation of findings for non-technical stakeholders require training to ensure the project's success.

A.3 Strategies for Skill Development

To address the identified gaps and ensure the project's feasibility, an action plan based on the SMART methodology (Specific, Measurable, Achievable, Relevant, Time-Bound) has been formulated. The following strategies are designed to secure the successful execution of the dissertation:

- **Static Reachability Analysis:** Conduct a theoretical study of Java bytecode analysis libraries to overcome the limited experience in generating CGs. Consolidate the theoretical studies with the development of a basic call-graph generator capable of mapping method invocations within a simple JAR file. This is scheduled for completion within three weeks.
- **Formal Methods:** Successfully encode a simplified dependency version conflict into logical constraints using an SMT solver. This competency is allocated a four-week learning period.
- **Data Mining and Large-Scale API Processing:** Develop an automated script that queries the Maven Central API to extract adoption trends for a dataset of fifty artefacts within three weeks.
- **Dependency Resolution and Risk Metrics:** Analyse the documentation of Maven's dependency resolution mechanism to understand the origins of conflicts. Simultaneously, audit a generated report containing over thirty CVEs to identify false

positives and enrich the understanding of CVSS metrics. This task, intended to enhance heuristic filtering skills, is scheduled for completion within two weeks.

- **Methodological Rigour and Communication:** Analyse the structure of five high-impact papers in the field of software security to identify best practices regarding the PRISMA methodology. Incorporate iterative feedback sessions into the thesis drafting process to practice translating complex technical findings for non-specialist audiences.

A.4 Evaluation and Control Mechanisms

The effectiveness of the skill development plan detailed in section A.3 Strategies for Skill Development is verified through three specific control mechanisms, ensuring alignment with the project's requirements.

The acquisition of technical skills is evidenced by the functional success in developing the proposed artefacts. The accurate generation of a CG from a test JAR and the effective resolution of a dependency conflict by the SMT engine serve as objective criteria to confirm the proper implementation of the plan.

Academic and methodological adherence is monitored through iterative validation by the academic supervisor. Specifically, the formal approval of the State of the Art chapter ensures that the application of the PRISMA methodology and the standards of scientific writing meet the expected academic rigour.

Finally, a skills review was conducted prior to the implementation phase to serve as a reflective checkpoint for reassessing the proficiency levels outlined in Table 24. Specifically, it assessed whether the gap in Static Reachability Analysis has been sufficiently closed to proceed with the implementation phase. If a gap remains (below level 4), the mitigation strategy defined in the Risk Management Plan (Risk 1), presented in section B.6 Risk Identification and Management, is implemented to prevent project derailment.

Appendix B

Project Planning

This section details the project planning strategy designed to ensure the successful delivery of the dissertation. It establishes the project charter and scope of work to define the project boundaries, stakeholders and deliverables, followed by an integrated schedule to structure the execution timeline. In addition, it addresses the involved costs and outlines the monitoring and control mechanisms, concluding with a risk management plan to mitigate potential threats to the project success.

B.1 Project Charter

The project charter is a formal document that formalises the project's preliminary planning and guides its following steps. This document defines stakeholders, benefits, deliverables, constraints, and assumptions, thereby ensuring a systematic approach to achieving the objectives defined in section 1.3.

The success of the project depends on the engagement of distinct groups, categorised by their influence and interest in the results:

- **Primary Stakeholders:**
    - **Software Developers and Security Teams:** The direct beneficiaries who will leverage the framework to reduce operational risks and manual effort during dependency updates. Additionally, the open-source projects maintained by this community serve as a critical dataset for evaluating the framework against baseline tools.
    - **Academic Advisor:** Provides guidance on technical accuracy, methodological rigour, and adherence to academic standards.
- **Secondary Stakeholders:**

- **Contributors to the Maven Ecosystem:** They can benefit from a more confident decision-making process regarding dependency updates.
- **Academic Community:** Seeks advancements in approaches that can combine various techniques (such as static analysis and SMT) that provide higher precision and coverage of dependency management.

The project is structured to generate both technical artefacts and academic documentation. The primary technical deliverable is the Contextualised Risk-Based Framework and the respective support artefacts, which include the dataset of open-source projects and the experimental results. The academic deliverables comprise the Preparation Report and the Final Dissertation, along with their respective defence presentations.

Execution is bounded by the academic calendar. Institutional resources and open-source ecosystems cover the tooling, so there is no financial cost. Personal hardware, however, limits how deep the transitive analysis can reasonably go.

External services introduce a second constraint. The plan assumes that GitHub and Maven Central will not enforce restrictive rate limits during the work, and that the baseline tools stay operational through the evaluation.

B.2 Scope of Work

Following the DSR methodology, the project focuses on three core activities: designing, implementing, and evaluating a central artefact (the contextualised risk-based framework).

As illustrated in Figure 12, the work is organised into a Work Breakdown Structure (WBS) that breaks down the effort into six distinct phases, directly mapping to the DSR methodology. The work progresses from the theoretical definition of the problem and objectives (Preparation) to the technical construction of the artefact (Design and Implementation), to its application on a selected dataset (Demonstration) and the experimental realisation (Evaluation), culminating in the dissemination of the findings (Communication). The phases are subdivided into deliverables and tasks, which describe important activities.

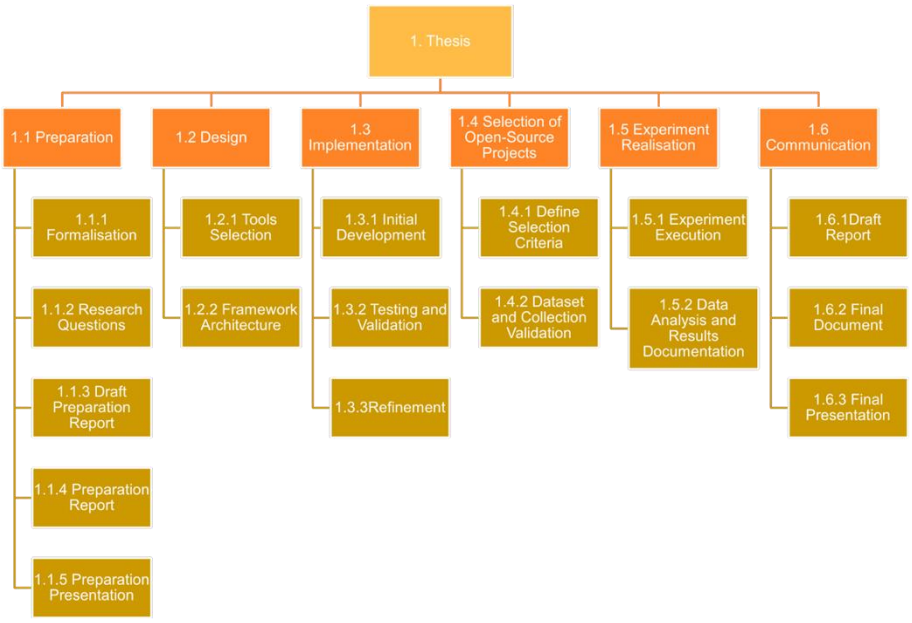


Figure 12 - WBS Project Deliveries

B.2.1 WBS

The WBS dictionary is a component that complements the graphical WBS by providing additional information about its elements. As presented in Table 25, the dictionary adds depth to each WBS element, defining its scope through comprehensive descriptions, verifiable acceptance criteria, and measurable progress indicators.

Table 25 - WBS Dictionary

WBS ID	Type	Description / Acceptance Criteria	Progress Criteria
1. Thesis	Project	Dissertation Project. Includes all work required to prepare, design, implement, demonstrate, evaluate and communicate the proposed framework.	Preparation – 25%. Design – 40%. Implementation – 55%. Selection of open-Source projects – 70%. Experiment realisation – 85%. Communication – 100%.
1.1 Preparation	Phase	Includes the formalisation of the problem and the review of the state of the art. This phase will only be concluded after the formal approval of its deliverables.	Formalisation delivered – 15%. Formalisation approved – 30%. Preparation document draft delivered – 45%.

WBS ID	Type	Description / Acceptance Criteria	Progress Criteria
			Preparation document draft reviewed – 60%. Preparation document finalised – 75%. Presentation document delivered – 90%. Preparation phase presented - 100%.
1.1.1 Formalisation	Deliverable	It should define the dissertation context and motivation, problem, objectives, and contributions.	Draft completed - 50%. Formalisation delivered and approved - 100%.
1.1.2 Research Questions	Deliverable	The research questions must be defined and approved by the advisor before proceeding to the next deliverable.	Research questions delivered – 50%. Research questions approved – 100%.
1.1.3 Draft Preparation Report	Deliverable	An initial version of the preparation document should be defined and delivered.	Background – 15%. State of the Art – 30%. Skills Management – 45%. Project Charter – 60%. WBS – 75%. Planning – 90%. Draft delivered – 100%.
1.1.3.1 Background	Task	Analyse and describe the necessary information to comprehend the state of the art.	Research performed - 50%. Background finalised - 100%.
1.1.3.2 State of the Art	Task	Systematic literature review on dependency management, static reachability analysis, and ecosystem stability metrics.	Selection of sources – 25%. Research methodology definition – 50% Research performed – 75%. Findings documented – 100%.
1.1.3.3 Skills Management	Task	Identify the necessary skills, perform a gap analysis against current knowledge, and establish a plan to acquire them.	Required skills – 25%. Gaps identified – 50%. Plan developed – 75%. Skills Management documented – 100%.
1.1.3.4 Project Charter	Task	Finalise the preliminary planning document. This includes identifying stakeholders, and defining the scope, constraints and assumptions.	Stakeholders identified – 35%. Constraints and assumptions defined – 70%. Project Charter documented – 100%.

WBS ID	Type	Description / Acceptance Criteria	Progress Criteria
1.1.3.5 WBS	Task	Decompose the project scope into a WBS aligned with the DSR methodology. Must include a graphical representation of WBS and a WBS dictionary.	Graphical representation drafted – 30%. WBS dictionary defined – 70%. WBS completed – 100%.
1.1.3.6 Planning	Task	Develop the integrated Project Schedule, Cost estimation, and Risk Management Plan. It should include a Gantt Chart for visual representation.	Milestones and deadlines defined – 25%. Risk assessment completed – 50%. Gantt Chart finalised – 75%. Planning completed and documented – 100%.
1.1.4 Preparation Report	Deliverable	Revise the draft document to ensure all information is accurate, well-structured, and aligned with the thesis goals. Deliver the revised document.	Document updated – 50%. Document delivered – 100%.
1.1.5 Preparation Presentation	Deliverable	Create a presentation of the preparation document and present it.	Presentation prepared – 50%. Presentation completed – 100%.
1.2 Design	Phase	Encompasses activities to select tools and define the framework architecture.	Tools selected – 30%. Framework architecture defined – 70%. Design documented and validated – 100%.
1.2.1 Tools Selection	Task	Evaluate and select the core tools and libraries for each framework component.	Candidates evaluated – 50%. Selection documented – 100%.
1.2.2 Framework Architecture	Deliverable	A high-level component architecture, data flows, and interfaces between modules should be defined and approved by the advisor.	Framework architecture drafted – 60%. Architecture approved – 100%.
1.3 Implementation	Phase	Implement an MVP of the framework, following the established design, with iterative testing and refinement of each module.	Initial version – 40%. Perform tests – 60%. Refine the tool based on results – 100%.
1.3.1 Initial Development	Task	Build the first functional version of the framework based on the design defined in Phase 1.2. The framework should process a	Development started – 25%. Core functionality implemented – 75%.

WBS ID	Type	Description / Acceptance Criteria	Progress Criteria
		basic test case and produce a risk output.	Initial version validated – 100%.
1.3.2 Testing and Validation	Task	Conduct a systematic testing of the MVP to identify defects and verify that the output is consistent with the expected behaviour. All identified defects should be documented and addressed.	Test plan defined – 25%. Tests executed – 75%. Defects resolved – 100%.
1.3.3 Refinement	Task	Iteratively improve the framework based on the results of the testing phase to address performance, accuracy, and usability issues.	Issues identified and prioritised – 30%. Improvements implemented – 70%. Refined version validated – 100%.
1.4 Selection of Open-Source Projects	Phase	Select a dataset of real-world open-source Maven projects to evaluate the framework, ensuring sufficient transitive complexity and known vulnerabilities.	Selection criteria defined – 25%. Candidate pool identified – 60%. Final dataset validated and documented – 100%.
1.4.1 Define Selection Criteria	Deliverable	Quantitative inclusion criteria (e.g., activity level, known CVEs) should be defined and approved to ensure the dataset is adequate for evaluation.	Criteria drafted – 50%. Criteria approved – 100%.
1.4.2 Dataset Collection and Validation	Task	Apply the defined criteria to identify and validate the final set of open-source projects.	Candidate projects identified – 50%. Final dataset gathered and documented – 100%.
1.5 Experiment Realisation	Phase	Perform the evaluation of the developed framework against baseline tools using the previously selected dataset and carry out a comparative analysis of the results.	Experiments executed – 50%. Evaluation analysis completed – 100%.
1.5.1 Experiment Execution	Task	Run the framework and baseline tools against all the projects in the dataset, recording the output metrics for each run.	Framework executions completed – 50%. Baseline executions completed – 100%.
1.5.2 Data Analysis and Results Documentation	Task	Perform a comparative analysis of the results and document the findings.	Analysis completed – 50%. Results documented – 100%.
1.6 Communication	Phase	Final documentation of the thesis, validation of results, and communication of findings.	Final draft delivered – 25%. Final draft approved – 50%.

WBS ID	Type	Description / Acceptance Criteria	Progress Criteria
			Final document delivered – 75%. Final presentation delivered – 90%. Phase presented – 100%.
1.6.1 Draft Report	Deliverable	A complete draft of the final dissertation document, integrating all chapters, experiment results, and conclusions. The draft must be submitted to the academic advisor for review.	Draft completed – 50%. Draft submitted for review – 100%.
1.6.2 Final Document	Deliverable	Create the revised dissertation document incorporating all feedback from the advisor review.	Revisions implemented – 50%. Final document submitted – 100%.
1.6.3 Final Presentation	Deliverable	Create a presentation document summarising all the dissertation findings and present it.	Presentation drafted – 40%. Presentation reviewed – 70%. Presentation completed – 100%.

B.3 Integrated Project Schedule

To ensure the project adheres to a structured timeline, key milestones and durations were defined utilising a Gantt chart. This tool maps tasks against a calendar, providing a visual representation of the start and finish dates for each project element. Figure 13 and Figure 14 illustrate the high-level timeline of the project, from the initial preparation phase to the final communication phase, while outlining key milestones.

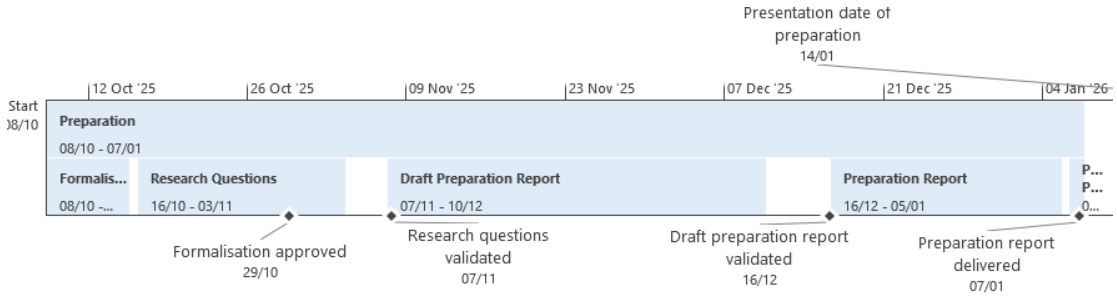


Figure 13 - Gantt Project Timeline - Part 1

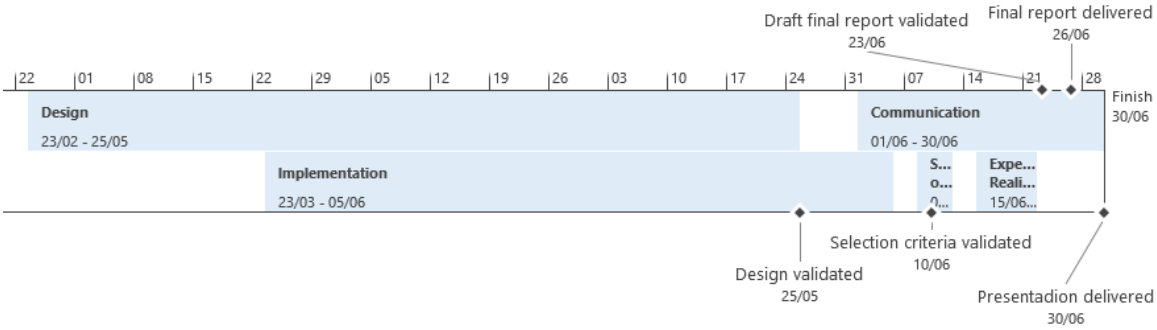


Figure 14 - Gantt Project Timeline - Part 2

While Figure 13 and Figure 14 provide a high-level view of the schedule, a more granular representation is required to reason about task-level dependencies and duration. Figure 15 details the specific task durations and dependencies, decomposed by phase.

Task Mode	WBS	Task Name	WBS Level	Duration	Start	Finish	Predecessors	Resource Names
	1	**1 Thesis**	**Project**	**190 days?**	**Wed 08/10/25**	**Tue 30/06/26**		**Student**
	1.1	**1.1 Preparation**	**Phase**	**66 days**	**Wed 08/10/25**	**Wed 07/01/26**		**Student**
	1.1.1	1.1.1 Formalisation	Deliverable	6 days	Wed 08/10/25	Wed 15/10/25		Student
	1.1.2	1.1.2 Research Questions	Deliverable	13 days	Thu 16/10/25	Mon 03/11/25	3	Student
	1.1.3	**1.1.3 Draft Preparation Report**	**Deliverable**	**24 days**	**Fri 07/11/25**	**Wed 10/12/25**	**4**	**Student**
	1.1.3.1	1.1.3.1 Background	Task	6 days	Fri 07/11/25	Fri 14/11/25		Student
	1.1.3.2	1.1.3.2 State of the Art	Task	11 days	Mon 17/11/25	Mon 01/12/25		Student
	1.1.3.3	1.1.3.3 Skills Management	Task	2 days	Tue 02/12/25	Wed 03/12/25		Student
	1.1.3.4	1.1.3.4 Project Charter	Task	2 days	Thu 04/12/25	Fri 05/12/25		Student
	1.1.3.5	1.1.3.5 WBS	Task	1 day	Mon 08/12/25	Mon 08/12/25		Student
	1.1.3.6	1.1.3.6 Planning	Task	2 days	Mon 08/12/25	Tue 09/12/25		Student
	1.1.4	**1.1.4 Preparation Report**	**Deliverable**	**15 days**	**Tue 16/12/25**	**Mon 05/01/26**	**5**	**Student**
	1.1.4.1	1.1.4.1 Review Advisor Feedback	Task	5 days	Tue 09/12/25	Mon 15/12/25		Student
	1.1.4.2	1.1.4.2 Implement Corrections	Task	15 days	Tue 16/12/25	Mon 05/01/26		Student
	1.1.5	1.1.5 Presentation Preparation	Deliverable	2 days	Tue 06/01/26	Wed 07/01/26	12	Student
	1.2	**1.2 Design**	**Phase**	**66 days?**	**Mon 23/02/26**	**Mon 25/05/26**	**2**	**Student;supervisor**
	1.2.1	1.2.1 Tools Selection	Task	20 days?	Mon 23/02/26	Fri 20/03/26		Student
	1.2.2	1.2.2 Framework Architecture	Deliverable	46 days?	Mon 23/03/26	Mon 25/05/26	17	Student
	1.3	**1.3 Implementation**	**Phase**	**55 days?**	**Mon 23/03/26**	**Fri 05/06/26**	**16**	**Student**
	1.3.1	1.3.1 Initial Development	Task	46 days?	Mon 23/03/26	Mon 25/05/26		Student
	1.3.2	1.3.2 Testing and Validation	Task	4 days?	Tue 26/05/26	Fri 29/05/26	20	Student
	1.3.3	1.3.3 Refinement	Task	5 days?	Mon 01/06/26	Fri 05/06/26	21	Student
	1.4	**1.4 Selection of Open-Source Projects**	**Phase**	**5 days?**	**Mon 08/06/26**	**Fri 12/06/26**	**19**	**Student;supervisor**
	1.4.1	1.4.1 Define Selection Criteria	Deliverable	2 days?	Mon 08/06/26	Tue 09/06/26		Student;supervisor
	1.4.2	1.4.2 Dataset Collection and Validation	Task	3 days?	Wed 10/06/26	Fri 12/06/26	24	Student
	1.5	**1.5 Experiment Realisation**	**Phase**	**6 days?**	**Mon 15/06/26**	**Mon 22/06/26**	**23**	**Student**
	1.5.1	1.5.1 Experiment Execution	Task	3 days?	Mon 15/06/26	Wed 17/06/26		Student
	1.5.2	1.5.2 Data Analysis and Results Documentation	Task	3 days?	Thu 18/06/26	Mon 22/06/26	27	Student
	1.6	**1.6 Communication**	**Phase**	**22 days?**	**Mon 01/06/26**	**Tue 30/06/26**	**23;19;16;26**	**Student**
	1.6.1	1.6.1 Draft Report	Deliverable	16 days?	Mon 01/06/26	Mon 22/06/26		Student
	1.6.2	1.6.2 Final Document	Deliverable	4 days?	Tue 23/06/26	Fri 26/06/26	30	Student
	1.6.3	1.6.3 Final presentation	Deliverable	2 days?	Mon 29/06/26	Tue 30/06/26	31	Student
	1.7	**1.7 Milestones**	**Milestones**	**174 days**	**Wed 29/10/25**	**Tue 30/06/26**		
	1.7.1	1.7.1 Formalisation approved	Milestones	0 days	Wed 29/10/25	Wed 29/10/25	3	
	1.7.2	1.7.2 Research questions validated	Milestones	0 days	Fri 07/11/25	Fri 07/11/25	4	
	1.7.3	1.7.3 Draft preparation report validated	Milestones	0 days	Tue 16/12/25	Tue 16/12/25	5	
	1.7.4	1.7.4 Preparation report and presentation delivered	Milestones	0 days	Wed 07/01/26	Wed 07/01/26	12;15	
	1.7.5	1.7.5 Presentation date of preparation	Milestones	0 days	Wed 14/01/26	Wed 14/01/26	37	
	1.7.6	1.7.6 Design validated	Milestones	0 days	Mon 25/05/26	Mon 25/05/26	16	
	1.7.7	1.7.7 Selection criteria validated	Milestones	0 days	Wed 10/06/26	Wed 10/06/26	24	
	1.7.8	1.7.8 Draft final report validated	Milestones	0 days	Tue 23/06/26	Tue 23/06/26	30	
	1.7.9	1.7.9 Final report delivered	Milestones	0 days	Fri 26/06/26	Fri 26/06/26	31	
	1.7.10	1.7.10 Presentadion delivered	Milestones	0 days	Tue 30/06/26	Tue 30/06/26	32	

Figure 15 - Detailed project schedule.

Figure 16 provides the complete graphical representation of the Gantt chart, integrating the task decomposition with the temporal axis and making visible the sequencing, parallelisms, and overlaps between activities.

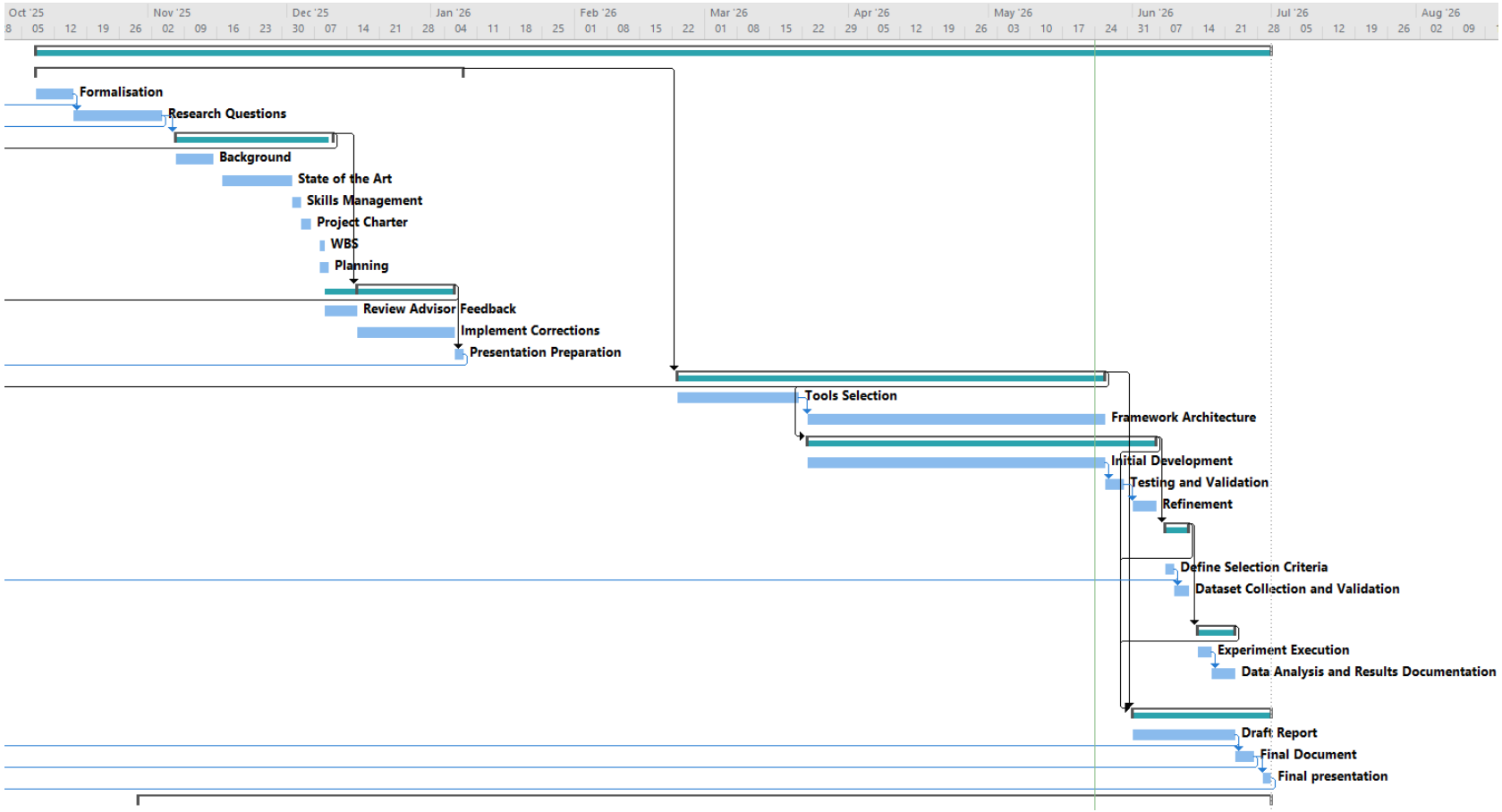


Figure 16 - Gantt chart providing the graphical timeline representation of the integrated project schedule

The project schedule was developed based on a hybrid planning strategy, recognising the distinction between short-term and long-term needs. For the short-term, a detailed planning approach was applied, breaking down the work into specific tasks to ensure clarity in execution. For the subsequent stages, the planning focused on defining the schedule through broader phases and key milestones, allowing for necessary adjustments as the project progresses.

B.4 Costs

The financial implications of this project are minimal, as it is designed to leverage open-source resources and existing infrastructure. All necessary software tools, including OpenJDK and the Maven build system, are available under open-source licences. The development environment relies on Visual Studio Code, which also includes an open-source licence. The experiments will be conducted on personal computing hardware. Access to scientific knowledge bases, such as IEEE Xplore and the ACM Digital Library, is provided through the institution's academic credentials.

B.5 Monitor and Control

To ensure the project adheres to the defined schedule and objectives, monitoring and control mechanisms were established. The monitoring of the project schedule is conducted using Microsoft Project, which serves as the primary tool for tracking the integrated schedule. This allows for a continuous comparison between the actual progress of deliverables and the planned baselines, facilitating the early identification of deviations and allowing necessary adjustments to the timeline.

At the strategic level, control is exercised through status meetings with the academic supervisor, serving as checkpoints to validate completed milestones, review the project trajectory, and define corrective actions for risks identified during development.

B.6 Risk Identification and Management

A comprehensive risk assessment was conducted to identify potential threats to the successful completion of the dissertation. The risk management process followed a structured approach involving identification, analysis, and response planning.

Each identified risk was assessed through a qualitative analysis based on two dimensions: Probability (P), reflecting the likelihood of occurrence, and Impact (I), measuring the severity of the consequences. Both dimensions were quantified on a scale from 1 (Low) to 5 (Very High). A Risk Score (PI) was derived by multiplying these factors to facilitate risk prioritisation, ensuring a focus on those that could compromise the project objectives or schedule. Below is a detailed analysis of the main identified risks:

Risk 1: Complexity of Static Reachability Analysis

- **Description:** The potential delay is caused by the complexity of integrating static analysis tools to generate Call Graphs.
- **Cause:** While tools exist, configuring them to handle transitive Maven dependencies and modern Java bytecode features is complex.
- **Effect:** Delays in the "Implementation" phase.
- **Risk Owner:** Author.
- **Probability / Impact:** High impact (4) due to technical difficulty, with a very high impact (5) on the project's success.
- **PI Score:** 20 (4 x 5).
- **Expected Result (No Action):** Without mitigation, the project would fail to meet objective 3, resulting in an incomplete framework implementation that lacks context awareness.
- **Response:** Mitigate. A dedicated period was defined in the project plan. If full CG analysis proves unfeasible, the framework will adopt a simpler "Method Signature Matching" approach.

Risk 2: SMT Solver Performance on Large Graphs

- **Description:** The risk that the SMT solver may experience timeouts or resource exhaustion when processing deep dependency trees.
- **Cause**: Encoding thousands of transitive constraints may result in resource exhaustion.
- **Effect:** The framework may become unresponsive or fail to generate recommendations for large real-world projects, invalidating the evaluation results.
- **Risk Owner:** Author.
- **Probability / Impact:** Moderate Probability (3), with a High Impact (4).
- **PI Score:** 12 (4 x 3).
- **Expected Result (No Action):** The tool would be unusable for large real-world projects, invalidating the answer to RQ2.
- **Response Strategy:** Mitigate. Implement a strict timeout mechanism for the solver. If the timeout is reached, the system fallbacks to a heuristic-based selection strategy (e.g., "nearest-wins") to ensure a result is returned.

Risk 3: Data Unavailability via External APIs

- **Description:** Inability to collect sufficient ecosystem stability metrics (e.g., adoption rates) due to external API restrictions.
- **Cause:** Maven Central could enforce strict rate limits or experience downtime during the data collection phase.

- **Effect**: Incomplete datasets would prevent the calculation of a “Stability Score”, compromising the accurate quantification of the operational risk associated with dependency updates.
- **Risk Owner**: Author.
- **Probability / Impact**: Moderate Probability (3), with a Very High (5) Impact.
- **PI Score**: 15 (5 x 3).
- **Expected Result (No Action)**: The thesis would be compromised regarding RQ1, as the stability metrics are a core contribution.
- **Response Strategy**: Mitigate. Implement a caching mechanism for API requests. Additionally, a static snapshot of the ecosystem metadata could be downloaded at the start of the project to ensure offline capability.

Risk 4: Integration Complexity Between Framework Components

- **Description:** The individual components of the framework may fail to integrate correctly, leading to incompatibilities or cascading failures.
- **Cause:** Each component is developed in isolation and relies on distinct inputs and outputs. Integration issues are often only discovered when components are first combined during the testing phase.
- **Effect**: Significant rework in the testing task (1.3.2), delaying the Implementation Phase and potentially affecting the evaluation timeline.
- **Risk Owner**: Author.
- **Probability / Impact**: High Probability (4), with a Very High (5) Impact.
- **PI Score**: 20 (4 x 5).
- **Expected Result (No Action)**: The framework fails to operate as a cohesive pipeline, preventing any evaluation from being performed.
- **Response Strategy**: Mitigate. Define clear component interfaces and shared data formats during the Design Phase (1.2) before implementation begins. Perform integration tests incrementally as each component is completed to detect issues early.

Risk 5: Inadequate Evaluation Dataset

- **Description:** The risk that selected open-source projects for the experiment phase do not contain a sufficient number of reachable vulnerabilities or breaking changes to properly evaluate the framework.
- **Cause:** Selected projects lack sufficient transitive complexity, which would fail to trigger the deep graph analysis logic of the framework.
- **Effect**: Evaluation results would be inconclusive or statistically insignificant.
- **Risk Owner**: Author.
- **Probability / Impact**: Moderate Probability (3), with a Very High (5) Impact.
- **PI Score**: 15 (5 x 3).

- **Expected Result (No Action)**: Failure to demonstrate a measurable improvement over existing baselines.
- **Response Strategy**: Mitigate. Define quantitative inclusion criteria to filter the pool of candidates, ensuring only complex, active projects are evaluated.

Risk 6: Personal Hardware Limitations During Experimentation

- **Description:** The processing power of the available personal hardware may be insufficient to execute the framework on large real-world projects within an acceptable time frame.
- **Cause:** Acknowledged in the project charter as a known technical constraint. The analysis performed by the framework is computationally intensive and may not scale well on standard hardware.
- **Effect**: Experiments may take significantly longer than planned, causing schedule overrun. The analytical depth may need to be reduced, which could limit the validity of the results.
- **Risk Owner**: Author.
- **Probability / Impact**: Moderate Probability (3), with a High (4) Impact.
- **PI Score**: 12 (3 x 4).
- **Expected Result (No Action)**: The experimental phase exceeds the available time budget, resulting in incomplete results or a reduced dataset.
- **Response Strategy**: Mitigate. Implement a configurable analysis depth threshold in the development phase (1.3.1). Profile performance benchmarks early during the testing phase (1.3.2) and adjust the threshold based on observed hardware constraints before the full experimental run.

Risk 7: Unsatisfactory Framework Accuracy

- **Description:** The framework may produce a high proportion of incorrect assessments (false positives or false negatives), reducing its practical utility and undermining evaluation results.
- **Cause:** The composite risk score combines multiple factors – such as vulnerability severity and reachability context – each contributing with a relative weight. If those weights are not properly balanced, the final score may systematically over-penalise or under-penalise certain dependencies regardless of the accuracy of the individual components. Additionally, over-approximations in the reachability analysis may flag methods as reachable when they are not, further inflating risk scores for safe dependencies.
- **Effect**: The framework would offer no measurable improvement over existing tools in terms of precision, prejudicing its core contribution and producing recommendations that developers cannot trust.
- **Risk Owner**: Author.

- **Probability / Impact**: Moderate Probability (3), with a High (4) Impact.
- **PI Score**: 12 (3 x 4).
- **Expected Result (No Action)**: The framework fails to demonstrate a meaningful improvement in precision compared to baseline tools.
- **Response Strategy**: Mitigate. Validate the framework output against a set of manually classified test cases during the refinement task (1.3.3), where the expected risks are known in advance. Conduct an analysis to measure how changes to individual factor weights affect the final score and refine the weights until the output is consistent with the expected results.

Risk 8: Schedule Overrun Due to Underestimated Technical Complexity

- **Description:** The overall project schedule may be exceeded due to the cumulative effect of unforeseen technical difficulties, particularly during the Implementation Phase.
- **Cause:** The project addresses a combination of techniques with limited prior implementation experience, making accurate effort estimation inherently uncertain.
- **Effect**: Insufficient time to complete the Experiment Realisation (Phase 1.5) or Communication (Phase 1.6) within the academic year.
- **Risk Owner**: Author.
- **Probability / Impact**: Moderate Probability (3), with a High Impact (4).
- **PI Score**: 12 (3 x 4).
- **Expected Result (No Action)**: The final dissertation is delivered incomplete or with a reduced experimental scope.
- **Response Strategy**: Accept and mitigate. Review and adjust the schedule at each advisor checkpoint. If the Implementation Phase is significantly delayed, the scope of the evaluation dataset will be reduced as a contingency measure to protect the Communication Phase deadline.